RMIT UNIVERSITY

ELLY DEFENCE - AUTONOMOUS ASSISTED PATCHING AGENT (ELECTRAWIRELESS)

PROJECT REPORT

Product-owner: ElectraWireless

Supervisor: Dr. Attiq-Ur Rehman

Authors : **Kanishk Sharma, s3944265**
Sanskar Pokharel, s4160991
Hemanth Petlu, s4112949
Anthony MacKenzi Fernando, s4129857

DOCUMENT CONTROL

Version #	Implemented By	Implementation Date	Reviewed By	Approval Date	Reason
1	Sanskar Pokharel	23 Oct 2026	Kanishk	23 Oct 2026	Project Report for submission
2	Kanishk Sharma	23 Oct 2026	Hemanth	23 Oct 2026	Final report for submission
3	Hemanth Petlu	23 Oct 2026	Group 4	23 Oct 2026	Final Project report for submission
4					
5					

TABLE OF CONTENTS

1 PROJECT CHARTER 6
1.1 Project Summary 6
1.2 Project Sponsor / Product Owners 6
1.3 Stakeholders and End Users 7
1.4 Appointment of Scrum Master 7
1.5 Project Team Members 8
1.6 Project Methodology and Approach 9
1.7 Project Governance 10
1.8 Project Scope & Deliverables 11

2 ISSUE AND RISK REGISTER 16
2.1 Issue Register 16
2.2 Risk Register 18

3 PRODUCT BACKLOG 20
3.1 User Stories and Backlog Items 20
3.2 Sprints and Velocity 23
3.3 Story Points per Person 23
3.4 Burn-down 24

4 TECHNICAL SOLUTION DESIGN 25
4.1 Introduction 25
4.2 Technical Environment 28
4.3 Overall Architecture 30
4.4 System Architecture 33
4.5 Database Architecture 42
4.6 Implementation Instructions 43
4.7 Non-functional specifications 44
4.8 Summary of test results 44
4.9 Known Issues & Risks 45
4.10 Other Considerations 45
4.11 Appendix 46

5 TEST PLANNING AND RESULTS 48
5.1 Version History 48
5.2 Introduction 48
5.3 Test Items 49
5.4 Features to be Tested 49
5.5 Features not to be Tested 49
5.6 Approach 50
5.7 Item Pass and Fail Criteria 50
5.8 Test Deliverables 57
5.9 Testing Tasks 58
5.10 Environmental Needs 59
5.11 Responsibilities 59
5.12 Schedule 59
5.13 Risks and Contingencies 59
5.14 Approvals 60

6 DEVELOPMENT GUIDE 61

6.1 Project Summary 61
6.2 Setup client and developer machine 62
6.3 Code and Comment Standard 65
6.4 Version Control Policy 66
6.5 Getting Started 67

7 DEPLOYMENT INSTRUCTIONS 74
7.1 Recommended runner and dashboard route 74
7.2 Inbox and queue behaviour 74
7.3 Manual approval and component deployment 75
7.4 Dashboard acceptance check 75

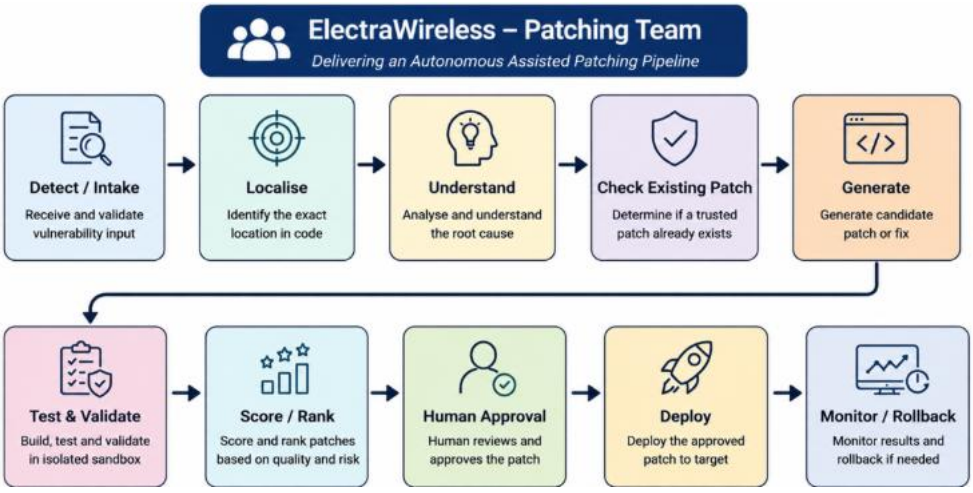
8 OTHER CONSIDERATIONS 76
8.1 Source Code Submitted for Audit 76
8.2 Confidentiality and Privacy 76
8.3 Responsible Use of AI 76
8.4 Handover 76

9 REFERENCES 77

1 PROJECT CHARTER

1.1 PROJECT SUMMARY

The Elly Defence Autonomous Assisted Patching Agent is an ElectraWireless cybersecurity proof of concept for assisted software-vulnerability remediation. Its objectives were to turn structured findings into candidate fixes, verify those fixes in an isolated environment, and provide evidence for a human release decision. By Week 12, the team had implemented the modular workflow and demonstrated a credential finding through validation, approval and deployment to a test copy. Evaluation used Juliet and CVEfixes examples, controlled IoTGoat cases and recorded integration runs.



The vendor requirements informed the workstream, including Asset Discovery, Vulnerability Scanning, Automated Deployment and Compliance Reporting. Asset discovery and vulnerability scanning were primarily addressed through the separate Vulnerability Analysis Team and the agreed scanner-to-patching handoff. Within the Patching Agent workstream, the team concentrated on secure remediation decisions, validation evidence, human approval, traceability and controlled deployment demonstrations. Privileged access, network communication, staggered rollout and organisational controls were considered as design requirements, but full enterprise implementation and compliance certification were not established.

The initial phase delivered representative Juliet and CVEfixes examples, separate IoTGoat validation cases, a Docker-based sandbox and independently testable patching modules. Later work connected stage adapters and scanner findings through the Orchestrator, while exposing real limitations in source resolution, multi-file upstream fixes, model availability and case-specific test profiles.

The demonstrated flow preserves finding_id through source resolution, localisation, explanation, patch lookup, candidate generation, validation, security checks, ranking and human review. The final run demonstrated one credential finding through approval and deployment to a test copy with a passing health check. Separate cases demonstrated rejection and rollback.

From approximately Week 6, the team coordinated with the Vulnerability Analysis workstream on scanner reports, finding identity, source references, stage interfaces and integration tests. The final run outcomes and remaining constraints are recorded in the methodology and deliverables sections.

1.2 PROJECT SPONSOR / PRODUCT OWNERS

ElectraWireless is the host organisation and project sponsor, providing the industry context, requirements and stakeholder oversight. Jose and Larry were early ElectraWireless contacts and left during the semester. Ongoing industry coordination continued with Shivam and Sarvesh.

Shivam and the Vendor Team provided client requirements, technical feedback and review of project deliverables. Their input informed the intended use, design priorities and operational expectations of the Patching Agent.

Dr Attiq Ur Rehman provided academic mentoring, reviewed progress and supported the team in clarifying project expectations and the cybersecurity relevance of the work.

1.3 STAKEHOLDERS AND END USERS

ElectraWireless - Industry Sponsor

Provides the project environment, industry context, technical direction and oversight for the Elly Defence workstream.

Shivam / Vendor Team - Client Stakeholder

Represents the vendor/client team for whom the project is being undertaken. Provides requirements and feedback and has a direct interest in the usefulness, security and practicality of the Patching Agent outcome. Helps resolve requirement questions and reviews or accepts relevant project outcomes.

Sarvesh - ElectraWireless Industry Stakeholder

Provides industry coordination, requirement clarification and support with demonstrations and handover.

RMIT Mentor - Attiq Ur Rehman

Provides mentoring and project guidance, including the identified Patching Agent functions of Asset Discovery, Vulnerability Scanning, Automated Deployment and Compliance Reporting, together with cybersecurity considerations such as Privileged Access & Network Communication.

Patching Agent Team

Researches, designs, develops, tests and documents the autonomous assisted vulnerability-remediation pipeline.

Vulnerability Analysis Team

Develops the upstream vulnerability-analysis workstream. From approximately Week 6 onward, its structured vulnerability findings became inputs to the Patching Agent through the shared Orchestrator and finding record contract.

Future Security Analysts / Elly Defence Users

Potential users of the workflow who require clear vulnerability context, patch evidence, validation results, traceability and appropriate human decision points before remediation is applied.

1.4 APPOINTMENT OF SCRUM MASTER

Kanishk is appointed as the Scrum Master and project coordination lead for the Patching Agent Team. In this role, Kanishk supports the team's iterative delivery process while continuing to contribute technically to the Patching Agent workstream.

The Scrum Master's responsibilities include coordinating regular team meetings; helping maintain visibility of assigned tasks, priorities and dependencies; following up agreed actions; identifying and communicating blockers; supporting alignment between the Patching Agent Team and relevant stakeholders; and escalating issues that may affect scope, schedule, security or delivery to the appropriate ElectraWireless, vendor/client or RMIT contact.

Kanishk also supported the transition from initial Patching Agent development to orchestration with the Vulnerability Analysis Team from approximately Week 6. This included coordinating discussions, tracking cross-team actions and making integration dependencies visible. The Scrum

Master facilitated coordination and transparency, while technical and stakeholder decisions remained with the relevant team members and stakeholders.

1.5 PROJECT TEAM MEMBERS

The project team members and their responsibilities are outlined below. Roles developed throughout the project as the work expanded from individual modules into testing and integration.

1.5.1 Kanishk - Scrum Master / Cyber Security Intern, Patching Agent Team

Coordinated team delivery, progress updates, stakeholder communication and integration priorities. Maintained the Project Charter and helped document the team's scope, completed work, outstanding requirements and handover arrangements.

Contributed to Task 3 dataset preparation and validation, Task 4 sandbox testing, and Task 8 existing-patch lookup and compatibility checks. This included checking data preservation, invalid inputs, duplicate records, source and version compatibility, and safe handling of failures. Prepared controlled IoTGoat examples and repeatable tests with supporting evidence.

Developed Task 10's patch-validation pipeline and Task 12's scoring and ranking module, and integrated Task 11 exploit and fuzz checks into validation. Worked on Task 9 compatibility changes so generated candidates could be passed into validation with applicable diffs, finding identifiers, source information and checksums. Added handling for duplicate candidates, partial generation results and cross-platform patch-file issues.

Contributed to Orchestrator integration, connecting patching stages and testing the scanner-to-patching handoff. Prepared demonstrations of candidate validation, human approval, component deployment, health checks and rollback. Produced test evidence, demonstration recordings, the Development Guide and project reporting, while distinguishing controlled component results from outstanding full-firmware testing.

Primary documentation responsibility: Project Charter.

1.5.2 Hemanth - Cyber Security Intern, Patching Agent Team

Contributed to Task 2 by establishing the Patching Engine's initial architecture and module structure, providing a foundation for subsequent development. Developed Task 6 vulnerability localisation, which identifies the relevant source location associated with a finding. Worked on localisation inputs, model interaction and test evidence to support the following vulnerability-understanding stage.

Led Orchestrator coordination from the patching side, aligning finding records, source references and stage handoffs with the Vulnerability team. Developed the orchestration pipeline and source-resolution handling, synchronised engine code between repositories, and connected the patching stages through adapters.

Implemented lifecycle handling and reason codes so stopped findings identify the responsible team and next action. Worked on the runner and dashboard connection, integration tests and the full 49-finding run, investigating blockers and aligning the workflow with the shared record contract.

Maintained the Product Backlog and supported prioritisation, dependency tracking and progress reporting. Prepared the Technical Solution Design, Issue Register, release instructions and handover documentation.

Primary documentation responsibilities: Technical Solution Design, Product Backlog, Issue Register, readme and handover documentation.

1.5.3 Anthony - Cyber Security Intern, Patching Agent Team

Developed Task 7 vulnerability understanding, which uses the localisation output to explain the defect and provide context for patch selection or generation. Updated the handoff to use Task 6's LLM localisation results and prepared corresponding input and understanding-result files.

Tested the Task 7 workflow, checked consistency between supplied locations and generated results, and prepared evidence for review. Supported integration discussions concerning the module's input/output requirements and its connection to the wider patching workflow.

Developed Task 14 deployment-related functionality, including health-check and rollback behaviour, and prepared associated tests and documentation. Worked with the team to align deployment with the preceding validation and approval stages. Demonstrated results should be understood within the supported component-testing environment rather than as proof of production or full-firmware deployment.

Maintained sprint planning documents, meeting agendas and minutes, recording goals, responsibilities, decisions and follow-up actions.

Primary documentation responsibilities: Sprint Planning Documents, Meeting Agendas and Minutes.

1.5.4 Sanskar - Cyber Security Intern, Patching Agent Team

Developed Task 5 intake functionality for preparing vulnerability information for the patching workflow. Worked with the dataset inputs used during development and participated in discussions about aligning these inputs with scanner findings and the shared finding identifier.

Developed Task 9 LLM-based patch generation, including the batch workflow used to produce candidate fixes. Prepared generation tests and results and collaborated on the handoff requirements needed for candidates to enter Task 10 validation. Subsequent compatibility work added stronger candidate metadata, applicable diffs and rejection handling while preserving the batch workflow.

Developed Task 13 human-approval functionality and worked through its testing. Contributed to presenting patch information for review and aligning the approval process with the agreed Orchestrator-controlled gate.

Also contributed to user-interface work and maintained documentation covering project risks and the infrastructure supporting development and execution.

Primary documentation responsibilities: Risk Register and Infrastructure Summary.

1.5.5 All Team Members

All team members participated in research, development, testing, integration discussions, evidence preparation and project reporting. Members reviewed dependencies between tasks, investigated failures, prepared demonstrations and contributed to the final handover.

Individual contributions are supported by the backlog, commit history, meeting minutes and timesheets. Module development and integration involved collaboration across the team.

1.6 PROJECT METHODOLOGY AND APPROACH

The team used an iterative, Agile-style approach appropriate to a research and proof-of-concept cybersecurity project. Work was divided into tasks and reviewed through regular coordination meetings, testing cycles and stakeholder feedback. Dependencies between the scanner, source resolver, model services, patching modules and Orchestrator were tracked and addressed incrementally.

Week 12 project status

Tasks 1 and 2 established the development environment and system architecture. Task 3 prepared 30 Juliet and six CVEfixes examples, with separate controlled IoTGoat cases. Task 4 established the Docker sandbox and test runners.

Tasks 5-9 implemented structured finding intake and handoff, source localisation, vulnerability understanding, existing-patch lookup and candidate patch generation.

Task 10 validated candidate patches through application, build and normal-function checks. Task 11 added exploit replay, security assertions and bounded fuzz testing where supported. Task 12 scored and ranked eligible candidates. Tasks 13 and 14 supported human approval and controlled component deployment, including health checks and rollback.

Testing covered individual module functions and selected integrated workflows. Implementation, integration and successful validation were tracked separately because results varied by test case and repository version. These results demonstrate supported prototype capabilities, not full-firmware or production readiness.

The final integration run on 10 October 2026 processed all 49 findings in the Vulnerability Analysis Team report. Every finding had a recorded outcome, reason code and responsible team. One hardcoded credential finding progressed from the scanner report to a tested patch, human approval and deployment to a test copy with a passing health check. Task 10 blocked an unsafe model patch for the second credential finding before approval. Ten findings awaited C-code validation settings or review of a model answer. The remaining 37 were returned to the Vulnerability team: 30 lacked a fix commit and seven lacked a single source file to patch. Separate demonstrations exercised rejection and rollback after a deliberately failing health check.

Development environment and repository

Development used Python, Git/GitHub, Docker, structured JSON contracts and case-specific validation profiles. Elly-Defense contains the patching work; Elly-Defense-Orchestration coordinates the connected workflow. Final runs used Qwen3-Coder locally through Ollama, keeping firmware and source code on the laptop for those runs. OpenRouter was used during development and remains an optional alternative for the client. Source references, finding_id continuity, saved evidence and release instructions support reproducibility. Release packaging and repository differences are recorded in the release documentation.

The initial Task 2 design established a shared PatchJob contract, modular components, centralised configuration and a model-client interface. The connected Orchestrator uses finding records keyed by finding_id, with stage adapters translating records into each module's inputs and returning results to the shared workflow. The Technical Solution Design and record_contract.md describe these interfaces in detail.

Delivery phases and closure

Weeks 1-6 concentrated on architecture, datasets, sandbox and independently testable modules. The later phase concentrated on scanner handoff, source resolution, stage adapters, integration, validation, approval and deployment demonstrations, evaluation and documentation. At Week 12, the team completed a controlled proof-of-concept demonstration and supporting evidence, while retaining clearly identified limits in general source resolution, multi-file fixes, external model reliability and deployment scope.

Team coordination

WhatsApp supported daily coordination, Trello tracked tasks and GitHub pull requests supported code review. Meeting minutes, issue records and timesheets documented decisions and contributions.

1.7 PROJECT GOVERNANCE

The following controls supported delivery and review:

The project was governed through ElectraWireless stakeholder oversight, vendor/client input, RMIT mentoring, team planning and documented project controls.

Decision-making and reporting

Day-to-day technical decisions were discussed within the team. Material changes to scope, integration responsibilities, security controls and deliverables were escalated to the relevant ElectraWireless or academic contacts. Vendor/client requirements informed the proof-of-concept priorities; this charter does not assert formal client acceptance or production authorisation.

Communication and records

WhatsApp, meeting minutes, Trello and GitHub pull requests were used for coordination, task tracking and code review. Test reports and structured run evidence record successes, failures and skipped checks.

Milestones and close-out

The team progressed from setup and design through dataset preparation, isolated component tests, scanner handoff, integration and controlled demonstration. The final handover comprises source code, documented interfaces, test evidence, the Development Guide and project reporting. Outstanding integration and operational limitations are recorded rather than treated as completed features.

Risk and issue management

Key risks included incorrect or incomplete LLM output, regressions, unsafe execution, missing source provenance, multi-file patches, model timeouts, insufficient test profiles, deployment privileges and schedule/integration dependencies. These were addressed through isolation, bounded execution, evidence checks, fail-closed decisions, human review and issue reporting; residual risks remain.

Change and scope control

The project prioritised a defensible, reproducible proof of concept over unrestricted feature expansion. New functionality or broader deployment would require further validation and stakeholder approval.

Security and approval controls

Generated patches are untrusted until verified against the intended source and defined functional/security tests. Missing evidence, model errors and failed tests are not treated as passes. Approval is required before controlled release; rollback demonstrations apply to staged components, not complete production firmware fleets.

1.8 PROJECT SCOPE & DELIVERABLES

The final scope covers research, design, implementation, controlled testing, integration and evaluation of the Elly Defence assisted patching proof of concept. Scope is assessed against demonstrated outcomes and the available evidence, not only the initial planned architecture.

IN SCOPE - PHASE 1: PATCHING AGENT DEVELOPMENT (WEEKS 1-6; EXTENDED REFINEMENT THROUGH WEEK 12)

1. Research vulnerability remediation, secure program repair, CVE/CWE context and LLM-assisted patching.

2. Prepare representative Juliet/CVEfixes datasets and separate IoTGoat source/credential test fixtures.

3. Develop and test a modular Python/Git/Docker pipeline with controlled sandbox execution.

4. Establish structured vulnerability/finding identity, provenance and module handoff contracts.

5. Localise and explain vulnerabilities using available source context and configured model services.

6. Check existing compatible patches before requesting a generated candidate.

7. Generate candidate patches, retaining errors and partial outputs as evidence rather than assuming success.

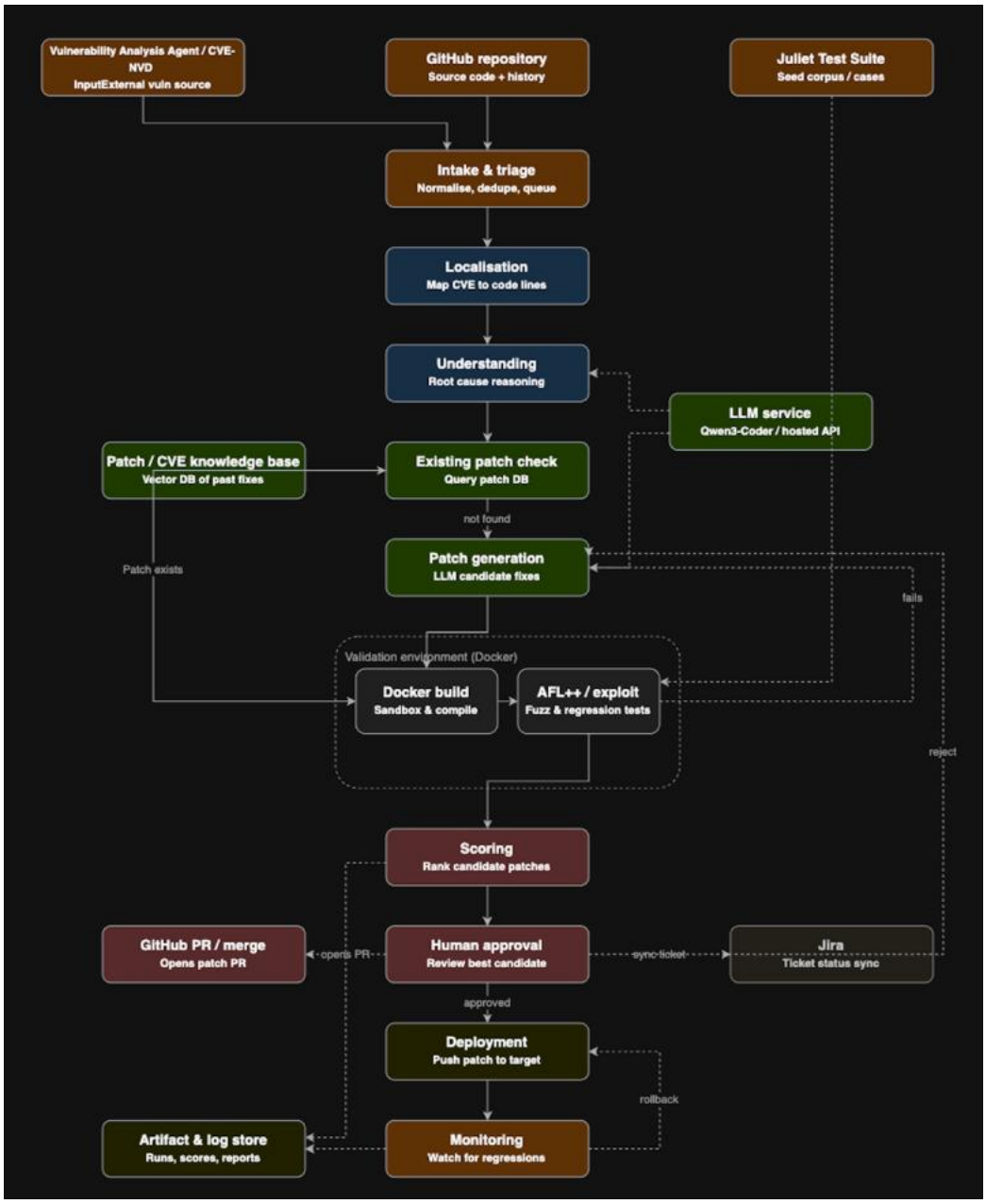
8. Apply candidates to controlled source copies and run build, normal-behaviour and security checks, including exploit assertions or fuzzing where supported.

9. Filter and rank candidates that satisfy mandatory validation gates.

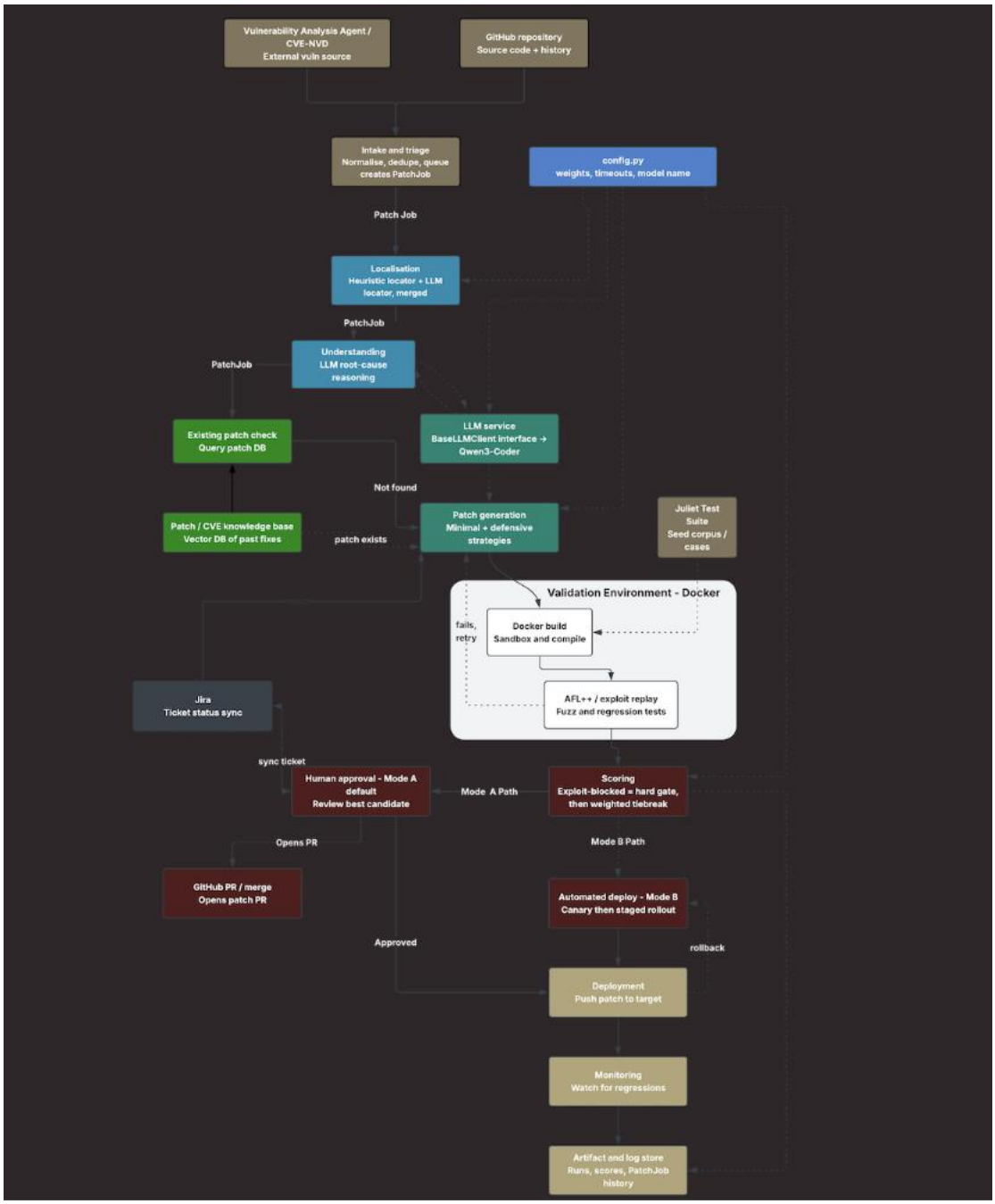
10. Demonstrate human approval/rejection, audit trail, controlled component staging, health checking and rollback.

11. Document repeatable commands, test results, security limitations and integration assumptions.

12. Address vendor functions through the combined workstreams at prototype level, without claiming complete asset management, fleet deployment or compliance certification.



Patching Agent Architecture - Version 1 (Initial High-Level Design)



Patching Agent Architecture - Version 2 (Assignment 1 Design)
The current as-built architecture is Version 3 in the Technical Solution Design, Figure 4.

IN SCOPE - PHASE 2: ORCHESTRATION WITH VULNERABILITY ANALYSIS (APPROXIMATELY WEEKS 6-12)

1. Review the two workstreams and agree the scanner-report-to-patching handoff.

2. Preserve scanner finding_id, nullable CVE, source references and original finding evidence through the Orchestrator.

3. Integrate source resolution and stage adapters for localisation, understanding, patch lookup, generation, validation and ranking.

4. Record failures, skipped stages and review-required outcomes without misrepresenting them as patch success.

5. Demonstrate a credential finding through validation, human approval and deployment to a test copy, with separate rejection and rollback demonstrations; record outcomes for the full scanner report.

OUT OF SCOPE - FINAL BOUNDARIES

• Unauthorised scanning, exploitation or remediation of public, customer or third-party systems.

• Unattended production deployment of AI-generated patches or removal of mandatory human approval.

• A production firmware flash, device reboot, live fleet rollout or enterprise rollback guarantee.

• Universal source resolution, multi-file patch handling and successful remediation for every scanner finding, CVE, language or software ecosystem.

• A fully production-ready commercial service, enterprise identity/privilege management or compliance certification.

• Training a foundation-scale model from scratch or guaranteeing deterministic model responses.

• Treating mocked tests, isolated component checks or a successful single-case demo as proof of general production security.

KEY DELIVERABLES - WEEK 12 POSITION

1. Research and technical findings: completed at prototype/reporting level.

2. Architecture, data contracts and design rationale: produced and refined; separate checkout differences remain documented.

3. Juliet/CVEfixes dataset subset and IoTGoat fixtures: prepared and tested as controlled examples.

4. Structured scanner-to-patching handoff: implemented in integration checkouts, with source-selection limitations recorded.

5. Localisation, explanation, existing-patch lookup and candidate generation: implemented and demonstrated on supported cases, with stopped outcomes recorded.

6. Docker/component validation with functional and security checks: implemented; effectiveness depends on case-specific trusted test profiles.

7. Eligibility filtering and patch ranking: implemented for candidates passing defined gates.

8. Human approval/rejection and audit trail: demonstrated separately from unattended pipeline execution.

9. Controlled component staging, health check and rollback: demonstrated; full firmware/fleet deployment remains out of scope.

10. Orchestrator interfaces and integration testing: developed and exercised, with unresolved scanner/source/model constraints.

11. Reproducible test outputs, failure records and demonstration evidence: prepared for evaluation.

12. Final Development Guide, project reporting and supporting technical documentation: prepared for Week 12 handover.

13. Final integrated proof-of-concept demonstration: achieved for one credential finding through human approval and deployment to a test copy; outcomes for all 49 findings are recorded.

14. Task 15 Documentation, Reproducibility and Packaging: prepared release instructions, project documentation and supporting evidence for handover, with outstanding limitations recorded.

Testing covered individual module functions, selected CVE-based cases and controlled IoTGoat examples, including invalid inputs, failure handling and integration checks. The planned full end-to-end evaluation of 10-20 SEC-bench or CVE-Bench cases was not completed. Broader benchmark evaluation and measured detection, localisation and safe-patching success rates remain future work.

TIMING AND QUALITY - FINAL REVIEW

By Week 12, the project had progressed from independently testable modules to a connected proof of concept. The final demonstration used controlled source material, explicit validation profiles and a human release decision. Remaining work includes broader source resolution, multi-file fixes, additional validation profiles and deployment beyond test copies.

Quality was evaluated through reproducibility, traceability and observed validation evidence, informed by the NIST Secure Software Development Framework (SSDF), SP 800-218, and applicable secure testing principles. Candidate generation or compilation alone does not constitute successful remediation. A candidate must apply to the intended source, preserve defined normal behaviour, satisfy relevant security assertions and remain subject to human approval before controlled release. Failed, skipped, incomplete or uncertain checks remain visible in the results. No formal SSDF compliance certification or production safety assurance is claimed.

2 ISSUE AND RISK REGISTER

2.1 ISSUE REGISTER

No	Description	Impact	Date Raised	Phase Raised	Allocated to	Resolution Strategies / Actions	Date Closed
1	Finding IDs changed when the firmware path was typed differently, so one finding could appear twice.	High	Week 9 (21-27 Sep 2026)	Testing	Hemanth (found), Alan - Vulnerability Analysis Team (fixed)	IDs now hash the firmware file's content, not its path. Store rebuilt on the new IDs, old store archived.	29 Sep 2026
2	Pipeline could set a finding to 'matched_with_patch' when no patch was returned.	High	23 Sep 2026	Development	Hemanth	Safety check added: the status change is refused when no patch exists.	23 Sep 2026
3	Task 9 errors: file path, indentation, number of candidates, patch search and timeout.	Medium	9 Sep 2026	Testing	Sanskar	Fixed on the Task 9 branch (9-22 Sep) and merged into main with PR #87.	6 Oct 2026
4	Pipeline did not detect stages built in new folders.	Medium	25 Sep 2026	Development	Hemanth	Stages detected by folder, stages 10-12 added, 13-14 shown but never run by the pipeline.	25 Sep 2026
5	Task 6 contained machine-specific paths and did not record the prompt version.	Medium	26 Sep 2026	Development	Hemanth	Paths replaced with a --repo argument, prompt version recorded. PR #75.	26 Sep 2026
6	Patch-agent copy in the Orchestration repository was out of date.	Medium	25 Sep 2026	Implementation	Hemanth	Copy synced from Elly-Defense main and checked file by file. Synced again after the Task 7 to 14 merges (10 Oct).	26 Sep 2026
7	Task 14 file was named pipeline.py and clashed with the orchestration pipeline.	Low	26 Sep 2026	Development	Anthony	Moved to the pipeline/deploy package and merged into main with PR #73.	7 Oct 2026
8	dnsmasq records carried a landing-page link that could not be downloaded.	Medium	28 Sep 2026	Analysis	Rahul - Vulnerability Analysis Team, Hemanth (follow-up)	Git link fixed on Elly-Defense main (30 Sep) and synced (6 Oct). The Vulnerability Analysis Team issued a new report on 8 Oct, loaded on 9 Oct and the dnsmasq source now downloads.	9 Oct 2026
9	Firmware image mismatch: our copy (SHA-256 b9847dd4...) differed from the image the store was built from (13f9a540...).	High	Week 10 (28 Sep - 4 Oct 2026)	Testing	Hemanth, Alan - Vulnerability Analysis Team	Vulnerability Analysis Team committed the agreed image (gzipped) to the Orchestration repo (f285290). SHA-256 checked, extracted into extracted_firmware/asset_13f9a5407a85/, both credential findings now resolve.	5 Oct 2026
10	Both credential findings pointed at the same lines (1 and 8) of the same file.	Medium	26 Sep 2026	Analysis	Hemanth, Patching team	Each finding is now narrowed to its own line (iotgoatuser line 8, root line 1) and gets its own patch. Integration test INT-06 checks it.	10 Oct 2026
11	Component findings needed the source code at the exact version before they could be patched.	High	3 Oct 2026	Testing	Hemanth	fetch_sources.py downloads busybox 1_28_4, dropbear DROPBEAR_2017.75 and dnsmasq v2.73 into a local cache. Committed 8 Oct.	8 Oct 2026
12	Tasks 7, 9 and 10-14 were pushed on branches but not merged into main.	High	3 Oct 2026	Implementation	All members	Reviewed and merged into Elly-Defense main between 6 and 10 Oct (PRs #84, #85, #87, #88, #29, #73, #91) and synced into the Orchestration repo.	10 Oct 2026

No	Description	Impact	Date Raised	Phase Raised	Allocated to	Resolution Strategies / Actions	Date Closed
13	Integration tests INT-01 and INT-06 could not run.	Medium	25 Sep 2026	Testing	Hemanth	Run on the pipeline laptop with the extracted firmware: all 10 integration tests passed on the 50 records in the store at that time.	8 Oct 2026
14	Component finding records do not name the affected source file, their 'source' is where the scanner read the version inside the firmware.	High	6 Oct 2026	Testing	Alan - Vulnerability Analysis Team, Hemanth (follow-up)	The 17 findings with a public fix commit get their file from it (10 go on to Task 6, 7 go back to the Vulnerability Analysis Team because the fix touches several files, the file was not in that version, or the fix is already in). The other 30 have no public fix commit, so no file can be named; they go back with NO_FIX_COMMIT and a next step. Patching-side handling completed (backlog ORC-16). Closed.	10 Oct 2026
15	The model sometimes gives an answer the pipeline cannot use, and answers change between runs (3 dnsmasq findings stop at Task 6, 1 at Task 9).	Medium	9 Oct 2026	Testing	Hemanth	Ollama context window raised, Task 6 and Task 9 now accept a long answer instead of failing (10 Oct), the runner retries these stops up to 3 times per finding (15 Oct). Bad patches are still caught by Task 10 and the reviewer.	Open
16	C code findings have no Task 10 test profile, so 6 findings with a candidate patch wait at Task 10 (NO_VALIDATION_PROFILE).	High	10 Oct 2026	Testing	Kanishk, Hemanth	Fuzz mode exists in Task 10. The C-code test spec, AFL++ image and harness were added (backlog ORC-24, Kanishk's branch feature/dnsmasq-validation-source), so the 6 C-code findings that waited at Task 10 in the 9 October run can be tested on the next run. Closed.	15 Oct 2026
17	The model's patch for the root account set an empty password.	High	9 Oct 2026	Testing	Hemanth	Task 10 rejected it, so nothing unsafe reached approval: the check worked as designed. No separate rule-based fix is needed; the team removed rule-based code from the pipeline in favour of the model plus Task 10's checks. Closed, backlog item REL-06 descoped.	23 Oct 2026
18	The risk score shows nan for some findings on the review screen.	Low	Week 11 (5-11 Oct 2026)	Testing	Hemanth	Not a defect. No task produces a separate risk score in this release; patches are ranked by Task 12's score, so "nan" only means that field has no value. Closed as not an issue (backlog item BUG-11).	23 Oct 2026
19	Two approved patches for the same file: deploy order is not handled.	Medium	Week 12 (12-18 Oct 2026)	Development	Hemanth	Deploy one patch per file at a time and re-test the second on top of the first.	Open
20	Deploy only writes to a test copy of the firmware file, there is no deploy to a real device or manual rollback command.	Medium	Week 11 (5-11 Oct 2026)	Implementation	Patching team with the client	Agree a test device with ElectraWireless, then add device deploy and a manual rollback.	Open
21	The dashboard has no login, so anyone using the pipeline laptop could accept a patch.	Medium	15 Oct 2026	Implementation	Dashboard owner, Hemanth	Accept and Reject work only on the pipeline laptop and record the reviewer's git name. Add a login before wider use.	Open
22	There is no live hosted dashboard yet. A hosted copy could not reach the pipeline laptop, so it could only show a snapshot.	Low	14 Oct 2026	Implementation	ElectraWireless, dashboard owner	The project runs on localhost on the pipeline laptop, where all decisions are made. ElectraWireless will set up hosting when they go live; until then a snapshot can be exported (export-patching-snapshot.mjs).	Open

No	Description	Impact	Date Raised	Phase Raised	Allocated to	Resolution Strategies / Actions	Date Closed
23	The runner, the reason-code change and the dashboard work are on the pipeline laptop but not committed, and orchestrator-dev is not merged into main.	High	15 Oct 2026	Implementation	ElectraWireless team lead (handed over by Hemanth)	Handed over in the source zip, which holds the working copy from the pipeline laptop. Committing this work and merging orchestrator-dev into main is ElectraWireless's decision; their team lead will do it.	Open
24	A hydration warning appeared in the dashboard when browser extensions (grammar checkers) changed the page.	Low	15 Oct 2026	Testing	Hemanth	layout.tsx now ignores attributes that extensions add to the html and body tags. Copied to the dashboard folder on the laptop on 17 Oct.	17 Oct 2026
25	The dashboard showed laptop folder paths (for example the store path in System Health).	Medium	15 Oct 2026	Testing	Hemanth	Paths shortened or hidden in everything the dashboard shows, password hashes hidden too.	15 Oct 2026
26	Raw model replies, markdown and emoji showed in the Investigations panel.	Low	15 Oct 2026	Testing	Hemanth	Model text is cleaned before display and each stop is shown as a plain sentence.	15 Oct 2026
27	FIX_CHANGES_SEVERAL_FILES and FIX_FILE_NOT_IN_VERSION were assigned to the Patching team, but only the scanner side can say which file a finding is about.	Low	14 Oct 2026	Analysis	Hemanth, Vulnerability Analysis Team	Both codes now point to the Vulnerability Analysis Team, with the next step to put the file in affected_file. Changed on 14 Oct in the release working copy; closes when issue 23 is closed.	Open (with issue 23)
28	Patch bytes changed between Windows and Linux (line endings) when Task 10 and Task 14 wrote patched files.	High	Week 11 (5-11 Oct 2026)	Testing	Kanishk	Patches are now read and written as bytes, and a test checks that Windows line endings are kept. Merged into main with PR #92. Product Backlog BUG-13.	9 Oct 2026
29	Task 6 and Task 9 failed when the model's answer was too long and was cut off.	Medium	Week 11 (5-11 Oct 2026)	Testing	Hemanth	Long answers are now accepted, and Task 9 is given the faulty lines with a margin instead of the whole file (Orchestration repo 5bf587c). Product Backlog BUG-09. The remaining unusable answers are issue 15.	10 Oct 2026

2.2 RISK REGISTER

No	Description	Impact	L'hood	Owner	Mitigation Strategies	Contingency Plan
1	Inconsistent environment setup among team members.	Medium	Low	All members	Write clear setup guide and a test everyone environment.	If someone got issue during the setup other team member can help to resolve.
2	Assignments Overlaps	Medium	Medium	All members	Plan early, manage time wisely.	Ask for extension for submission.
3	Disagreement on changes like of datasets, LLM,	Low	Low	All member	Early discussion, before finalising datasets or making big changes.	Mention the reason for making change like unviability, licensing issue.
4	Ran out of free tokens or hit daily limit which may slow the development	Medium	Medium	All member	Look out for how much we are using the tokens, and don't waste on repeated testing.	Switch to another free model temporarily
5	Runing fuzzing tools outside the sandbox environment which will affect computer	High	Low	All member	Put test in the script to check for sandbox environment first, if sandbox environment is not detected it won't run	Check the changes that might happen in the computer and disconnect it from the network.
6	Patch passes through our test but does not fix the vulnerability.	High	Medium	All members	Rerun the exploits against the patched code	Add a new test patch

No	Description	Impact	L'hood	Owner	Mitigation Strategies	Contingency Plan
7	Auto deployment of patch	High	Low	All members	Keep the auto approve off by default and test the approval system.	If the auto approve function doesn't work as intended undo the deployment and check for issues.
8	Recording only passed patches	High	Low	All members	Make sure as a team to record every pass and failed patch deployment.	Proofread all recording, if something is missing, discuss with team.
9	Team members overwrite each other work on shared repo	Low	Medium	All members	Understand the repo structure and its branches.	Sort out the conflict and every member should commit and put files more.
10	Slow LLM response when generating multiple patches	High	Low	All Members	Test on smaller number of datasets, limit the number of candidate patches per bug,	Switch to faster or smaller model, refine the LLM prompts if possible after discussion with the team.

3 PRODUCT BACKLOG

The Product Backlog lists every user story and work item with its effort in story points (SP), priority, owner, sprint and status, as at 23 October 2026. PT items come from the Assignment 1 backlog; ORC (orchestration), BUG and REL (release) items were added during the project. Each item has a Trello card. Sprints are two weeks long: Sprint 0 Weeks 2-3, Sprint 1 Weeks 4-5, Sprint 2 Weeks 6-7 (with the mid-semester break), Sprint 3 Weeks 8-9, Sprint 4 Weeks 10-11 and Sprint 5 Weeks 12-13. The full workbook is Product_Backlog_Velocity_Burndown_Group4.xlsx.

Totals: 289 SP in scope, 267 SP completed and 22 SP remaining. REL-04 (merge into main, the ElectraWireless team lead's decision) and REL-06 (rule-based credential fix, not needed because Task 10 already blocks the unsafe patch, Issue 17) are descoped. BUG-11 ("nan" risk score) was closed as expected behaviour on 23 October (Issue 18).

3.1 USER STORIES AND BACKLOG ITEMS

Table PB-1: Product Backlog items

ID	Item	Type	Owner	Priority	SP	Planned sprint	Sprint(s)	Status	Completed
PT-01	Environment Setup	Feature	Whole Team	High	3	Sprint 1	Sprint 1	Completed	13 Aug 2026
PT-02	Architecture & Design Document	Feature	Hemanth	High	5	Sprint 1	Sprint 1	Completed	15 Aug 2026
ORC-01	Agent orchestration integration document for the Product Owner	Orchestration	Kanishk	Medium	2	Sprint 1	Sprint 1	Completed	14 Aug 2026
PT-17	Web UI (Flutter) - landing page, theme toggle, defense map	Feature	Sanskar	Medium	5	Sprint 1	Sprint 1	Completed	18 Aug 2026
PT-03	Dataset Preparation	Feature	Kanishk	High	8	Sprint 1	Sprint 1, finished in Sprint 2	Completed	24 Aug 2026
PT-04	Sandbox Build	Feature	Kanishk	High	13	Sprint 1	Sprint 1, finished in Sprint 2	Completed	24 Aug 2026
PT-08	Existing Patch Check Module	Feature	Kanishk	Medium	5	Sprint 2	Sprint 2	Completed	27 Aug 2026
PT-06	Localization Module (Heuristic + LLM)	Feature	Hemanth	High	13	Sprint 1	Sprint 1, finished in Sprint 2	Completed	29 Aug 2026
PT-09	Patch Generation & Validation (Minimal + Defensive)	Feature	Sanskar	High	13	Sprint 1	Sprint 1, finished in Sprint 4	Completed	06 Oct 2026
ORC-02	Stable finding ID agreed with the Vulnerability team as the hand-off key	Orchestration	Hemanth	High	2	Sprint 2	Sprint 2	Completed	28 Aug 2026
PT-05	Vulnerability Detection Module	Feature	Sanskar	High	5	Sprint 1	Sprint 1, finished in Sprint 3	Completed	16 Sep 2026
ORC-03	Task 6 on real firmware findings (multi-site reporting, test cases)	Orchestration	Hemanth	High	5	Sprint 2	Sprint 2	Completed	10 Sep 2026
ORC-04	outtake.py - hand findings from the shared store to the Patching Engine	Orchestration	Hemanth	High	5	Sprint 2	Sprint 2	Completed	11 Sep 2026
ORC-05	Integration test runner with evidence	Orchestration	Hemanth	High	5	Sprint 2	Sprint 2	Completed	12 Sep 2026
PT-18	Automated tests for Tasks 3, 4, 8 and IoTGoat patch validation cases	Feature	Kanishk	Medium	5	Sprint 2	Sprint 2	Completed	12 Sep 2026
PT-07	Root Cause Understanding Module	Feature	Anthony	Medium	8	Sprint 2	Sprint 2, finished in Sprint 4	Completed	07 Oct 2026

ID	Item	Type	Owner	Priority	SP	Planned sprint	Sprint(s)	Status	Completed
ORC-06	Task 6 to Task 7 hand-off integration	Orchestration	Anthony	Medium	3	Sprint 2	Sprint 2, finished in Sprint 4	Completed	10 Oct 2026
ORC-07	Pipeline: walk every finding through the stages in order, with lifecycle and per-finding source lookup	Orchestration	Hemanth	High	8	Sprint 3	Sprint 3	Completed	20 Sep 2026
BUG-01	Pipeline could mark a finding 'matched_with_patch' with no patch	Bug	Hemanth	High	3	Sprint 3	Sprint 3	Completed	23 Sep 2026
BUG-02	Task 9 fixes: file path, indentation, candidate count, patch search, timeout	Bug	Sanskar	High	5	Sprint 2	Sprint 2, finished in Sprint 4	Completed	06 Oct 2026
BUG-03	Pipeline did not detect stages built in new folders; add stages 10-12, show 13-14	Bug	Hemanth	Medium	3	Sprint 3	Sprint 3	Completed	25 Sep 2026
ORC-08	Integration tests rewritten to drive pipeline.py (8 pass, 2 not run)	Orchestration	Hemanth	High	3	Sprint 3	Sprint 3	Completed	25 Sep 2026
BUG-04	Patch-agent copy in the Orchestration repo was out of date	Bug	Hemanth	Medium	3	Sprint 3	Sprint 3	Completed	26 Sep 2026
BUG-05	Hardcoded machine paths in Task 6; prompt version not recorded	Bug	Hemanth	Medium	2	Sprint 3	Sprint 3	Completed	26 Sep 2026
ORC-09	First real findings past step 0 (both localisation methods agree on lines 1 and 8)	Orchestration	Hemanth	High	3	Sprint 3	Sprint 3	Completed	26 Sep 2026
BUG-06	Finding ID changed when the firmware path was typed differently	Bug	Hemanth	High	2	Sprint 3	Sprint 3, finished in Sprint 4	Completed	29 Sep 2026
BUG-07	Task 14 file named pipeline.py clashed with the pipeline; moved to deploy package	Bug	Anthony	Low	1	Sprint 3	Sprint 3, finished in Sprint 4	Completed	07 Oct 2026
BUG-08	dnsmasq records carry a landing-page link that cannot be downloaded	Bug	Hemanth	Medium	1	Sprint 4	Sprint 4	Completed	09 Oct 2026
PT-10	Docker Build & Test Runner	Feature	Kanishk	High	8	Sprint 3	Sprint 3, finished in Sprint 4	Completed	06 Oct 2026
PT-11	Exploit Replay & AFL++ Fuzzing	Feature	Kanishk	High	13	Sprint 3	Sprint 3, finished in Sprint 4	Completed	06 Oct 2026
PT-12	Scoring Engine	Feature	Kanishk	High	8	Sprint 3	Sprint 3, finished in Sprint 4	Completed	06 Oct 2026
PT-13	Approval Workflow (Mode A)	Feature	Sanskar	Medium	5	Sprint 3	Sprint 3, finished in Sprint 4	Completed	07 Oct 2026
PT-14	Automated Deployment & Monitoring (Mode B)	Feature	Anthony	Medium	8	Sprint 3	Sprint 3, finished in Sprint 4	Completed	07 Oct 2026
ORC-10	Dashboard proposal: what both teams can and cannot supply	Orchestration	Hemanth	Low	2	Sprint 4	Sprint 4	Completed	02 Oct 2026
ORC-11	Find each finding's firmware from its own asset_id (--firmware-dir)	Orchestration	Hemanth	High	3	Sprint 4	Sprint 4	Completed	06 Oct 2026
REL-01	Merge Tasks 7, 9-14 into Elly-Defense main	Release	Whole Team	High	5	Sprint 4	Sprint 4	Completed	10 Oct 2026
REL-02	Sync merged tasks into the Orchestration repo and connect them in the pipeline	Release	Hemanth	High	5	Sprint 4	Sprint 4	Completed	10 Oct 2026
REL-03	Decide credential findings: one shared patch or each on its own line	Release	Hemanth	Medium	1	Sprint 4	Sprint 4	Completed	10 Oct 2026
PT-16	Final Integration & End-to-End Pipeline	Release	Whole Team	High	13	Sprint 4	Sprint 4	Completed	10 Oct 2026

ID	Item	Type	Owner	Priority	SP	Planned sprint	Sprint(s)	Status	Completed
REL-04	Merge orchestrator-dev into main	Release	ElectraWireless team lead	High	2	Sprint 5	Sprint 5	Descoped	
PT-15	Benchmarking & Reporting	Feature	Whole Team	Medium	5	Sprint 5	Sprint 5	In Progress	
ORC-12	Download upstream source automatically from source_repo_url and source_ref	Orchestration	Hemanth	Medium	5	Sprint 4	Sprint 4	Completed	08 Oct 2026
ORC-13	Read-only data export for the Elly Defence dashboard	Orchestration	Hemanth	Low	3	Sprint 5	Sprint 5	In Review	
ORC-14	Extract the agreed firmware image, checked by SHA-256	Orchestration	Hemanth	High	2	Sprint 4	Sprint 4	Completed	05 Oct 2026
ORC-15	Sync the vuln-scanner copy from Elly-Defense main	Orchestration	Hemanth	Medium	1	Sprint 4	Sprint 4	Completed	06 Oct 2026
ORC-16	Name the affected source file for each component finding	Orchestration	Hemanth, Vulnerability Analysis Team	High	5	Sprint 4	Sprint 4	Completed	10 Oct 2026
ORC-17	Reason code, who acts and next step for every stop	Orchestration	Hemanth	High	3	Sprint 4	Sprint 4	Completed	10 Oct 2026
ORC-18	Status report and full run guide for the team	Orchestration	Hemanth	Medium	2	Sprint 4	Sprint 4	Completed	10 Oct 2026
ORC-19	Move Task 7 into patch-agent/pipeline/understand	Orchestration	Hemanth	Low	1	Sprint 4	Sprint 4	Completed	10 Oct 2026
ORC-20	Approve and deploy scripts for Tasks 13 and 14	Orchestration	Hemanth	High	3	Sprint 4	Sprint 4	Completed	10 Oct 2026
ORC-21	Task 10 test profiles for the credential findings	Orchestration	Hemanth	High	2	Sprint 4	Sprint 4	Completed	10 Oct 2026
BUG-09	Task 6 and Task 9 failed when the model answer was too long	Bug	Hemanth	Medium	2	Sprint 4	Sprint 4	Completed	10 Oct 2026
ORC-22	Show patching results in the existing Elly Defence dashboard panels	Orchestration	Hemanth	High	5	Sprint 5	Sprint 5	Completed	15 Oct 2026
ORC-23	Runner: pipeline runs on its own; Accept / Reject from the dashboard	Orchestration	Hemanth	High	5	Sprint 5	Sprint 5	Completed	15 Oct 2026
ORC-24	Task 10 test spec, AFL++ image and harness for C code findings	Orchestration	Kanishk, Hemanth	High	8	Sprint 5	Sprint 5	Completed	15 Oct 2026
BUG-10	Model answer not usable at Task 6 for 3 dnsmasq findings	Bug	Hemanth	Medium	2	Sprint 5	Sprint 5	Completed	15 Oct 2026
BUG-11	Risk score shows nan for some findings	Bug	Hemanth	Low	1	Sprint 5	Sprint 5	Completed	23 Oct 2026
BUG-12	Dashboard showed laptop paths, raw model text and a hydration warning	Bug	Hemanth	Medium	2	Sprint 5	Sprint 5	Completed	17 Oct 2026
REL-05	Handover documents and pending task list for the client's team lead	Release	Hemanth	Medium	3	Sprint 5	Sprint 5	Completed	15 Oct 2026
REL-06	Rule-based fix for hard-coded credentials (root account)	Release	Hemanth	Medium	3	Backlog	Backlog (Issue 17, closed)	Descoped	
REL-07	Deploy to a real test device with a manual rollback	Release	Patching team	Medium	8	Backlog	Backlog (Issue 20)	Not Started	
REL-08	Deploy two approved patches for the same file in a safe order	Release	Hemanth	Medium	3	Backlog	Backlog (Issue 19)	Not Started	
REL-09	Login for the dashboard before wider use	Release	Dashboard owner	Medium	3	Backlog	Backlog (Issue 21)	Not Started	
ORC-25	Stage run contract; Task 9 output aligned with Task 10 validation	Orchestration	Kanishk	High	3	Sprint 4	Sprint 4	Completed	06 Oct 2026
BUG-13	Patch bytes changed between Windows and Linux (line endings) in Task 10 and Task 14	Bug	Kanishk	High	2	Sprint 4	Sprint 4	Completed	09 Oct 2026

3.2 SPRINTS AND VELOCITY

Table PB-2: Sprints and velocity (planned SP = items whose work started in the sprint; completed SP = items finished in it)

Sprint	Dates (2026)	Weeks	Focus	Planned SP	Completed SP (velocity)	Planned items finished (SP)	Carried over (SP)
Sprint 1	10 Aug - 23 Aug	Weeks 4-5	Foundation and first modules: environment, architecture and design, orchestration document, web UI, datasets and sandbox, Task 6 started	67	15	15	52
Sprint 2	24 Aug - 13 Sep	Weeks 6-7	Core modules finished (datasets, sandbox, existing patch check, localisation) and Phase 2 integration with the Vulnerability team: real findings, hand-off, integration tests	43	61	27	16
Sprint 3	14 Sep - 27 Sep	Weeks 8-9	Pipeline and bug / issue sprint: pipeline through every stage, status safety check, stage detection, finding ID; Tasks 10-14 built	70	30	25	45
Sprint 4	28 Sep - 11 Oct	Weeks 10-11	Integration sprint: merge all tasks, connect Tasks 6-14, agreed firmware image, first full run, approval and deploy scripts	61	135	61	0
Sprint 5	12 Oct - 25 Oct	Weeks 12-13	Release sprint: dashboard and runner, fixes found in testing, handover documents, submission	36	26	26	10

Average velocity 53.4 SP per sprint over Sprints 1-5, and 60.25 SP over the finished Sprints 1-4. Sprint 0 (Weeks 2-3) was onboarding with no backlog items. Many Sprint 3 items (Tasks 9 to 14) were finished on 6-7 October, when all tasks were merged and connected, which is why Sprint 4 shows the highest velocity.

3.3 STORY POINTS PER PERSON

Table PB-3: Completed story points per person per week (an item's points are split between the members who own it)

Week	Sprint	Kanishk	Sanskar	Hemanth	Anthony	Team total
Week 2	Sprint 0	0	0	0	0	0
Week 3	Sprint 0	0	0	0	0	0
Week 4	Sprint 1	2.75	0.75	5.75	0.75	10
Week 5	Sprint 1	0	5	0	0	5
Week 6	Sprint 2	26	0	15	0	41
Mid-semester break	Sprint 2	0	0	0	0	0
Week 7	Sprint 2	5	0	15	0	20
Week 8	Sprint 3	0	5	8	0	13
Week 9	Sprint 3	0	0	17	0	17
Week 10	Sprint 4	0	0	4	0	4
Week 11	Sprint 4	38.5	27.5	40.5	24.5	131
Week 12	Sprint 5	4	0	21	0	25
Week 13	Sprint 5	0	0	1	0	1
Total		76.25	38.25	127.25	25.25	267

Story points measure finished backlog items, not hours worked; hours are in each member's timesheet.

3.4 BURN-DOWN

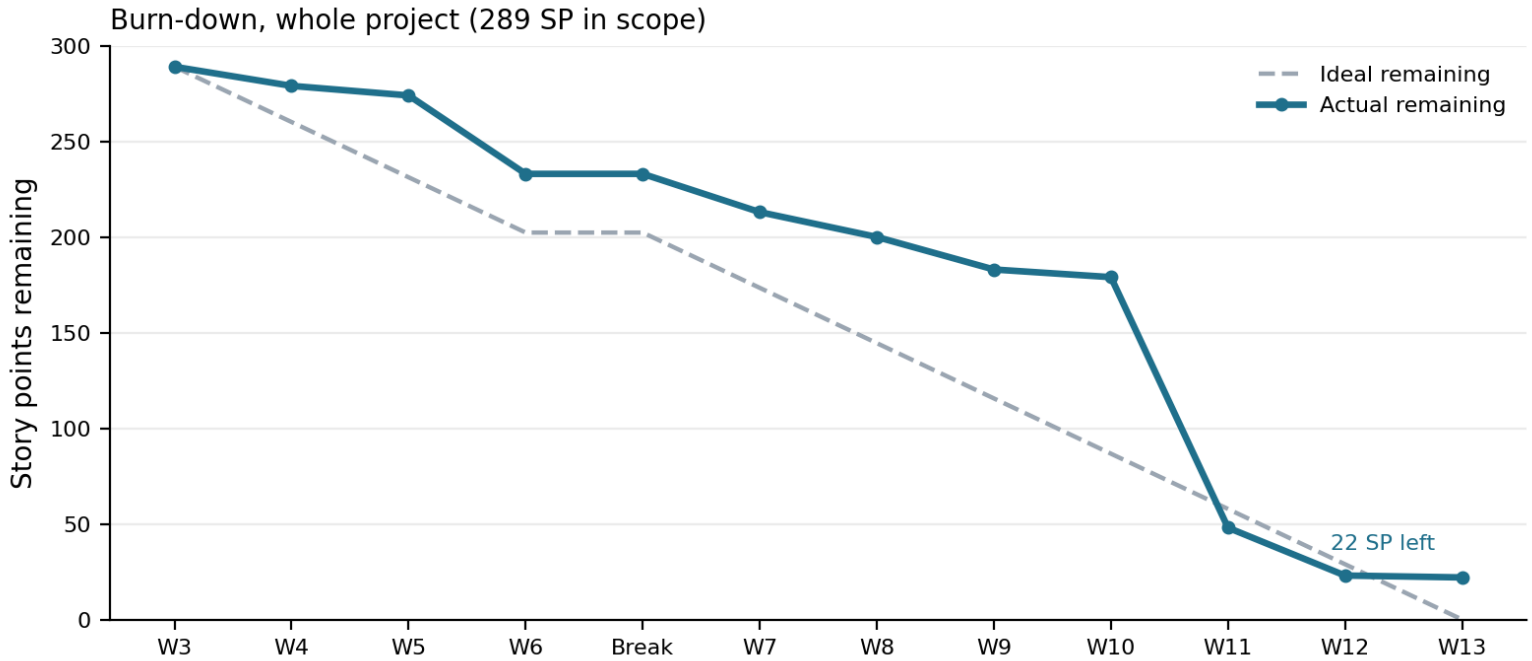


Figure PB-1. Burn-down for the whole project: remaining story points at the end of each week against an ideal line to Week 13. The 22 SP left are items in review, in progress or kept in the backlog for later work (Issues 19 to 21).

4 TECHNICAL SOLUTION DESIGN

4.1 INTRODUCTION

The Elly Defence Autonomous Assisted Patching Agent takes a vulnerability found in a firmware image and tries to turn it into a tested patch that a person can accept or reject. The Project Charter set the goal and two phases: build a self-contained patching pipeline, then connect it to the Vulnerability Analysis Team so their findings become our input. Both phases were delivered.

The whole patching side runs on one Windows laptop. It is written in Python, tests every patch inside Docker, and uses Qwen3-Coder through Ollama as a local language model, so no firmware or source code leaves the machine. Results are shown in the existing Elly Defence dashboard.

We rate the complexity as high. The pipeline has fourteen tasks, written by four people on separate branches, the input comes from another team, and the language model does not always give the same answer twice. Every step therefore checks its input and records clearly why it stopped.

Table 1: What the solution achieved by Sprint 5

Result	Evidence
A hard-coded password in the firmware went from scanner finding to a tested, approved and deployed patch with no manual edits (Section 4.4.2)	Full run on the scanner report loaded on 9 October, finding *1ce6ca579626ca0a*
Every one of 49 real findings was sorted in one run, each with a reason and the team that has to act	reports/status_report.md
Localisation picked the right function for 21 of 21 labelled Juliet examples	Task 6 testing
An unsafe patch (empty root password) was caught before anyone could approve it	Task 10 result for finding 42b8a6012f4947f8
All 10 integration tests passed on 50 records	reports/integration_test_evidence.json (8 October)
A reviewer can accept or reject a patch from the dashboard, and the pipeline runs on its own	Runner and dashboard tests (15 October)

4.1.1 Background: what the solution works on

A router or camera runs software that is built into it, called firmware. The manufacturer ships it as one image file. Inside the image is a small Linux system: programs, settings files, and web pages, packed into a compressed read-only file system called SquashFS. When the image is unpacked, that file system appears as a folder named squashfs-root, which is why every file in our findings starts with squashfs-root/.

Our test device is OWASP IoTGoat v1.0, a router firmware based on OpenWrt that was made insecure on purpose for security training. Testing on it is legal and safe, and its weaknesses are known, so we can tell whether a patch is right. The team agreed on one copy of the image (IoTGoat-x86.img.gz). The scanner gives it the ID asset_13f9a5407a85, and the pipeline finds the unpacked files through that ID.

The Vulnerability Analysis Team's scanner unpacks the image, looks through it, and writes what it finds to report.json. Each problem it reports is a finding. There are two kinds:

- **Credential findings.** A password is built into a file of the firmware itself, so the fix is a change to that file. 2 of the 49 findings are of this kind, both in /etc/shadow.

- **Component findings.** A program inside the firmware, such as busybox 1.28.4, dropbear 2017.75 or dnsmasq 2.73, is a version with a published vulnerability (a CVE). The fix belongs in that program's source code, which is not in the firmware, so the pipeline downloads the source at that exact version. 47 findings are of this kind.

/etc/shadow is the standard Linux file that holds account passwords. It has one line per account, and the fields on a line are separated by colons. The second field is not the password itself but a hash of it: a one-way scrambled form. "1" at the start means MD5-crypt, an old method that can be cracked offline. Figure 1 shows where the file sits in the firmware, what line 8 of it means, and how the fix works.

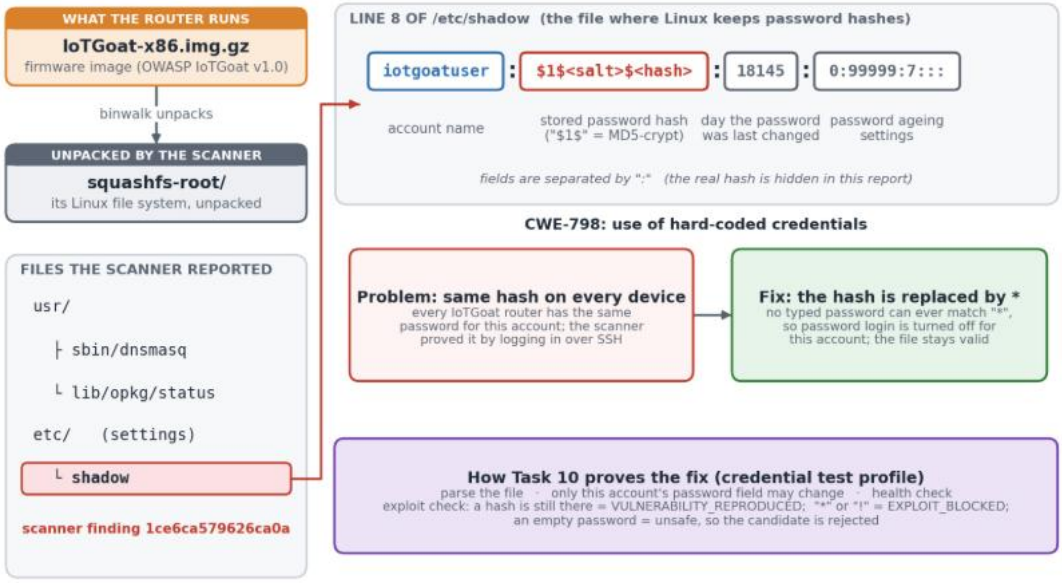


Figure 1. Where /etc/shadow comes from, what one line of it means, and how the patch is checked

The problem is that the hash is part of the firmware image. Every IoTGoat router ever built ships with the same iotgoatuser password, so anyone who learns it once can log in to all of them. This weakness has its own number, CWE-798 "use of hard-coded credentials". The scanner did not just guess: it logged in over SSH to a running IoTGoat with that account and recorded "SSH login succeeded as 'iotgoatuser'". The safe fix is to replace the hash with "*". No typed password can produce "*", so password login for that account is turned off, while the line keeps its shape and the rest of the system still reads the file normally.

4.1.2 Key terms

Table 2: Terms used in this document

Term	Meaning here
Firmware image	The single file a device runs from. Ours is IoTGoat-x86.img.gz.
SquashFS, squashfs-root	The compressed file system inside the image, and the folder it becomes when unpacked.
asset_id	The scanner's ID for one firmware image (asset_13f9a5407a85 for IoTGoat).
Finding, finding_id	One problem reported by the scanner, kept as one JSON record. Its ID is a hash of the firmware content, so it never changes between scans.
CVE	A public ID for one known vulnerability in one product, for example CVE-2015-8899 in dnsmasq.
CWE	The type of weakness, for example CWE-798 hard-coded credentials or CWE-787 out-of-bounds write.
Upstream source, fix commit	The original project's source code, and the change in its git history that fixed a CVE. The fix commit tells us which file and lines were wrong.

Localisation	Finding the exact lines that are wrong (Task 6).
Candidate patch, diff	A proposed change, written as a diff: lines starting with - are removed and lines starting with + are added.
Test profile	The settings Task 10 uses for one finding: Docker image, build, test, exploit and health commands. Made once from a spec a person has reviewed.
Exploit replay	Running a check that proves the problem on the original file, then running it again on the patched file to show it is blocked.
AFL++	A fuzzer: it feeds a program large numbers of random inputs to look for crashes. Used when there is no exploit to replay.
Health check, rollback	A quick command run after deploying to confirm the file still works. If it fails, the original bytes are put back.
Ollama, qwen3-coder	Ollama runs language models on the laptop; qwen3-coder is the coding model we use with it.
OpenRouter	A hosted service that gives access to many language models through one API key. Used during development (Section 4.2.1).
Reason code	A short code saying why a finding stopped, for example NO_FIX_COMMIT, with the team that has to act (Section 4.11.2).
Runner	A background program that runs the pipeline on its own and carries out Accept, Reject and Run again requests.

4.1.3 Team roles and who built what

The Patching Agent Team had four members. Each task was written and tested by one person on their own branch and merged after review. The orchestration, which joins all tasks into one pipeline and connects it to the other team and to the dashboard, was built by Hemanth. Table 3 shows who built what, and Table 4 gives more detail on the orchestration and integration work, because that part is what the rest of this document describes.

Table 3: Who built what

Who	Role	Built
Kanishk Sharma	Scrum Master	Project Charter. Tasks 3 and 4 (datasets and Docker sandbox, PR #23), Task 8 (existing patch check, PR #26), Tasks 10 to 12 (validation, security checks and scoring, PR #84), and build inputs for C projects (branch feature/dnsmasq-validation-source, 14 October).
Sanskar Pokharel	Developer	Task 9 patch generation (PRs #85 and #87) and Task 13 approval (PR #88). Risk Register.
Anthony MacKenzi Fernando	Developer	Task 7 understanding (PRs #29 and #91) and Task 14 deploy, monitoring and rollback (PR #73).
Hemanth Petlu	Architecture and orchestration	Task 2 design (Assignment 1). Task 6 localisation. The orchestration pipeline, Task 0, reason codes, release scripts, runner and integration tests. Integration with the Vulnerability Analysis Team and with the Elly Defence dashboard. This document, the Issue Register, the Product Backlog and readme.txt.
Vulnerability Analysis Team (Alan and team)	Other workstream	The scanner and its reports, the record contract, the first versions of store.py and intake.py, content-based finding IDs, and the agreed IoTGoat image.
Dashboard owner	Other workstream	The Elly Defence dashboard (Next.js). We added our data to its existing panels.

Table 4: Orchestration and integration work in detail (Hemanth)

Area	What was done	When / evidence
Task 2 design	design.md and design_rationale.md: the PatchJob record, module layout, seven stages, config.py, BaseLLMClient, exploit-gated scoring and Architecture Version 2 (Figure 3).	13 and 15 Aug, amended 7 Oct
Task 6 localisation	First version with a rule-based locator and the model (PR #28) and an accuracy report. Machine paths removed and prompt version recorded (PR #75). Redesigned to use the fix commit first, otherwise three model answers and a vote (PR #89), local model only (PR #90). Tested on 21 labelled Juliet examples.	29 Aug to 8 Oct, TASK6_TESTING.md
Pipeline and lifecycle	pipeline.py reads every record, runs the next task and follows the allowed status moves. Safety check so a finding cannot reach matched_with_patch without a patch. Tasks found by folder, so new tasks join without code changes.	20, 23 and 25 Sep
Task 0 and source cache	sources.py works out which file each finding is about, and fetch_sources.py downloads busybox, dropbear and dnsmasq at the exact version into a local cache.	6 to 8 Oct
Connecting Tasks 6 to 14	One wrapper per task in pipeline/stages/ that calls the member's code, checks its result and writes it to the record. Reason codes with an owner for every stop. make_profile.py and the credential spec for Task 10. approve.py and deploy.py, which call Task 13 and Task 14 and are never called by the pipeline. status_report.py and FULL_RUN_GUIDE.md.	10 Oct, orchestrator-dev
Integration tests	run_integration_tests.py: 10 cases run on a throwaway copy of the store with real records; 10 of 10 passed.	8 Oct, integration_test_evidence.json
Runner	runner.py and start_elly.ps1: loads new reports, runs the pipeline, retries stops a second try can fix, carries out dashboard requests and deploys accepted patches.	15 Oct (Issue 23)
Work with the Vulnerability Analysis Team	Found and proved the finding ID bug (Issue 1), the firmware image mismatch (Issue 9), the two credential findings pointing at the same lines (Issue 10) and the missing affected file on component findings (Issue 14). Added the fields we need to intake.py and store.py within the record contract.	26 Sep to 14 Oct
Elly Defence dashboard	lib/patching/ (seven files) turns records into the items the existing panels already show; eight existing API routes now include our findings, app/api/patching/jobs talks to the runner, a snapshot export for the hosted copy, password hashes and laptop paths hidden (Section 4.4.3).	14 and 15 Oct

Jose and Larry, the ElectraWireless contacts named in the Project Charter, left the project in the middle of the semester, so they are not on the distribution list of this version.

4.2 TECHNICAL ENVIRONMENT

Two limits set at the start shaped most choices: code and firmware stay on our own machines, and no patch reaches a person until it has been tested. Table 5 lists the technologies used and why each was chosen.

Table 5: Technologies and the reasons for choosing them

Technology	Used for	Why
Python 3.10+	Pipeline, orchestrator scripts, all tasks	Known by the whole team. The scanner is also Python, so both teams share store.py and lifecycle.py.
Ollama with qwen3-coder (qwen2.5-coder:7b on smaller laptops)	Tasks 6, 7 and 9 in the final runs	Runs offline and is trained for code. Only about 3B of its 30B parameters are active at a time, so it runs on a laptop.
OpenRouter API (hosted models)	Tasks 7 and 9 during development, still an option	Gave the team a strong model before the local set-up was ready. Replaced by Ollama for final runs (Section 4.2.1).
Docker Desktop (elly-iotgoat-component:local, gcc:13, aflplusplus/aflplusplus)	Tasks 10 and 11 testing, Task 14 health check	Every test starts from the same clean container, and a bad patch cannot harm the laptop.
JSON files, one per finding	Findings store shared with the Vulnerability Analysis Team	Easy for both teams to read and compare; no database server on the laptop (Section 4.5).
Git and GitHub (ElectraWireless/Elly-Defense and Elly-Defense-Orchestration)	Source control, one branch per task, pull requests	The client's own organisation; pull requests gave a review step before code reached main.
PowerShell	start_elly.ps1	The pipeline laptop runs Windows.
Next.js (Elly Defence dashboard)	Showing results, Accept / Reject	Already the Elly Defence interface, so we used its existing panels instead of a new screen.
OWASP IoTGoat v1.0, Juliet Test Suite, CVEfixes	Test device, measuring localisation, early testing	Cases where the right answer is known, so results can be checked.
Trello and Microsoft Teams	Backlog, sprint board, team communication	Used by the team since Sprint 1, as set out in the charter.

Each task was written on its own branch in the Elly-Defense repository and merged into main through a pull request. The Elly-Defense-Orchestration repository holds the orchestrator, which connects the tasks, plus a synced copy of the task code (patch-agent).

4.2.1 Language models: local and hosted

Three tasks use a language model. Task 6 uses QwenClient, which follows the BaseLLMClient interface from the Assignment 1 design. Tasks 7 and 9 were written by their owners against the OpenAI-style interface that OpenRouter offers. When the pipeline runs with --local-model, common.py points Tasks 7 and 9 at the same local Ollama server and model that QwenClient uses,

so there is one model setting for the whole pipeline. Without that option, Tasks 7 and 9 need OPENROUTER_API_KEY. Table 6 shows every model used.

Table 6: Language models used and where

Model	Where it runs	Used by	Status at release
qwen3-coder	On the laptop, through Ollama	Tasks 6, 7 and 9	Used for every final run (pipeline.py --local-model)
z-ai/glm-5.3-flash	OpenRouter (hosted)	Task 7, as written by its owner	Used whenever --local-model is not given (needs OPENROUTER_API_KEY)
nvidia/nemotron-3-ultra-550b-a55b:free	OpenRouter (hosted, free tier)	Task 9, as written by its owner	Used whenever --local-model is not given; two of its patches were used in the Task 12 test run
Two hosted providers	Hosted	Task 6 trial	Rejected (see below)

OpenRouter let Task 7 and Task 9 be built and tested early with large models before the local set-up was ready. For the final runs we moved every task to the local model, for three reasons found during Task 6 testing: the free quotas ran out partway through a single 21-example test, a hosted model can be changed by its provider without notice so a measured accuracy cannot be repeated, and a local model keeps the client's firmware and source code on the laptop. The OpenRouter code is still in Tasks 7 and 9 and works if a key is put in .env. Whether to use it is the client's decision (Section 4.10.3). No API key is stored in either repository.

4.3 OVERALL ARCHITECTURE

4.3.1 System context

The Patching Agent sits between the Vulnerability Analysis Team's scanner and the person who approves a fix (Figure 2). The scanner writes report.json. intake.py loads every finding into a shared store as its own JSON file. From then on the pipeline only works on the store and never calls the scanner, so the two teams only had to agree on one thing: the format of a finding record (record_contract.md).

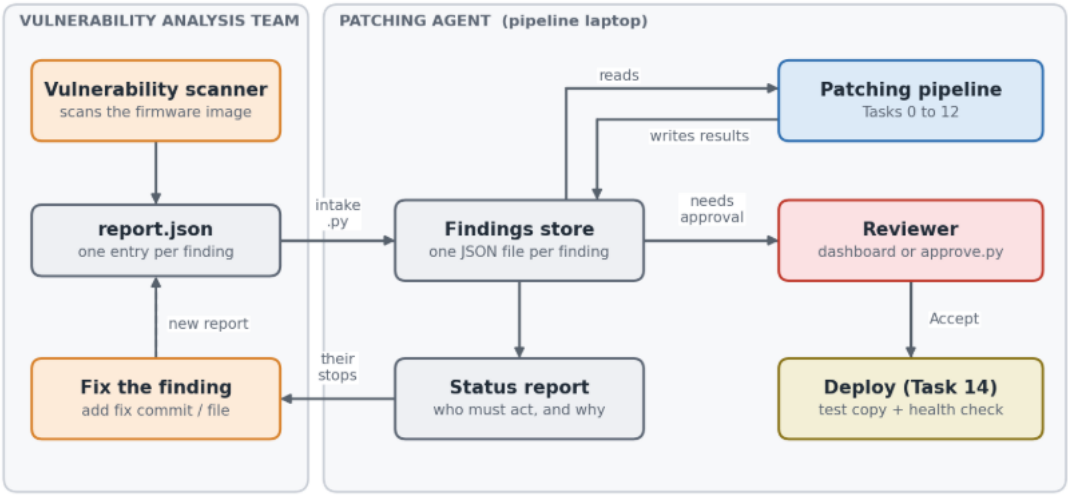


Figure 2. System context: the Patching Agent and the systems around it

Results also flow back. Every finding that stops because of missing scanner data carries a reason code and a next step, and the status report groups these by reason (for example, 30 findings with NO_FIX_COMMIT: add the fix commit or the affected file). The Vulnerability Analysis Team sends a corrected report, intake.py loads it, and findings that had not started take the new details on the next run.

The dashboard reads the same store. When someone clicks Accept, Reject or Run again, the dashboard does not run anything itself: it writes a small request file that the runner checks again before acting. Approval and deploy are separate scripts from the pipeline, so the pipeline can never approve or deploy on its own. Outside the laptop, the only connection is a one-time download of the upstream source code (busybox, dropbear and dnsmasq) at the exact version the device runs.

4.3.2 Design changes since Assignment 1

Figure 3 is Patching Agent Architecture Version 2 from Assignment 1, as it appears in the Project Charter. The final design keeps its shape: intake, localisation, understanding, existing patch check, generation, validation in Docker, exploit-gated scoring, human approval and deployment. What changed came mostly from phase 2, when our input became the Vulnerability Analysis Team's findings instead of our own intake, and from what we learned running real IoTGoat findings. Table 7 lists each change.

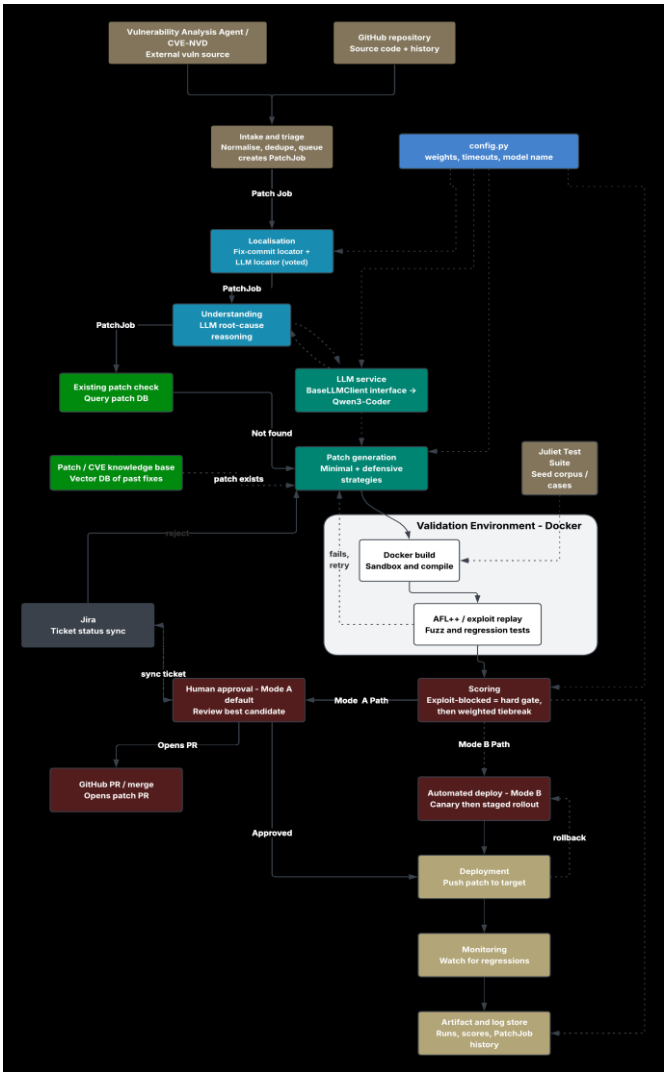


Figure 3. Patching Agent Architecture Version 2 - Final Version

Table 7: Assignment 1 design compared with the final solution

Area	Assignment 1 design	Final solution	Why it changed
Input (Task 5)	Our own intake and triage of CVE / NVD / scanner data into a PatchJob	The Vulnerability Analysis Team's report, loaded by intake.py, Task 5 dropped	Their scanner already produces normalised findings; two intakes would disagree.
Data contract	PatchJob object passed between stages in memory	One JSON record per finding, with status and history, shared by both teams	Two teams and many runs: results must last between runs and be visible to both sides.
Localisation (Task 6)	Rule-based locator plus language model	Lines from the public fix commit, otherwise the model asked 3 times	The rules covered 5 CWEs; real findings span many more. Changed 7 October.
Existing patch check (Task 8)	Vector database of past fixes	Exact match in local JSON patch stores	Simple to check and explain; version ranges are not guessed.
Generation (Task 9)	Two strategies: minimal and defensive	Up to 3 candidates around the faulty lines, asked again if none pass	Task 10, not the strategy label, decides which patch is safe.
Language model	Qwen3-Coder behind BaseLLMClient	Qwen3-Coder through Ollama for every final run, OpenRouter kept as an option	Hosted quotas ran out and results could not be repeated (Section 4.2.1).
Validation (Tasks 10-11)	Docker build, tests, exploit replay, AFL++	Same, plus a reviewed test profile per finding	A patch is only tested against settings a person has checked.
Scoring (Task 12)	Exploit-blocked gate, then weights 0.3 / 0.3 / 0.4	Kept as designed	Still fits; test-count credit waits for per-test results.
Approval (Task 13)	Mode A, Jira ticket, GitHub pull request	Mode A in approve.py or the dashboard, name from git, no Jira or PR step	Patches change firmware files, not our repository, and the team used Trello.
Deployment (Task 14)	Mode B: canary, then staged rollout	Deploy to a test copy with health check and rollback, Mode B off	No test device was available, staying safe came first.
Monitoring and records	Artefact and log store	Status report, evidence folders, runner job logs, dashboard	Same purpose, using what the pipeline already writes.
Configuration	config.py	config.py plus runner.config.json per laptop	No laptop folder is written into the code.
New in the final design	-	Reason code and owner for every stop, runner, dashboard connection	Needed once real findings and another team were involved.

4.3.3 How the solution was built: timeline

Table 8 lists the main steps from the Assignment 1 design to the release, with who did each one. Dates come from the git history of both repositories and the pull requests.

Table 8: Build timeline

Date (2026)	What happened	Who
13-15 Aug	Task 2 design written and reviewed (design.md, design_rationale.md)	Hemanth
19 Aug	Project Charter V1.3 issued	Kanishk
24-27 Aug	Datasets and Docker sandbox (Tasks 3 and 4), existing patch check (Task 8)	Kanishk
29 Aug	Task 6 first version with accuracy report (PR #28)	Hemanth
Sep	Task 7 tests and evidence, Task 9 fixes on its branch, Tasks 10 to 12 built	Anthony, Sanskar, Kanishk
17 Sep	Phase 2 starts: record contract agreed with the Vulnerability Analysis Team	Alan
20-25 Sep	Orchestration pipeline: reads every record, status rules, safety check, Tasks 10 to 12 joined	Hemanth
29 Sep	Store rebuilt on content-based finding IDs, after the ID bug was shown (Issue 1)	Alan, Hemanth
5 Oct	Agreed IoTGoat image committed, firmware mismatch closed (Issue 9)	Vulnerability Analysis Team, Hemanth
6-10 Oct	Tasks 7, 9, 10 to 14 reviewed and merged into main (PRs #29, #73, #84, #85, #87 to #92)	All members
7-8 Oct	Task 6 redesigned (PRs #89, #90) and connected, source cache, 10 of 10 integration tests passed	Hemanth
9 Oct	New scanner report loaded, first patch approved (finding 1ce6ca579626ca0a)	Hemanth
10 Oct	Tasks 6 to 14 connected with reason codes, approve.py and deploy.py, full run of 49 findings, patch deployed	Hemanth
14 Oct	Two reason codes moved to the Vulnerability Analysis Team, build inputs for C projects	Hemanth, Kanishk

15 Oct	Runner, start script and dashboard connection	Hemanth
17-18 Oct	Final documents and handover	All members

4.4 SYSTEM ARCHITECTURE

Figure 4 shows the solution as built. The centre column is the path of one finding from the scanner report to the status report. The left box collects the reasons a finding can stop, and the right column shows the services each task uses.

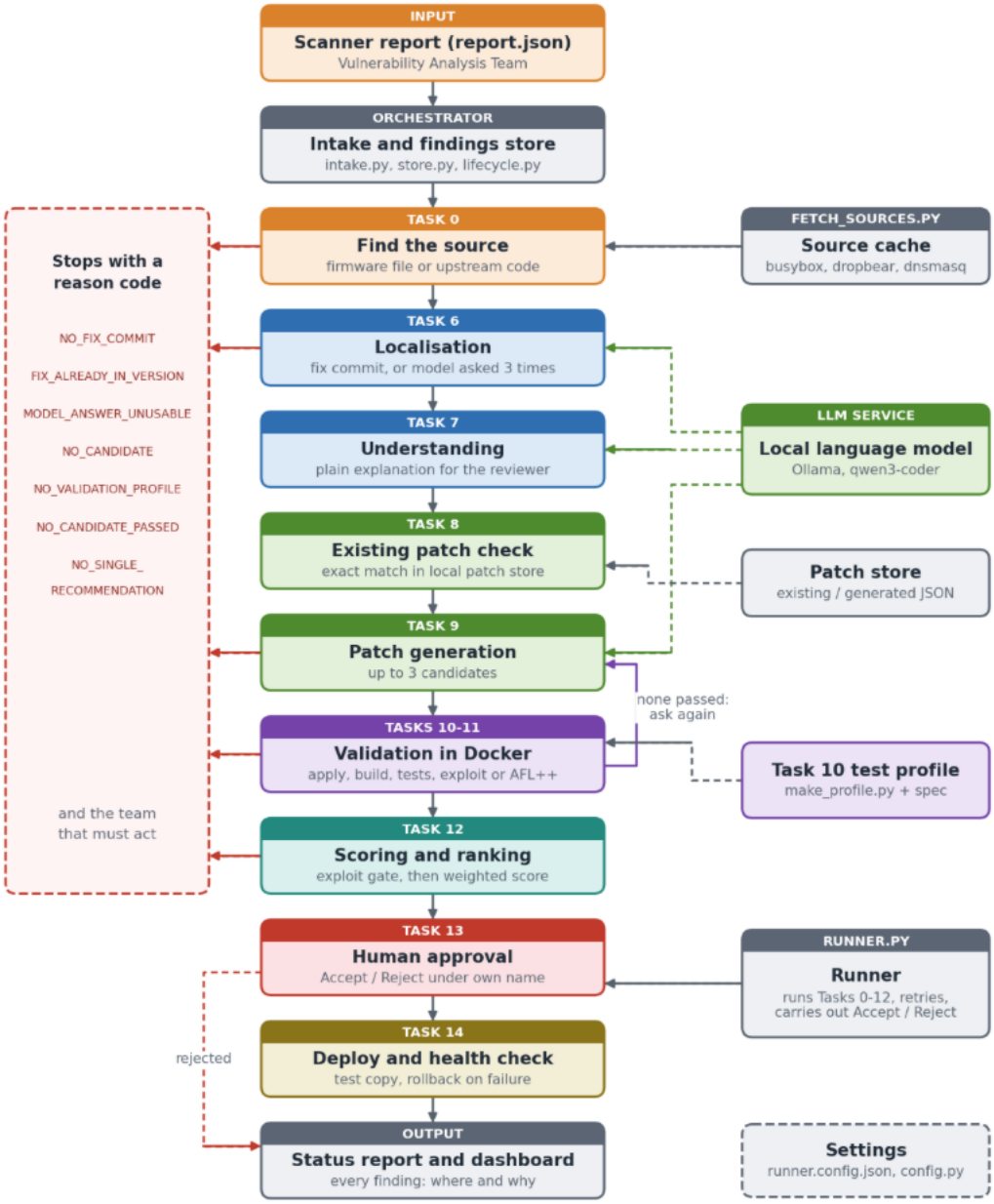


Figure 4. Patching Agent architecture as built (Version 3, October 2026)

The code sits in two folders of the Elly-Defense-Orchestration repository (Figure 5). orchestrator/ moves findings through the steps and records what happened. patch-agent/ holds the code behind each task, kept in step with the Elly-Defense main repository where each task was written and reviewed.

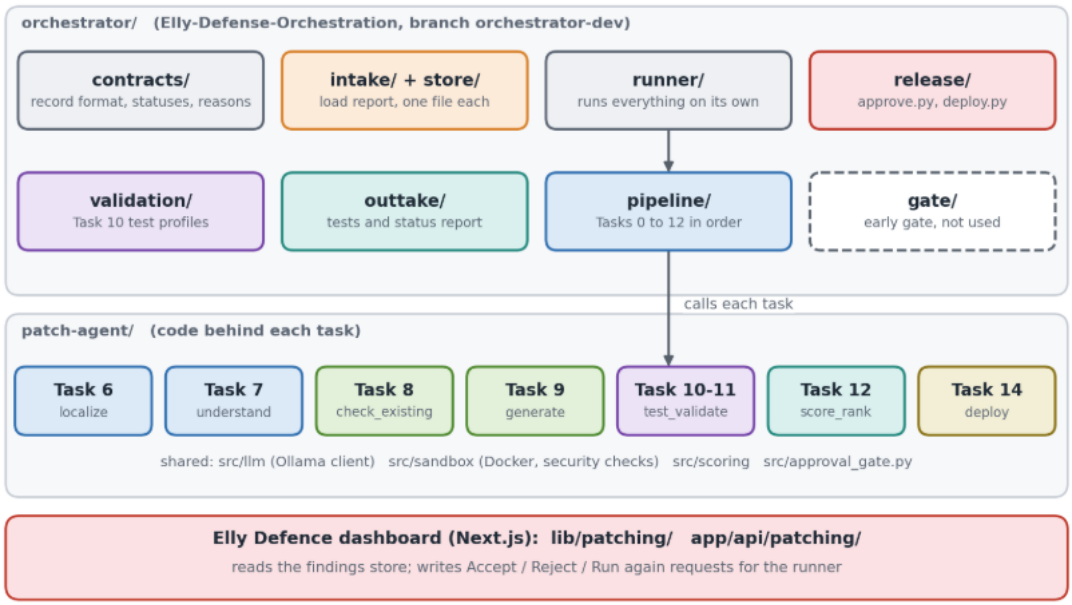


Figure 5. Where each part of the solution lives in the code

Table 9: Main components

Component	Main files	Responsibility	Built by
contracts/	sources.py, lifecycle.py, record_schema.json, record_contract.md	Record format, allowed status moves, and every reason code with the team that acts on it.	Alan (contract), Hemanth
intake/, store/	intake.py, store.py	Load a scanner report, one JSON file per finding, named by its finding_id.	Alan, Hemanth (additions)
pipeline/	pipeline.py, stages/task06 to task12, stages/common.py	Run Task 0 and Tasks 6 to 12 in order on every finding that can move, stop with a reason when a step cannot go on.	Hemanth
patch-agent/	pipeline/localize, understand, check_existing, generate, test_validate, score_rank, deploy, src/llm, src/sandbox, src/scoring	The work of each task.	Each task owner (Table 3)
validation/	make_profile.py, specs/	Build the Task 10 test profile for a finding from a reviewed spec.	Hemanth
release/	approve.py, deploy.py	Task 13 and Task 14. Never called by the pipeline.	Hemanth, calling Sanskar's and Anthony's code
outtake/	run_integration_tests.py, status_report.py	Integration tests on a throwaway copy of the store, the status report.	Hemanth
runner/	runner.py, start_elly.ps1, runner.config.example.json	Run everything on its own and carry out requests from the dashboard.	Hemanth
Dashboard	lib/patching/, app/api/patching/	Turn records into plain sentences for the existing panels, write Accept / Reject / Run again requests.	Hemanth

4.4.1 Functionalities/features

The solution has six main functions. Each is described below with a flow chart, in the order a finding meets them.

4.4.1.1 Loading findings and finding the source (intake and Task 0)

intake.py reads report.json and writes one record per finding. Loading a newer report is safe: findings that have not started take the newer details and findings in progress are left alone. The scanner's part of the record is copied as it is and never changed. Task 0 (sources.py) then works out which code the finding is about (Figure 6). A hard-coded credential points at a file inside the unpacked firmware, found through the finding's asset_id. A component finding such as busybox 1.28.4 needs the upstream code at that exact version, and the public fix commit tells us which file to open. fetch_sources.py downloads each project once into a local cache. Where Task 0 cannot be sure, it stops instead of guessing.

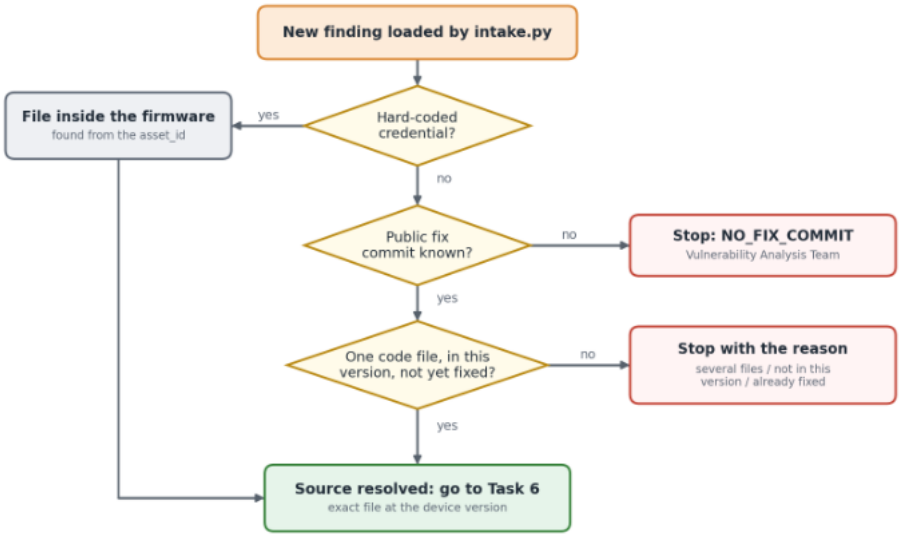


Figure 6. Loading a finding and finding its source code

4.4.1.2 Localisation (Task 6)

If the fix commit changes the file, the lines it changed are the answer. Otherwise the model reads the CVE description and the source file and names the line. It is asked three times and the most common answer is taken. If the three answers differ, the finding is flagged for a person to check (Figure 7). The record keeps the method used, the lines, the file read and any review reason. Task 6 was measured on 21 Juliet examples where the faulty line is known: it named the right function in all 21 and the exact line in 10.

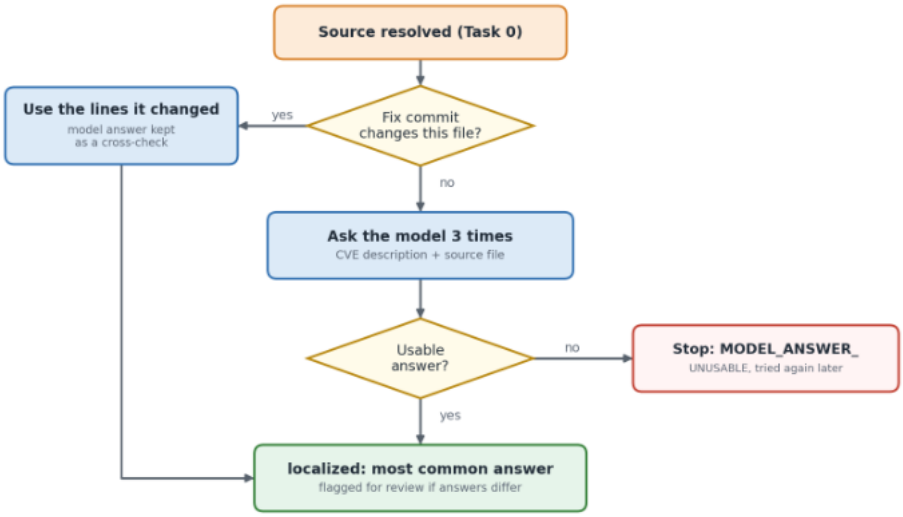


Figure 7. Finding the faulty line

4.4.1.3 Understanding, existing patch check and patch generation (Tasks 7 to 9)

Task 7 writes a short explanation of the defect for the reviewer. Task 8 looks in the local patch stores for a candidate made for exactly the same repository, version and component. If one exists, it is sent to validation instead of writing a new one. Otherwise Task 9 asks the model for up to three candidate patches, showing it the faulty lines plus 60 lines on each side. An answer is only kept if it contains exactly one block of code, and each candidate is tied to the hash of the file it was written for (Figure 8).

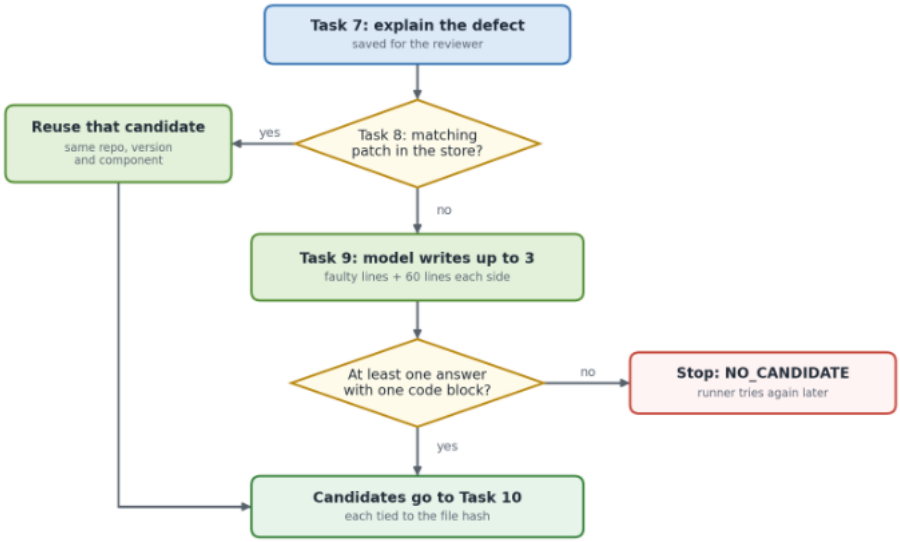


Figure 8. Explaining the defect and getting candidate patches

4.4.1.4 Validation and ranking (Tasks 10 to 12)

Task 10 will not test a patch without a test profile for that finding, made once with make_profile.py from a spec a person has reviewed. The original code must build and pass its tests first, then each candidate is applied, built, and tested in Docker. Task 11 replays the known exploit (the original must print VULNERABILITY_REPRODUCED and the patched code EXPLOIT_BLOCKED) or, when there is no exploit, runs AFL++ for a short, fixed time. Task 12 then ranks the candidates that passed (Figure 9).

Scoring keeps the Assignment 1 rule: a candidate that does not block the exploit cannot be ranked. The rest get 0.3 x size + 0.3 x sensitive + 0.4 x tests. Size is 1 / (1 + changed lines / 50), so small patches score higher. Sensitive is 1 / (1 + the number of sensitive areas the patch touches: authentication, permissions, cryptography, or command execution), so a patch that touches risky code scores lower and gets a closer look. Tests is the share of individual tests passed. Task 10 currently records that the test command passed but not how many tests ran, so this part is 0 and the highest possible score is 0.6. With one clear winner, the finding moves to pending_approval; a tie is left to a person.

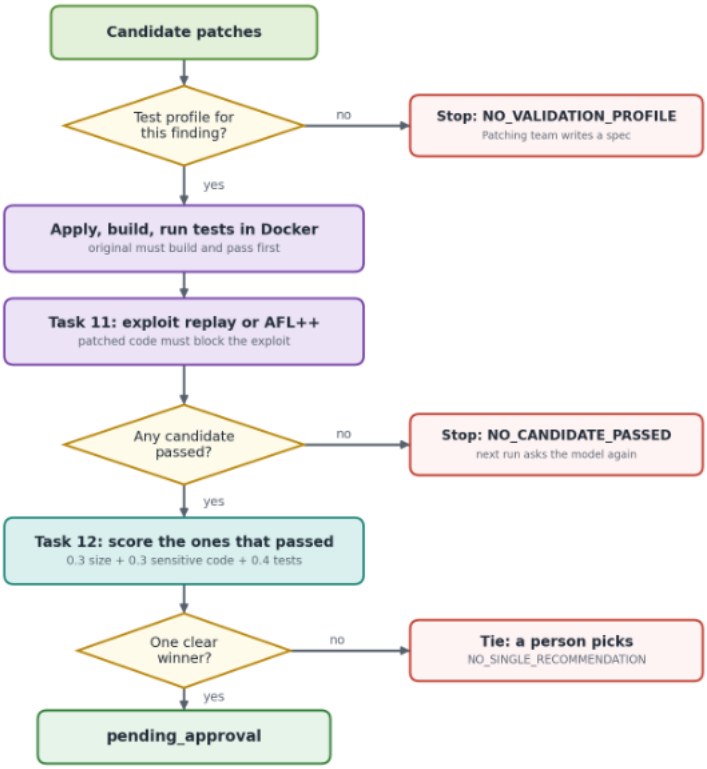


Figure 9. Testing and ranking candidate patches

4.4.1.5 Human approval, deployment and rollback (Tasks 13 and 14)

A reviewer sees the explanation, the change and every test result, in approve.py or in the dashboard's Priority Actions panel, and accepts or rejects. The decision is saved under the reviewer's git name rather than a typed name, together with where it came from. deploy.py then checks that the gcc:13 image is present and that the file still has the hash Task 10 tested, writes the patch into a test copy of the firmware file, and runs the profile's health check in Docker. If the check fails, the original bytes are put back and the finding is marked rolled_back (Figure 10). Deployment never touches a real device in this release.

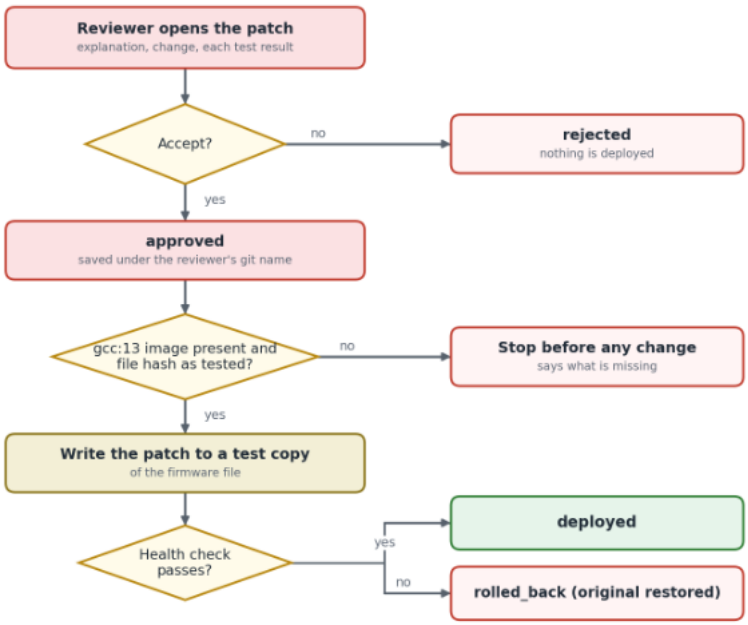


Figure 10. Approving, deploying and rolling back

4.4.1.6 Running on its own: runner and dashboard actions

runner.py keeps the pipeline going without typed commands. It loads reports dropped in an inbox folder, runs every finding that can move, retries stops that a second try can fix (an unusable model answer, no candidate passing, Docker not ready) up to three times per finding but not while Ollama is down, and deploys a patch once it is accepted. start_elly.ps1 checks Docker, Ollama, the images, and the folders, then starts the runner and the dashboard. Buttons only work when the dashboard is opened on the pipeline laptop itself, and the runner checks the finding ID, its status, and the action again before it acts (Figure 11).

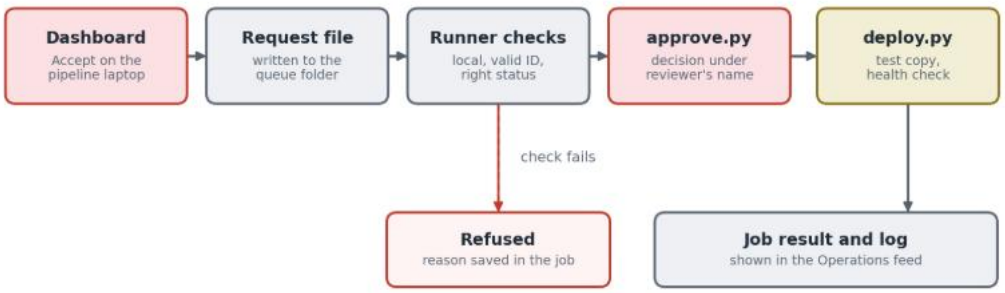


Figure 11. Accepting a patch from the dashboard

Whenever a finding stops, the record gets a reason code, the team that must act and the next step. status_report.py turns the whole store into a short report: totals, what is needed next grouped by reason, and one line per finding, with a CSV copy. The same reasons appear in plain words on the dashboard.

4.4.2 Worked example: one finding from scan to deployment

This section follows one real finding, 1ce6ca579626ca0a, the hard-coded iotgoatuser password described in Section 4.1.1, through every step. Each row of Table 10 is taken from the finding's own records, with times in UTC. Up to approval they come from the run on 9 October; the deployment comes from the full run on 10 October, in which the same finding went through the same steps again and was approved and deployed.

Table 10: Finding 1ce6ca579626ca0a step by step

Step	What happened	Recorded result
Scan (Vulnerability Analysis Team)	The scanner unpacked IoTGoat, read squashfs-root/etc/shadow and saw that the iotgoatuser line stores a password hash. It logged in over SSH to a running IoTGoat with that account to prove the password works, and its own AI check rated the finding 0.90 (0.80 in the newer report used for the 10 October run).	CWE-798, verify_method credential_login, "SSH login succeeded as 'iotgoatuser'", status Validated
Intake	intake.py loaded the report into the store.	Status: new, 29 Sep 22:41
Task 0, find the source	A credential finding is a file inside the firmware, so Task 0 found the unpacked firmware through asset_13f9a5407a85 and opened etc/shadow.	source type extracted_filesystem
Task 6, find the line	There is no CVE or fix commit for a built-in password, so the model was asked three times which line is wrong. All three answers said line 8.	Line 8, method llm-voted, "3 of 3 rounds agreed", status localized, 9 Oct 00:33
Task 7, explain	The model wrote a short explanation for the reviewer (shown in Figure 12). It says the hash is stored "in plain text", which is loose wording: the hash is readable by anyone with the file, but it is not the password itself. The reviewer has to read the model text with that in mind.	explanation saved, 00:34
Task 8, existing fix	No stored patch for this file and version, so a new one is needed.	no match
Task 9, write patches	Three candidates were asked for. Attempt 1 replaced the hash with "*". Attempt 2 was refused because it did not contain exactly one block of code, attempt 3 was refused as a duplicate of attempt 1.	1 of 3 candidates, status matched_with_patch, 00:35
Task 10, test the patch	The test profile came from the credential spec, with account=iotgoatuser. First, the original file: it parses and the exploit check finds the hash (VULNERABILITY_REPRODUCED), which proves the test can see the problem. Then the patched file: apply, parse, "only this account's password field changed" and the exploit check all pass. Fuzzing does not apply to a text file.	baseline PASS, apply, build, tests, exploit PASS, fuzz SKIPPED

Task 12, rank	Size 1/(1+2/50) = 0.96 (one line removed, one added), sensitive 0.5 (the file is about authentication), tests 0 (no per-test counts yet). 0.3 x 0.96 + 0.3 x 0.5 + 0.4 x 0 = 0.438. Only one candidate, so it is the recommendation.	score 0.438, status pending_approval, 00:36
Task 13, human approval	Hemanth read the explanation, the diff and the test results in approve.py and accepted the patch. The name comes from git, not from typing.	approved, mode A, reviewer Hemanth, 9 Oct
Task 14, deploy	deploy.py checked the Docker image and that the file still had the hash Task 10 tested, wrote the patch into a test copy of the shadow file, and ran the health check (every line must still have at least two fields). It passed, so nothing was rolled back.	status deployed, 10 Oct

The change itself is one line. The "-" line is removed and the "+" line is added. The hash is hidden here, as in the rest of this document:

-iotgoatuser:1<hash>:18145:0:99999:7:::
+iotgoatuser:*:18145:0:99999:7:::

Figure 12 shows the same finding as a reviewer sees it in the dashboard. The pipeline's own approval of this finding was made in the terminal on 9 October, before the dashboard connection existed. To take this picture, a test copy of the store was used in which the finding was set back to "Needs human approval".

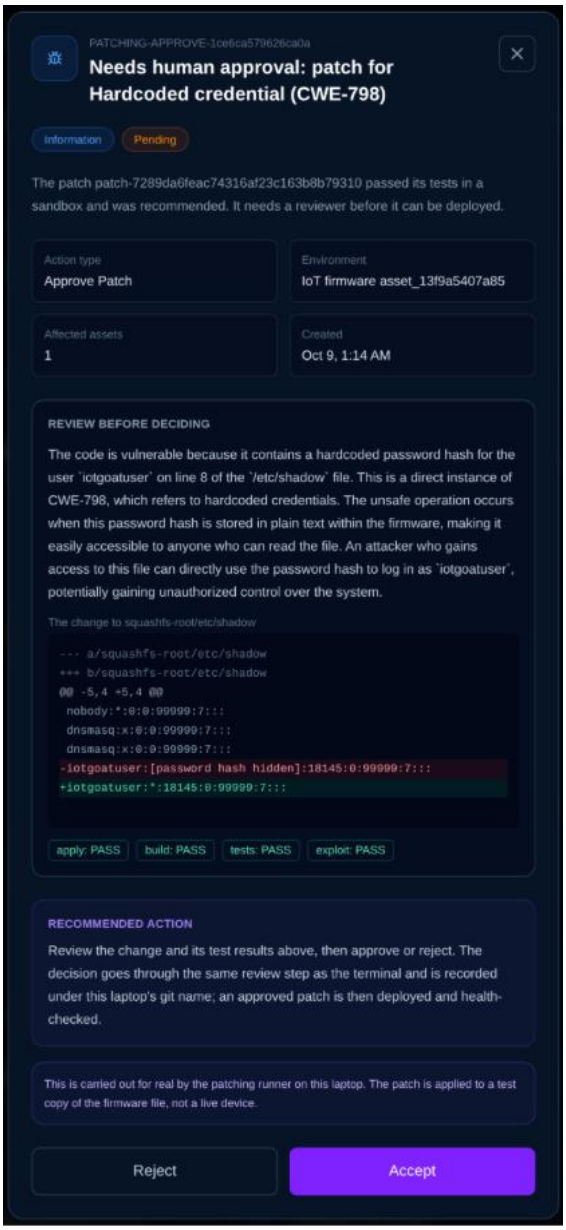


Figure 12. Review screen for finding 1ce6ca579626ca0a in the Elly Defence dashboard (test copy of the store)

Not every patch the model writes is safe, and the same steps show it. For the root account in the same file (finding 42b8a6012f4947f8) the model's candidate emptied the password field. An empty password does not lock the account; on many systems it lets anyone log in with no password at all,

which is worse than the original. The credential check treats an empty field as unsafe, so Task 10 failed the exploit check, no candidate passed, and nothing reached a reviewer. That is the check working as designed, so no separate rule-based fix is needed (Issue 17, closed).

4.4.3 Connection to the Elly Defence dashboard

Elly Defence already had a dashboard, built by another member of the wider project in Next.js, with panels for incidents, investigations, actions and system health filled with sample data. Rather than build a separate screen, we fed our real findings into those panels, so people see patching work where they already look. Nothing in the dashboard changes a record; it only reads the store and, for buttons, leaves a request for the runner.

Table 11: What was added to the dashboard

Part	What it does
lib/patching/load.ts	Reads the findings store on every request when ORCHESTRATOR_STORE is set (live mode), otherwise reads a saved snapshot (snapshot mode, for the hosted copy).
lib/patching/core.mjs, dashboard.ts	Turns each record into the items the existing panels use: feed events, priority actions, investigations, a timeline of the 9 steps and the insight bars. Every item is marked as coming from the patching pipeline.
lib/patching/health.ts	Real checks of what the pipeline needs, for System Health: Ollama, Docker, the store and the runner.
lib/patching/actions.ts, runner.ts, app/api/patching/jobs	Accept, Reject and Run again. Only from the pipeline laptop itself: a button writes a request file into the runner's queue and the runner checks it again before acting.
Eight existing API routes	Command Center (active incidents, operations feed, priority actions, threat level), Control Center (system health) and Intelligence (investigations, attack timeline, AI insights) now list our findings next to their sample items.
scripts/export-patching-snapshot.mjs	Writes the snapshot for the hosted copy, which cannot reach the laptop (Issue 22).
Safety in what is shown	Password hashes are replaced by "[password hash hidden]", laptop folder paths are shortened, and model text is cleaned of markdown before display (Issues 25 and 26).

The screens below were taken on 18 October from the dashboard running on a test copy of the store, not the live laptop, so that no real decision was made. Items that do not mention a finding ID, such as the ransomware and prompt-injection entries, are the dashboard's own sample data and not part of our work.

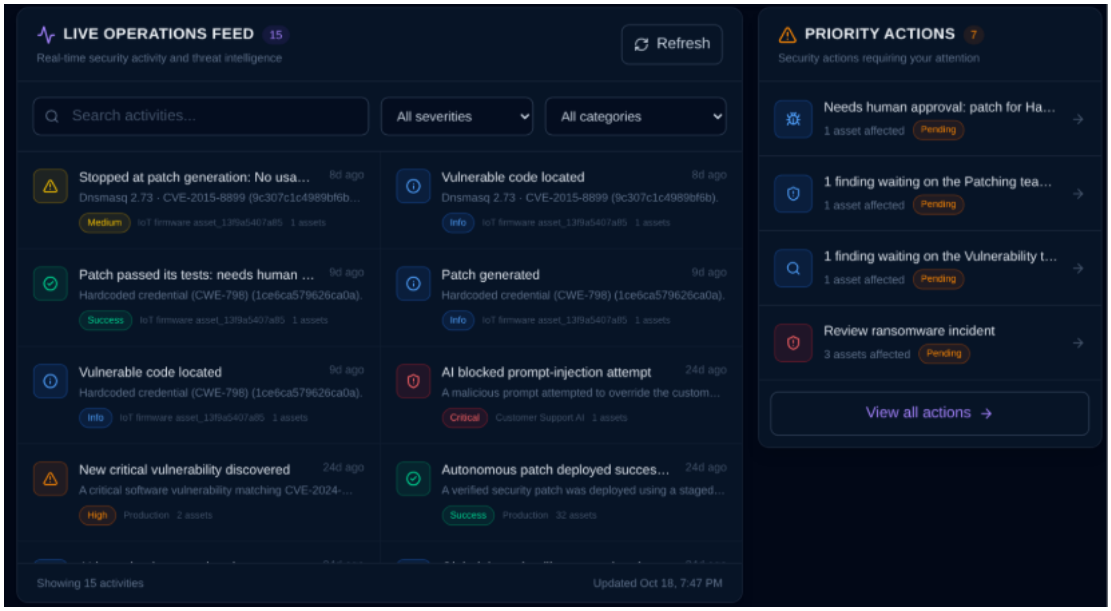


Figure 13. Command Center: patching events in the Live Operations Feed and the review waiting in Priority Actions (test copy)

In the Intelligence page (Figure 14) the Attack Timeline lists each pipeline step for the finding with its result, and AI Insights shows why the finding is trusted: the scanner team's 90% confidence, line 8 located, the root cause explained and one patch tested in the sandbox.

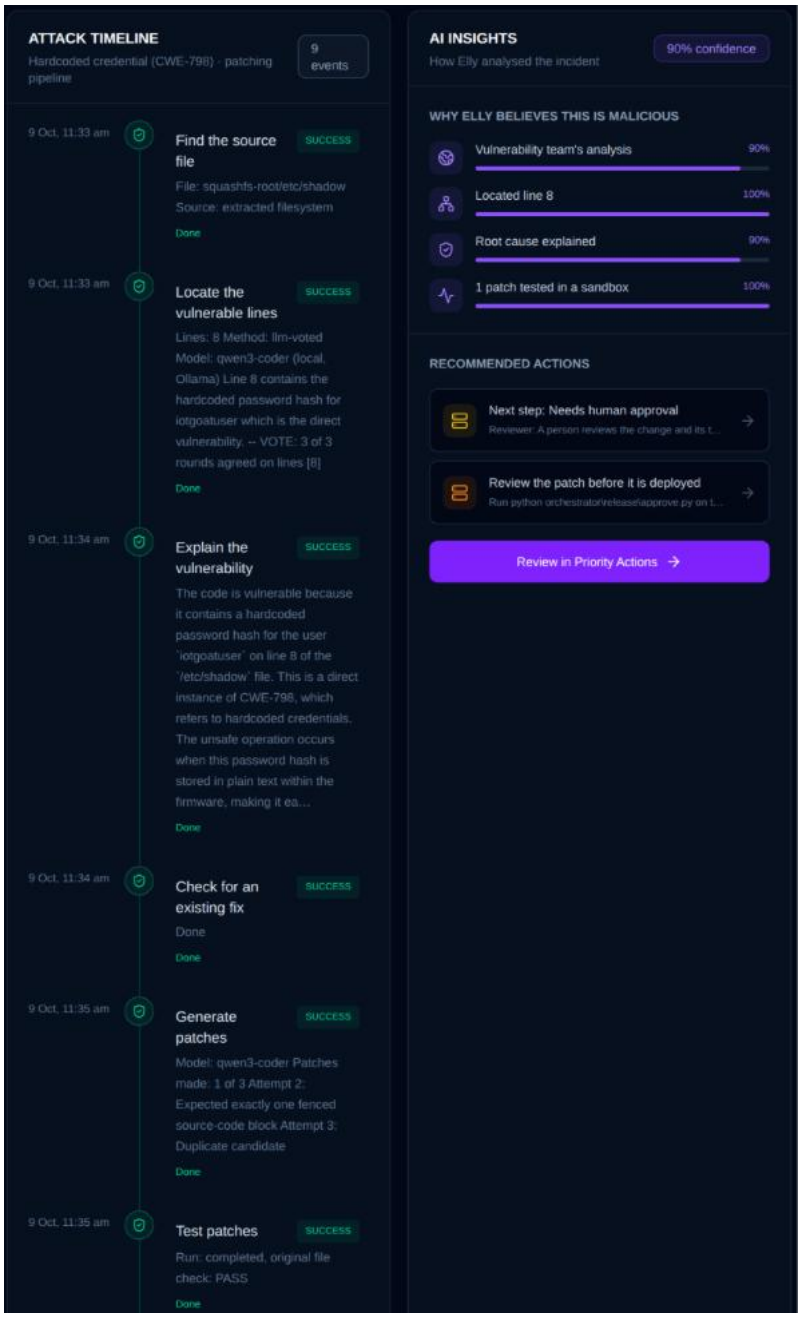


Figure 14. Intelligence page: Attack Timeline and AI Insights for finding 1ce6ca579626ca0a (test copy, first six steps shown)

System Health (Figure 15) shows the four services the pipeline needs. The picture was taken on a test machine without Ollama or Docker, so those two show Unavailable. This is the check working, and on the pipeline laptop all four show Operational. The other six services are the dashboard's own sample entries.

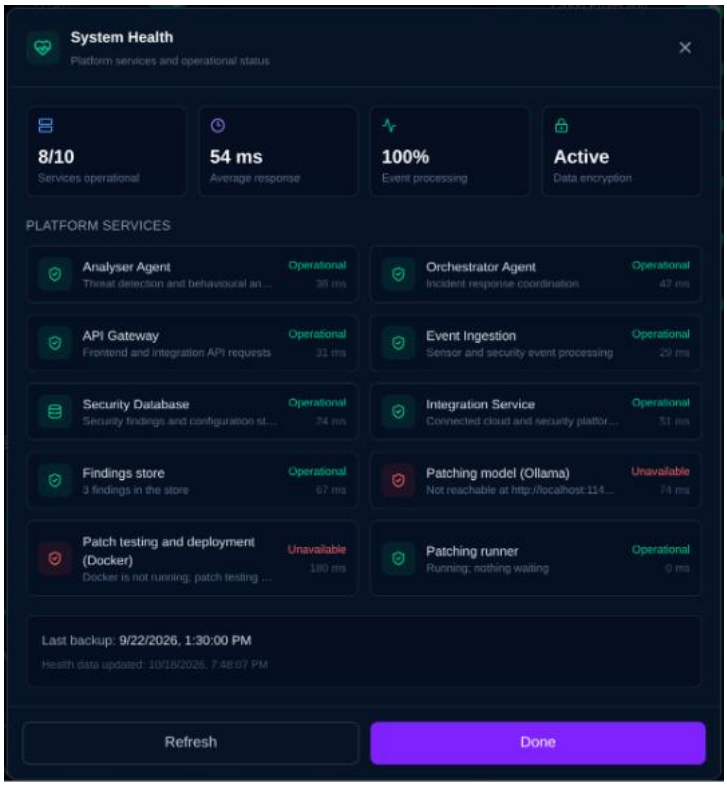


Figure 15. System Health with the patching services: findings store, Ollama, Docker and runner (test machine)

4.5 DATABASE ARCHITECTURE

There is no database server. The findings store is a folder with one JSON file per finding, named after its finding_id. The format is written down in record_contract.md, kept by the Vulnerability Analysis Team, and the rule both teams follow is that a step may add fields but never rename or remove one. Table 12 lists the parts of a record.

Table 12: Parts of a finding record

Field	Written by	Holds
finding_id	Vulnerability Analysis Team	Hash built from the firmware content, so a finding keeps the same ID however the firmware is loaded.
finding	Vulnerability Analysis Team	The scanner's finding as reported: component, version, CVE, CWE, evidence, source_repo_url, source_ref, fix_commit. Never changed.
status, history	Orchestrator	Current lifecycle status (Figure 16) and every change with time and note; history is only added to.
patching	Pipeline	What each task found, and last_run with where the finding stopped and why.
patch_candidates, patch	Tasks 9 and 12	The candidates and the one recommended patch.
approval, deployment	approve.py, deploy.py	The decision with the reviewer's name, the deploy or rollback result with its evidence folder.
created_at, updated_at	Orchestrator	Timestamps.

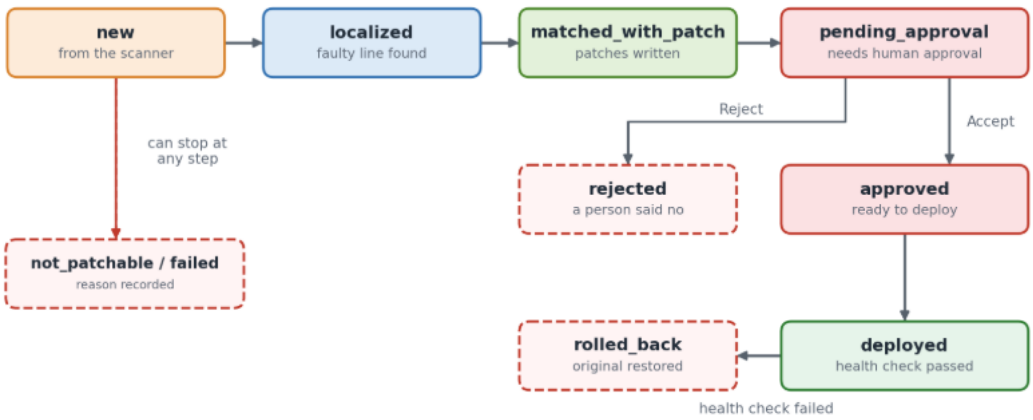


Figure 16. Finding statuses and the allowed moves

lifecycle.py holds the allowed moves and store.py checks them before every write, so an illegal move such as new straight to pending_approval is refused and nothing is saved. Only a person can move a finding out of pending_approval. The localized and not_patchable statuses were proposed by our team and are enforced in code, but they are still marked as awaiting formal sign-off in the contract. A record also carries the scanner's own confidence label inside finding; the pipeline never reads or writes it.

Why files and not a database: a firmware image produces around fifty findings, both teams can open and compare a record in any editor, and the laptop needs no database server. Because each finding is its own file, two steps working on different findings never block each other. For many devices, the same record shape can move into SQLite or PostgreSQL with one row per finding and a JSON column per block. The pipeline only reads and writes through store.py, so that is the only file that would change. The runner's queue also uses plain files: requests/, jobs/, logs/ and a state.json heartbeat.

4.6 IMPLEMENTATION INSTRUCTIONS

The solution runs on one laptop or desktop. Table 13 gives the specification we used and recommend.

Table 13: Recommended pipeline laptop

Item	Recommended
Operating system	Windows 10 or 11, 64-bit (macOS or Linux also work for the pipeline, start_elly.ps1 is for Windows)
Memory	32 GB, the model needs about 24 GB free while it runs
Disk	At least 40 GB free, the model alone is about 19 GB, plus Docker images and the source cache
Software	Git, Python 3.10+, Docker Desktop, Ollama, Node.js and npm, PowerShell
Python packages	openai, python-dotenv, pandas
Docker images	elly-iotgoat-component:local (built from patch-agent/datasets/iotgoat/case-001) and gcc:13
API keys	None. OPENROUTER_API_KEY in .env only if a hosted model is wanted (Section 4.2.1)

Steps (full commands are in FULL_RUN_GUIDE.md and readme.txt):

1. Clone ElectraWireless/Elly-Defense-Orchestration (a private repository; access is granted by ElectraWireless) and check out orchestrator-dev, or extract ProjectID-source.zip.
2. Install the Python packages, pull qwen3-coder, build and pull the two Docker images, set OLLAMA_CONTEXT_LENGTH to 32768 and set your git user name.
3. Create a work folder outside the repository with store, inbox, runner, extracted_firmware, source_cache, validation_profiles and pipeline_work.
4. Unpack the firmware into extracted_firmware/<asset_id>/ and run fetch_sources.py once.
5. Load the first report with intake.py and make the Task 10 profile for each credential finding.

6. Copy runner.config.example.json to runner.config.json, set the folders, and run start_elly.ps1 with the dashboard folder.

Data and migration: the solution does not replace an existing system, so there is no data to move in. The store is a folder, so a backup is a copy of that folder. When the Vulnerability Analysis Team sends a new report, intake.py updates only findings that have not started, so no work in progress is lost. Moving to another laptop means copying the store and setting runner.config.json again; no folder names are written into the code.

4.7 NON-FUNCTIONAL SPECIFICATIONS

Table 14: Non-functional requirements

Requirement	How it is met	How it was checked
Safety	No patch is deployed without a person accepting it; the pipeline has no approve or deploy step.	Integration test INT-11
Data integrity	The scanner's finding is never changed; statuses only move along allowed paths.	INT-02 and INT-05
Traceability	Full status history, reviewer name on each decision, reason and owner on every stop, evidence folder per run.	Status report on the 49-finding run
Privacy	The model runs locally; firmware and source stay on the laptop. Paths and password hashes are hidden on the dashboard.	Review of dashboard output
Repeatability	The first model answer uses temperature 0 and Task 6 takes the majority of three; a second run on the same store changes nothing.	INT-07 and the dry-run test
Reliability	Stops that a second try can fix are retried up to 3 times; deploy rolls back on a failed health check.	Runner tests, Task 14 tests
Security of actions	Dashboard actions only from the pipeline laptop; the runner re-checks every request.	Request from another host refused (HTTP 403)
Portability	All folders come from settings or options.	Ran from different folders in a test virtual machine
Usability	Results in plain words inside dashboard panels people already use.	Wording reviewed by the team
Performance	A few minutes per finding on the laptop model; findings that cannot be patched stop in seconds.	Full run, 9 to 10 October

4.8 SUMMARY OF TEST RESULTS

Full test cases are in the Test Planning and Results section of this report. Table 15 sums up the results and Figure 17 shows the outcome of each of the 49 real findings after the full run on 10 October.

Table 15: Test results

Test set	Date	Result
Integration tests (10 cases, 50 records, pipeline laptop)	8 Oct 2026	10 passed, 0 failed
Task 6 localisation on 21 Juliet examples	6 Oct 2026	Right function 21/21, exact line 10/21
Task 11 security checks	25 Sep 2026	76 tests passed, AFL++ demo ran 29,116 executions with no fault
Task 4 and 8 unit tests	11 Sep 2026	47 passed
Sandbox test cases in Docker	11 Sep 2026	63 passed
Runner and dashboard checks	15 Oct 2026	All 9 passed in a test virtual machine, deploy itself tested on the laptop
Full run on the scanner report loaded on 9 Oct (49 findings)	10 Oct 2026	1 patched and deployed, 1 unsafe patch blocked, 10 waiting for review or test settings, 37 returned to the Vulnerability Analysis Team; each with a reason

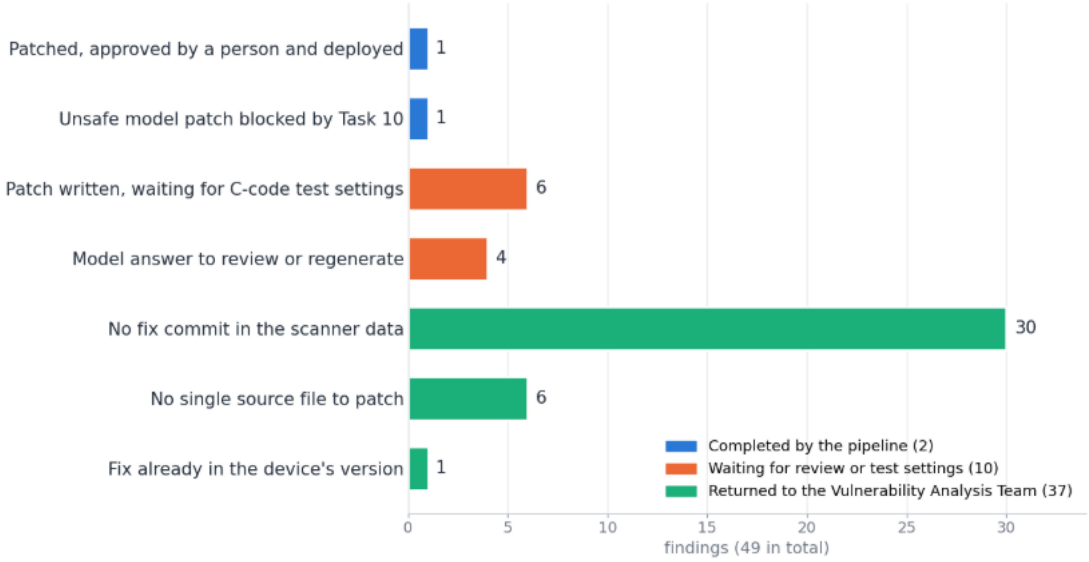


Figure 17. Outcome of the 49 findings after the full run on 10 October 2026

Both hard-coded credential findings came with complete input and went through every task. One was patched, approved by a person and deployed with a passing health check. For the other, the model's patch would have emptied the root password; Task 10 blocked it, so nothing unsafe reached a reviewer. Ten findings reached the patching tasks and are waiting for review or test settings: 6 had a candidate patch waiting for C-code test settings for Task 10 (added on 15 October, Issue 16 closed), and 4 need the model's answer reviewed or regenerated (Section 4.9). The other 37 were returned to the Vulnerability Analysis Team at the input check (Task 0), because the scanner data did not give what patching needs: 30 have no fix commit, 6 do not point to one source file that patching can use (4 fixes change several files, 2 name a file that is not in that version), and 1 is already fixed in the device's version. The pipeline does not guess in these cases; each finding goes back with a reason code and a next step.

4.9 KNOWN ISSUES & RISKS

The open issues below match the Issue Register and the known issues in readme.txt. Project risks are kept in the Risk Register.

Table 16: Open issues at release

No	Issue	Effect	What we do about it
15	Model answers are sometimes unusable and vary between runs	A finding can pass once and stop the next time	Retries, larger context window, Task 10 and the reviewer catch bad patches
19	Two approved patches on the same file	Deploy order not handled	One patch per file at a time, re-test the second
20	Deploy goes to a test copy only	Not yet usable on a device	Agree a test device with ElectraWireless
21	Dashboard has no login	Anyone at the pipeline laptop could accept	Buttons limited to that laptop, add a login
22	No live hosted dashboard yet	A hosted copy could only show a snapshot	ElectraWireless sets up hosting when they go live
23	Latest work not yet merged into main	main does not run the full pipeline	Merging is ElectraWireless's decision; their team lead will do it
27	Two reason codes still point to the Patching team in the committed code	Those findings show the wrong team to act	Already changed in the release working copy; closes with issue 23

4.10 OTHER CONSIDERATIONS

4.10.1 Human approval and safe use of AI

The charter treats language model output as an untrusted candidate, and the design keeps to that. A model writes the patches, but a wrong patch on firmware can lock people out of a device or open a new hole, so every patch has to pass Task 10 and then a named reviewer decides. Deployment is never automatic before that decision, and Mode B (automatic staged rollout) stays switched off.

4.10.2 Vendor requirements from the charter

Table 17: How the final solution covers the vendor-required functions

Function	How it is covered	Status
Asset Discovery	The Vulnerability Analysis Team's scanner gives every firmware image an asset_id; the pipeline finds the firmware by that ID.	Covered by the other team
Vulnerability Scanning	The scanner's report.json is our input.	Covered by the other team
Automated Deployment	After a person accepts, the runner deploys automatically to a test copy with a health check and rollback.	Partly: no real device yet
Compliance Reporting	Status report (Markdown and CSV), full history per finding, reviewer name on every decision, evidence folders.	Covered
Privileged Access	Actions only from the pipeline laptop; approvals under each person's git identity.	Partly: no login yet
Network Communication	Local model; the only outside traffic is downloading upstream source once.	Covered
Staggered Rollouts	Designed in Assignment 1 (Mode B) but not enabled.	Not built

4.10.3 Hosted models and handover

A large hosted model would probably write better patches for big C files than a laptop model. The team used OpenRouter during development for exactly this reason (Section 4.2.1). For real client firmware, though, a hosted model would send the firmware and its source code to a third party, and its answers could change without notice. That trade-off is the client's to make, so the release runs fully local and the OpenRouter option stays off unless a key is added.

The code is being handed over to the client's team lead with handover documents covering the architecture, set-up and deployment, models and datasets, test results and pending tasks. Committing the runner and dashboard work and merging orchestrator-dev into main (Issue 23) is ElectraWireless's decision; their team lead will do it.

4.11 APPENDIX

4.11.1 Traceability to the Project Charter scope

Requirements were tracked on the team's Trello board, one card per backlog item with an owner, story points and a link to the evidence. The full backlog is in the Product Backlog section. Table 18 maps each in-scope item of the charter to what was delivered.

Table 18: Charter scope items and their status

Charter item	Delivered	Status
P1-1 Research remediation and LLM patching	design.md, design_rationale.md	Done
P1-2 Analyse CVEs/CWEs, Juliet and CVEfixes	30 Juliet and 6 CVEfixes examples, 21 scored for Task 6	Done
P1-3 Repository, Git, Python, Docker, AFL++ / exploit testing	Two repositories, Docker sandbox, Task 11	Done
P1-4 Architecture and PatchJob contract	PatchJob, later carried inside the shared record	Done
P1-5 Receive and normalise input	Scanner report through intake.py	Done
P1-6 Localise and understand	Tasks 6 and 7	Done
P1-7 Check for existing fixes	Task 8, local patch stores	Done
P1-8 Model-agnostic LLM, Qwen3-Coder	QwenClient through Ollama, Tasks 7 and 9 switch between OpenRouter and the local model	Done
P1-9 Validate in Docker incl. exploit and fuzz	Tasks 10 and 11; C-code test spec, AFL++ image and harness added 15 October (Issue 16, closed)	Done
P1-10 Score and rank with exploit gate	Task 12	Done
P1-11 Human approval with evidence	Task 13 in terminal and dashboard	Done
P1-12 PR flow, deploy simulation, monitoring, logs, rollback	Deploy to test copy with health check and rollback, evidence and logs, no PR flow	Partly
P1-13 Vendor functions	Table 17	Partly
P2-1 Review both architectures, find the hand-off	Shared findings store	Done

P2-2 Agree the structured data	record_contract.md, finding_id and asset_id	Done
P2-3 Map their output to our intake	intake.py	Done
P2-4 Ownership, errors, status reporting	Reason code and owner per stop, status report	Done
P2-5 Test the hand-off end to end	Full run 10 October: one finding from scan to deploy	Done

4.11.2 Reason codes

FIX_CHANGES_SEVERAL_FILES and FIX_FILE_NOT_IN_VERSION moved to the Vulnerability Analysis Team on 14 October, that change is waiting to be committed (Issue 23).

Table 19: Reason codes and who acts on them

Who acts	Reason codes
Vulnerability Analysis Team	NO_FIX_COMMIT, NO_SOURCE_REPO_URL, NO_SOURCE_REF, NO_FILE_PATH, NO_ASSET_ID, UNSAFE_PATH, SOURCE_KIND_UNKNOWN, FIX_COMMIT_NOT_FOUND, FIX_ALREADY_IN_VERSION, FILE_NOT_IN_SOURCE, FIX_CHANGES_SEVERAL_FILES, FIX_FILE_NOT_IN_VERSION, SOURCE_IDENTITY_MISSING
Patching Agent Team	FIRMWARE_NOT_SUPPLIED, SOURCE_NOT_DOWNLOADED, CHECKOUT_WRONG_VERSION, ENGINE_NOT_SET, FIX_CHANGES_NO_CODE_FILE, FILE_IN_SEVERAL_FOLDERS, MODEL_NOT_USED, MODEL_UNAVAILABLE, MODEL_ANSWER_UNUSABLE, NO_LOCATION_FOUND, EXPLANATION_MISSING, LOCATION_MISSING, NO_CANDIDATE, NO_CANDIDATES, NO_VALIDATION_PROFILE, VALIDATION_INCOMPLETE, NO_CANDIDATE_PASSED, VALIDATION_MISSING, RANKING_ERROR, STAGE_NOT_BUILT, STAGE_ERROR
Reviewer	WAITING_FOR_APPROVAL, NO_SINGLE_RECOMMENDATION

5 TEST PLANNING AND RESULTS

Version: 1.0

Date: 21 October 2026

Sponsor: ElectraWireless

Project: Elly Defence - RMIT Group 4

Compiled: Kanishk Sharma

Team: Kanishk Sharma, Hemanth Petlu, Sanskar Pokharel and Anthony MacKenzi Fernando

5.1 VERSION HISTORY

Version	Date	Prepared by	Change
1.0	21 October 2026	Kanishk Sharma	Consolidated team results; added repository coverage for Tasks 7, 13 and 14.

Section 5.7 contains the case results. Section 5.8 identifies the evidence files. Result counts belong to their individual runs.

5.2 INTRODUCTION

5.2.1 Objectives

Verify that the Patching Agent prepares valid inputs, finds or generates candidate fixes, rejects unsafe changes and presents tested candidates for human approval. Verify that the Orchestrator preserves finding identity, records stage outcomes and provides clear reasons when a finding cannot proceed.

5.2.2 Tasks

The testing work covers dataset preparation, sandbox execution, CSV intake, localisation, understanding, patch lookup and generation, candidate validation, security checks, ranking and controlled approval/deployment. Shared integration, runner and dashboard tests extend this coverage. Sections 5.7.11 to 5.7.14 add repository evidence and code-defined coverage for Tasks 7, 13 and 14, with execution status stated separately.

A PASS means the stated assertion passed in the recorded environment. A negative test passes when an invalid input or unsafe candidate is correctly rejected. SKIPPED, NOT RUN and waiting outcomes are not passes. A module being implemented does not mean every finding has successfully traversed it.

5.2.3 Final results at a glance

Scope and owner	Recorded result	Meaning and limit
Tasks 3, 4 and 8 - Kanishk	63 checks passed; 0 failed; 0 skipped	23 dataset, 15 sandbox and 25 lookup checks, grouped into 18 planned cases.
Task 5 - Sanskar	20 cases marked PASS	CSV intake report; no execution date or raw run identifier given.
Task 9 - Sanskar	21 core cases and T9-E2E marked PASS	Generation and syntax checks, not deployment or exploit validation.
Tasks 10 to 12 - Kanishk	71 checks reported, including 7 live Docker checks	Development summary; no independent per-task count supplied.

Task 9 compatibility - Kanishk	26 checks reported, including Docker handoff	Separate contract suite; may overlap other checks.
Task 6 - Hemanth	4 passing cases plus 1 accuracy measurement	21/21 functions; 10/21 exact lines. Accuracy is not a 100% pass claim.
Integration - Hemanth	10/10 PASS on 8 October	Supersedes earlier incomplete integration runs, not proof all 49 findings patched.
49 finding run - Hemanth	1 deployed; 1 unsafe candidate blocked; 10 waiting; 37 before Task 6 sent back to Vulnerability Team	All outcomes recorded; component deployment only.
Deployment demo - Kanishk	Deployment and forced-failure rollback verified	Separate controlled runs with saved model candidates.

5.3 TEST ITEMS

Item	Implementation or input	Version control
Datasets and sandbox	patch-agent/tests and datasets/iotgoat	Case fixtures pinned to IoTGoat f67b7f961301d7a56b435fd7cffac73600f0c97b.
Task 5 and Task 9	CSV intake and candidate generator	Sanskar reports describe tested functionality; exact commit/date not stated.
Validation and ranking	patch-agent/pipeline/test_validate and scoring modules	Development checks and controlled runs recorded separately.
Orchestrator	orchestrator/pipeline/pipeline.py and integration runner	8 October evidence identifies commit 13ec737; later runs are separate snapshots.
Runner and dashboard	start_elly.ps1, runner configuration and command queue	19 October retests recorded; release/merge status requires confirmation.

5.4 FEATURES TO BE TESTED

Feature	Required behaviour	Risk
Input and identity	Reject malformed inputs and preserve the finding identity	Wrong vulnerability or source patched
Source and patch compatibility	Match file, revision, checksum and scope	Applying a fix to the wrong code
Security validation	Build and normal tests pass; exploit assertion is satisfied	An apparently successful patch leaves an unsafe state
Ranking and approval	Only eligible candidates ranked; approval remains human	Automation bypasses review
Deployment and recovery	Deploy reviewed component; verify health; recover on failure	Unusable or unapproved release
Orchestration and dashboard	Record reasons, restrict actions and preserve scanner evidence	Misleading progress or invalid state change

5.5 FEATURES NOT TO BE TESTED

The completed evidence does not cover production fleet rollout, booting and flashing a complete firmware image, live device login and recovery, platform-wide penetration testing, DoS/DDoS resilience or large-scale performance. Selected CVE-derived examples and module functions were tested, but the planned 10 to 20 case benchmark evaluation was not completed. Dashboard local-request restrictions are not equivalent to authentication and role-based access control.

5.6 APPROACH

5.6.1 Code Inspection

Review source identity, candidate diffs, required metadata, failure paths and stage boundaries. Review generated code separately from syntax checks. Preserve evidence of unsuccessful attempts and avoid widening success criteria merely to make a run pass.

5.6.2 Unit Testing

Temporary CSVs and patch stores exercise validation, duplicate handling, write failures and compatibility rules. Task 6 uses labelled examples with answer-bearing comments removed. Mocked failures are identified as such; they do not demonstrate Docker or model-service availability.

5.6.3 Functional Testing

Docker runs check normal input, deliberate overflow detection and controlled IoTGoat behaviour. Task 10 evaluates each candidate in order and skips later checks after an earlier failure. Integration tests copy the findings store before exercising the real pipeline. Manual approval and controlled health/rollback checks cover the release handoff.

5.6.4 Required Software

Python, project dependencies, Git and Docker are required for the relevant module tests. C examples use compiler and AddressSanitizer tooling. Supported fuzz tests use AFL++. Final local model runs used Qwen3-Coder through Ollama; development generation also used NVIDIA Nemotron through OpenRouter. Model/provider changes must be recorded with each run.

5.6.5 Constraints

Every candidate needs matching source and a suitable validation profile. A package inventory path is not necessarily vulnerable source code. Multiple-file fixes, unavailable revisions, model failures and missing profiles can prevent validation. Candidate syntax or a generated diff alone does not establish security.

5.6.6 Logistics

Use separate stores and output directories, never overwrite original findings. Keep credentials outside version control. Capture the command, branch/commit, model, source revision, profile, candidate IDs, exit codes and evidence directory. Screenshots support the logs but do not replace machine-readable outcomes.

5.6.7 Versioning

Results are historical snapshots. The 19 October runner/dashboard workbook entries update earlier VM results. The recorded addition of a C-code profile on 15 October closes an implementation gap, but does not demonstrate a successful rerun of every previously blocked finding. No new execution was performed to compile this report.

5.7 ITEM PASS AND FAIL CRITERIA

Accept a test only when its expected assertion is supported by the recorded result. A rejected unsafe candidate can make the negative test PASS while the candidate remains FAIL. Skipped exploit checks after a build failure provide no exploit evidence. A syntax check does not prove functional correctness.

5.7.1 Dataset and lookup module cases

Case	Purpose	Recorded assertion	Checks
T3-01	Merge and preserve data	Rows, source code and ordering preserved	2
T3-02	Reject invalid datasets	Invalid columns/fields, duplicate IDs and missing CVEfixes corrected code rejected	9
T3-03	Check IoTGoat source integrity	Pinned fixtures accepted and modified copies rejected	4

T3-04	Repeat output and protect files	Repeatable results, overwrite prevention and failed-write cleanup	5
T3-05	Handle absent files and enrichment	Missing-file errors and optional enrichment handled	3

All five groups PASS in Kanishk evidence. Input consisted of 30 Juliet and six CVEfixes examples, with separate controlled IoTGoat cases. Sample IDs in the dataset are distinct from scanner finding IDs. Evidence K1.

Case	Purpose and recorded result
T8-01	Compatible existing patch takes priority
T8-02	Generated patch can be reused when compatible
T8-03	CVE/fallback identity and credential account scope are matched
T8-04	Component, version, repository and source revision compatibility checked
T8-05	Required patch metadata and provenance checked
T8-06	No match is distinguished from missing source requiring review
T8-07	Missing or malformed stores handled with structured errors
T8-08	Repeated lookup is stable and two findings remain independent

All eight groups PASS; 25 automated checks. Temporary lookup fixtures establish selection behaviour, not successful live upstream discovery or new patch generation. Evidence K1.

5.7.2 Sandbox and controlled IoTGoat cases

Case	Coverage and recorded result
T4-01	Build/run baseline C sample, normal input passes and ASan detects deliberate overflow.
T4-02	Simulated build/run failures and absent or misleading security evidence rejected.
T4-03	Live container restrictions checked: unprivileged execution, networking and source/resource restrictions.
T4-04	Lua and credential baseline, candidate and ineffective-patch controls.
T4-05	Clean reruns and timeout cleanup, image/source integrity checked.

All five groups PASS; 15 automated checks in the recorded Docker run. Evidence K1.

5.7.2.1 T4-04 Lua command execution

Field	Test record
Setup and test data	IoTGoat case-001; finding iotgoat-case-001. CWE-78. Actual Lua code with simulated LuCI functions.
Expected result	Baseline reproduces shell execution; candidate blocks it. Ineffective patch is rejected.
Actual result and status	PASS. Candidate and baseline checks exit 0, ineffective patch exits 1 as expected. Normal route/no-command responses preserved. Candidate disables diagnostic commands, so this does not prove full feature equivalence.
Evidence	K2: run 20260911T052038Z-30b9f8eb, baseline.log, candidate.log, ineffective_patch.log and result.json.

5.7.2.2 T4-04 Credential file validation

Field	Test record
Setup and test data	case-002-credentials; IOTGOAT-0001; CWE-798. Pinned passwd/shadow files.
Expected result	Remove both tested stored credential hashes without changing unrelated rows, reject ineffective patch.

Actual result and status	PASS for integrity, exact application, baseline, candidate and ineffective controls. Password fields locked with !. No device login, provisioning or firmware deployment tested.
Evidence	K3: run 62c10e4784e04bb19ce01092081a77ad.

5.7.2.3 T8-06 Missing source negative control

Field	Test record
Setup and test data	IOTGOAT-0003-NEGATIVE names this_file_does_not_exist.conf.
Expected result	Return a named missing-source error without guessing a location or invoking patching.
Actual result and status	PASS. SOURCE_FILE_NOT_FOUND and REQUIRES_MANUAL_REVIEW; source_path null; no lookup/Docker operations. Expected CLI exit 1.
Evidence	K3: run f0e101a0f9d74a0f8a18b0fb69f307b6.

5.7.3 Task 5 intake results from Sanskar

All 20 cases are marked PASS in the Task 5 report. These IDs refer to that report, not the smaller T5-01 to T5-05 groups in the original weekly plan. The input is iotgoat.csv and output is structured_reports.json. The seven schema columns are id, cwe, source, filename, language, vulnerable_code and fixed_code. Required row values exclude language and fixed_code.

Case	Test	Reported result
T5-01	Valid CSV Schema	PASS
T5-02	Missing CSV Column	PASS
T5-03	Missing Required ID	PASS
T5-04	Missing CWE	PASS
T5-05	Missing Source	PASS
T5-06	Missing Filename	PASS
T5-07	Missing Vulnerable Code	PASS
T5-08	Valid Vulnerability ID	PASS
T5-09	Invalid Vulnerability ID	PASS
T5-10	Non-String Vulnerability ID	PASS
T5-11	CWE Name Mapping	PASS
T5-12	Unknown CWE	PASS
T5-13	Missing Fixed Code	PASS
T5-14	Handling CSV NaN Values	PASS
T5-15	Vulnerability Description Generation	PASS
T5-16	Juliet Description Extraction	PASS
T5-17	Exploit Example Extraction	PASS
T5-18	Valid Row Processing	PASS
T5-19	Multiple Vulnerability Findings	PASS
T5-20	Invalid Row Handling	PASS

The report records fixes for schema validation not being called, inconsistent required-field validation, malformed IDs and NaN values in raw metadata. Retesting is reported. Missing fixed_code is accepted as null at intake so an unresolved finding can proceed. This does not conflict with Task 3 requiring corrected code for its CVEfixes dataset examples.

CWE mapping covers CWE-121, CWE-122, CWE-476 and CWE-798; an unknown CWE keeps its identifier with a null name. Extracting a main function supplies a driver example, not proof that an

exploit executes. Confidence is outside this CSV schema. The report uses id; scanner finding_id preservation must be checked at the integration adapter, not inferred from these CSV tests. Evidence S1.

5.7.4 Task 9 generation results from Sanskar

The report records 21 core cases and T9-E2E as PASS. Configuration: OpenRouter, nvidia/nemotron-3-ultra-550b-a55b:free, three requested candidates, temperature 0.6 and a 60-second request timeout. Output: patches/candidates/generated.json.

Case	Test	Reported result
T9-01	Load Vulnerability Information	PASS
T9-02	Required Field Validation	PASS
T9-03	Existing Generated Candidate Detection	PASS
T9-04	Prompt Context	PASS
T9-05	Candidate Count	PASS
T9-06	Candidate IDs	PASS
T9-07	Code Extraction	PASS
T9-08	Generic Language Fence Extraction	PASS
T9-09	Raw Response Extraction	PASS
T9-10	Unified Diff Generation	PASS
T9-11	Metadata Preservation	PASS
T9-12	Output Record Structure	PASS
T9-13	JSON Output	PASS
T9-14	Append to Existing JSON	PASS
T9-15	Multiple Vulnerability Processing	PASS
T9-16	Complete Candidate Code	PASS
T9-17	Existing Comments Preserved	PASS
T9-18	No Unrelated Changes	PASS
T9-19	Syntax Validation	PASS
T9-20	LLM Request Timeout	PASS
T9-21	Duplicate Generation Prevention	PASS
T9-E2E	Basic Candidate Generation	PASS

T9-E2E ends at candidate generation, syntax validation and stored output. It does not cover Task 10 security validation, human approval or deployment. The report does not give a run timestamp, source commit or raw execution-log reference. Its PASS labels are retained as owner-reported results.

Later integration uses full-file diffs, finding_id, source revision/checksums, duplicate and unchanged-candidate rejection and partial results. The reported 26 compatibility checks are a separate suite. Cache matching by vulnerability ID alone does not establish source compatibility. Raw-response extraction in the earlier generation report must not be treated as the current acceptance rule: later runs rejected unusable or ambiguous responses. Preserve partial usable results instead of requiring every request to succeed. Evidence S2 and K4.

5.7.5 Task 10 validation and Task 11 security checks

Candidate	Apply	Build	Tests	Exploit	Overall
demo-fixed	PASS	PASS	PASS	PASS	PASS
demo-ineffective	PASS	PASS	PASS	FAIL	FAIL

demo-build-failure	PASS	FAIL	SKIPPED	SKIPPED	FAIL
demo-regression	PASS	PASS	FAIL	SKIPPED	FAIL

The recorded demo verifies the ordered checks and stops after failure. Task 11 executes inside Task 10. Known attacks use the matching security assertion; bounded AFL++ checks apply only to supported executable targets. Fuzzing is not applicable to the credential text file. A short fuzz run without a crash is not proof that no vulnerability remains.

The development update reports 71 checks across Tasks 10 to 12, including seven live Docker checks. It does not provide a separate count for each module. Windows newline translation was addressed by writing exact UTF-8 bytes for patch/source content, with regression coverage reported. Evidence K4.

5.7.6 Task 12 scoring and approval handoff

Candidates failing required validation checks are ineligible for recommendation. Passing candidates are ranked using the configured scoring rules; the exact weights must be taken from the tested implementation. The recorded controlled demo produced two equally ranked candidates and presented them for human selection. A favourable score cannot override a failed security check.

In Hemanth's connected credential run, one eligible candidate received a score of 0.438. In Kanishk's separate review demo, the displayed risk value was nan; that display issue must not be described as a valid risk assessment. Ranking and risk display are distinct. Evidence H2 and K5.

5.7.7 Task 6 localisation results from Hemanth

Case	Recorded outcome
T6-01	Accuracy report: 21/21 correct functions, 10/21 exact lines and 10/21 within two lines. No pass threshold specified.
T6-02	PASS: credential sites at shadow lines 1 and 8 found in 3/3 rounds; two-site result flagged for review.
T6-03	PASS: missing file returns NOT FOUND, no fabricated location.
T6-04a	PASS: rc.local lines 4 and 5 identified; both known sites found.
T6-04b	PASS: repeat input stable and different findings remain distinct.

These results use the Task 6 case definitions in Hemanth's evidence, including the split T6-04 variants. They do not establish perfect line localisation. Evidence H1.

5.7.8 Orchestrator integration results

Case	Assertion	8 October
INT-01	Source resolved and finding localised	PASS
INT-02	Original scanner finding preserved	PASS
INT-04	Unavailable source produces named reason	PASS
INT-05	Illegal stage skipping rejected	PASS
INT-06	Findings on same file stay independent	PASS
INT-07	Second run preserves status/history	PASS
INT-09	Task 6 returns each finding's own line	PASS
INT-10	Component record names missing source reference	PASS
INT-11	Pipeline cannot approve or deploy autonomously	PASS
DRY	Dry run leaves all records unchanged	PASS

Final recorded integration result: 10 passed, 0 failed, 0 not run on a copy of 50 records. Earlier runs had 7 passes and 3 not run, or 8 passes and 2 not run, depending on available sources and checkout. These are different runs, not contradictory totals. Evidence H1.

5.7.9 Connected run on 49 findings

Outcome	Count	Interpretation
Deployed	1	Reviewed credential patch staged to a test component and health check passed
Unsafe candidate blocked	1	Empty root password rejected by exploit/security check
Waiting after reaching Task 6	10	Model review or validation settings required
Waiting before Task 6	37	Source/fix information prevented progression

The 48 stopped findings were classified as 30 NO_FIX_COMMIT, 6 NO_VALIDATION_PROFILE, 4 FIX_CHANGES_SEVERAL_FILES, 3 MODEL_ANSWER_UNUSABLE, 2 FIX_FILE_NOT_IN_VERSION, 1 NO_CANDIDATE_PASSED, 1 NO_CANDIDATE and 1 FIX_ALREADY_IN_VERSION. These reason counts describe the recorded run, not current unresolved totals after later changes. Evidence H2.

5.7.9.1 E03 to E05 Validated credential through approval and deployment

Field	Test record
Setup and test data	Finding 1ce6ca579626ca0a; iotgoatuser; credential-file.json profile.
Expected result	Candidate passes required checks; a person approves; component health check succeeds.
Actual result and status	PASS. One of three candidates usable; the other responses were duplicate/unusable. Task 10 checks passed; fuzz skipped as not applicable. Hemanth2025-sss approved, then health check exited 0. No rollback occurred in this run.
Evidence	H2: validation-a6cc215c6d17440095799f7073c204b9; patch-84cdc2d7dfb143509f91a9fdc8a961a9.

5.7.9.2 E06 Unsafe root candidate rejected

Field	Test record
Setup and test data	Finding 42b8a6012f4947f8; root credential; same controlled profile.
Expected result	An empty root password must not qualify for approval.
Actual result and status	Negative test PASS; candidate FAIL. Apply/build/tests passed but exploit check exited 3 with UNSAFE_OR_UNKNOWN. NO_CANDIDATE_PASSED prevented approval.
Evidence	H2: validation-c9020bba322e4466af8c6d3d4dc1d34f.

5.7.9.3 K DEMO Deployment and rollback demonstration

Field	Test record
Setup and test data	13 October saved live-model candidates; run 5a8233c743e1. Human chose one of two tied patches.
Expected result	Successful health check deploys; forced health failure restores previous component state.
Actual result and status	Both scenario verifications PASS: DEPLOYED and ROLLED_BACK. Controlled component copies only; no real firmware flashing. This is separate from Hemanth's successful deployment run.
Evidence	K5: success and rollback Task 14 evidence directories.

5.7.10 Runner and dashboard final checks

Hemanth's 19 October evidence updates the earlier runner tests. Six invalid requests were refused. A valid Accept request recorded approval and queued deployment, but that deployment failed because the gcc:13 image was unavailable. The finding remained approved and nothing was deployed. This is not an end-to-end deployment pass.

When Task 10 evidence was missing, approval was skipped even though the runner job exited successfully. The record stayed unchanged; job completion must not be displayed as approval success. Retry-limit and launcher stand-in tests were reported earlier, but were not rerun in the 19 October set.

Dashboard checks recorded remote Host rejection (403, no writes), a local queue request (200), invalid finding ID rejection (404), approval history in the timeline and redaction of hashes and laptop paths across nine panels. An initial missing-queue-folder error was corrected. Authentication, hosted live updates and a full deployment from this VM were not established. Evidence H3.

5.7.11 Task 7 vulnerability understanding

Repository evidence adds Anthony's Task 7 coverage. The local Orchestration snapshot is commit 570ac4dac29598ffd0d71608fa39176a6eb9a914. Its saved understanding_results.csv contains 21 rows, all with status success. That confirms stored generation outcomes, not independent correctness of every explanation or a newly executed model run.

Case	Repository evidence and interpretation
T7.01	PASS reported in T701.md: source-grounded root cause, vulnerable operation, impact and localised source reference. Saved CSV contains the 21 result rows.
T7.02	PASS reported in T702.md: IoTGoat command-injection explanation and remediation trade-offs. Explanation text is not evidence that intended device functionality was preserved by an executed patch.
T7.03	PASS reported in T703.md: incomplete context requires review without invented repository, commit or fix details. A raw execution trace is not included in the note.
T7.04	PASS reported in T704.md: traceability into remediation. The CSV actually uses id and does not contain finding_id, remediation_id or patch_id columns, complete downstream identity propagation therefore requires adapter/integration evidence.

Evidence A1: patch-agent/pipeline/understand/evidence/T701.md to T704.md, results/understanding_results.csv and input/task6_localized_input.csv. These are stored repository results. Task 7 remains an explanation stage, not patch validation.

5.7.12 Task 13 approval coverage

The repository contains patch-agent/src/approval_gate.py and orchestrator/release/approve.py. The following cases document observed implementation and linked shared-run evidence. They are not a newly passed standalone Task 13 suite. IDs R13-01 to R13-06 are assigned in this report for traceability.

Case	Expected check	Evidence status
R13-01	Show finding, explanation, diff and test outcomes; accept yes/no input.	Code inspected. Human approval observed in the connected credential run and separate demo.
R13-02	Reject input n/no and prevent release of that rejected selection.	Branch present in code, no separate rejection transcript supplied. Execution not verified here.
R13-03	Re-prompt on an unrecognised answer.	Loop present in request_human_approval, execution not verified here.
R13-04	Use human approval in the connected workflow.	approve.py explicitly sets Mode A. Standalone Task 13 also has Mode B, automatic mode is not used by this adapter.
R13-05	Skip a recommendation without matching Task 10 evidence.	Code path inspected; Hemanth's 19 October missing-evidence test records an unchanged finding.
R13-06	Present tied candidates and bind the chosen patch to the decision.	Code inspected, tied selection observed in Kanishk's deployment demo. Git/OS reviewer metadata is an audit record, not authenticated identity.

The standalone approval function displays test results but does not itself enforce a successful validation predicate before accepting a decision. The connected pipeline must maintain that eligibility boundary. The risk display is nan where no risk score exists; Task 12's ranking score must not be relabelled as a risk score. Evidence S3.

5.7.13 Task 14 deployment unit coverage

The file patch-agent/tests/test_task14_deploy.py defines three tests. These use temporary source directories and injected health_runner functions; they are not live Docker/device checks. Their existence and assertions were inspected, but this document does not assign new PASS results because they were not executed during this review.

Test class	Assertion in test code	Status
T1401ApprovalGate	Pending approval raises DeploymentError.	Defined, not rerun
T1402SuccessfulDeployment	Successful health result leaves patched deployment copy, keeps original source unchanged and returns DEPLOYED.	Defined, not rerun
T1403AutomaticRollback	Failed health result restores original file content, sets rollback_triggered and returns ROLLED_BACK.	Defined, not rerun

The test's rollout_stage value of 100pct describes its synthetic job state and does not prove fleet rollout. Shared evidence elsewhere in this report demonstrates a successful component release and a separate forced-failure rollback. Those results must remain distinct from the unexecuted unit suite. Evidence A2.

5.7.14 Repository review boundary

A1, A2 and S3 were inspected on 21 October 2026 in the local Elly-Defense-demo-20261013 checkout at the commit above. The checkout also contains untracked demo/check files, which were not used to establish these three module inventories. This inspection does not establish the latest GitHub state or attribute every test function to a particular author. No model calls, deployment or test execution were performed for this document update.

5.8 TEST DELIVERABLES

Ref	Evidence source	Use
K1	Test_Case_Evidence_Kanishk.xlsx; reports/test-cases/ALL-e0ffa7966c/result.json and run.log	63-check dataset/sandbox/lookup run
K2	datasets/iotgoat/case-001/evidence/20260911T052038Z-30b9f8eb/	Lua baseline, fix and ineffective control
K3	datasets/iotgoat/case-002-credentials/evidence/; run IDs in case records	Credential and missing-source controls
K4	Recorded development updates and Task 10 demo terminal output	71-check and 26-check summaries; ordered validation demo
K5	reports/deployment-demo/5a8233c743e1/	Separate successful deployment and forced rollback transcripts
S1	Task_5_Test_Result.docx - Sanskar	20 owner-reported CSV intake cases
S2	Task_9_Test_Result.docx - Sanskar	22 owner-reported generation cases
H1	Test_Plan_and_Results_Hemanth_Petlu.docx; Test_Case_Evidence_Hemanth.xlsx	Task 6 and integration cases

H2	reports/pipeline_run.json; reports/status_report.md; pipeline_work2 and store_test2 records referenced by Hemanth	49-finding run and credential release
H3	Test_Evidence_Hemanth_logs/runner_checks_19Oct/; runner_missing_evidence_19Oct/; dashboard_checks_19Oct/	Latest runner/dashboard outcomes
P1	Patching Team Test Plan.docx	Original weekly case grouping and ownership
P2	Test Plan.pdf	Document structure only; example-project results not reused
A1	patch-agent/pipeline/understand/evidence/T701.md to T704.md; results/understanding_results.csv	Saved Task 7 notes and 21 successful status rows
A2	patch-agent/tests/test_task14_deploy.py	Three code-defined unit tests; not rerun
S3	patch-agent/src/approval_gate.py; orchestrator/release/approve.py	Approval implementation and Orchestrator adapter review

The workbooks and owner reports record evidence references; raw files are not all embedded here. K4 and K5 retain development updates and terminal transcripts. A1 includes saved repository outputs; A2 and S3 include code inspection, with shared execution evidence identified separately. Suites overlap and use different groupings, so no combined unique-test total is asserted.

5.9 TESTING TASKS

5.9.1 Reproduce the module checks

Run from the Elly-Defense repository root with project dependencies installed. Commands below are the recorded test entry points; confirm they exist in the selected checkout. Docker must be running for live sandbox cases.

```
python3 patch-agent/tests/run_test_case.py T3

python3 patch-agent/tests/run_test_case.py T4 --docker

python3 patch-agent/tests/run_test_case.py T8

python3 patch-agent/tests/run_test_case.py ALL --docker

python3 patch-agent/datasets/iotgoat/case-002-credentials/run_case.py

python3 patch-agent/datasets/iotgoat/case-002-credentials/run_case.py --negative
```

Without --docker, the live cases can be skipped; do not count those as passes. Sanskar's Task 5 report leaves its script path as a placeholder, so no executable path is invented here. Use the release guide for the current intake/generation entry points.

5.9.2 Reproduce integration checks

From the Orchestration checkout, point PATCH_AGENT at the intended engine and supply the extracted firmware and source cache. Replace the placeholder values with the corresponding paths for the selected vulnerability and test case. The validation profile must match the source code and candidate patches being evaluated.

```
python orchestrator/pipeline/pipeline.py --stages

python orchestrator/outtake/run_integration_tests.py --firmware-dir <extracted_firmware> --source-cache <source_cache>
```

Use the release FULL_RUN_GUIDE.md for model configuration, runner startup and approval/deployment commands. The historical deployment demo helpers were local artifacts and

are not assumed to exist in the released repository. Do not automatically approve findings simply to complete a test.

5.10 ENVIRONMENTAL NEEDS

5.10.1 Server Requirements

A development laptop with project dependencies, Docker support and adequate resources for the selected local model. Windows and macOS runs are distinct environments. Source cache, firmware extraction and writable evidence directories are required. A missing Docker image blocks that scenario and must be reported.

5.10.2 Testing Frameworks

Python test runners, Docker, Git patch application, compiler/AddressSanitizer checks, Lua component assertions and supported AFL++ harnesses. Dashboard/runner checks additionally require the configured inbox and local service.

5.10.3 Test Data

30 Juliet and six CVEfixes examples; separate pinned IoTGoat Lua and credential fixtures; labelled Task 6 samples; copied 50-record integration store; fresh 49-finding scanner report. Preserve the source image hash and source revision for each run. Do not combine findings from different firmware copies without checking their identity.

5.11 RESPONSIBILITIES

Member	Testing contribution
Kanishk Sharma	Tasks 3, 4 and 8; validation/security/ranking checks; generation compatibility; controlled deployment/rollback demonstrations; consolidation of evidence.
Hemanth Petlu	Task 6 accuracy and controls; Orchestrator stage/identity integration; connected findings runs; runner/dashboard checks and evidence.
Sanskar Pokharel	Task 5 intake and Task 9 generation reports; Task 13 approval implementation reviewed alongside shared approval evidence. No new standalone approval suite result is asserted.
Anthony MacKenzi Fernando	Task 7 understanding and Task 14 deployment work. Saved understanding notes/results and deployment unit-test coverage are included, with shared release evidence distinguished from unexecuted tests.
Team and reviewer	Review results against the release; retain failures and limitations; human approval remains separate from automated validation.

5.12 SCHEDULE

Period	Recorded activity
September	Dataset, sandbox, lookup and controlled IoTGoat tests; candidate validation and generation compatibility work.
6 to 8 October	Task 6 rerun and integration suite completion.
9 to 10 October	49-finding connected run; credential approval and component deployment; unsafe root patch blocked.
13 October	Separate component deployment and forced health-failure rollback demo.
15 to 19 October	Profile and runner/dashboard updates; latest VM/request and redaction checks.
21 October	Evidence consolidation and issue-status update. No new test execution implied.

5.13 RISKS AND CONTINGENCIES

Remaining risk	Required action or limit

Missing source and multi-file fixes	Obtain exact source/file context; support the appropriate patch scope. Do not guess a vulnerable location.
Coverage beyond credentials	C profile addition recorded on 15 October; rerun affected findings with matching build, normal and security tests before claiming success.
Model response variability	Retain rejected/partial candidates, provider details and failure reasons. Retry does not guarantee a usable fix.
Docker availability	Provision required images and rerun blocked deployment checks.
Two findings change one file	Test deployment ordering and conflict handling for separately approved patches.
Dashboard security and release state	Confirm authentication and deployment model. Verify runner/dashboard files are in the selected release; hosted snapshot is not a live system.
Benchmark and regression coverage	Plan measured multi-case evaluation and a clean-checkout regression run. Keep blocked and skipped cases visible.

5.14 APPROVALS

Prepared by Kanishk Sharma for the Patching Team test evidence handover. Technical review should confirm the final release, evidence links and any changed results.

Role	Name or group	Status
Preparer	Kanishk Sharma	Compiled 21 October 2026
Technical reviewers	Hemanth Petlu, Sanskar Pokharel and Anthony MacKenzi Fernando	Review to be recorded
Academic mentor	Dr Attiq Ur Rehman	Acknowledgement to be recorded
Client acceptance	ElectraWireless nominated reviewer	Decision to be recorded

6 DEVELOPMENT GUIDE

6.1 PROJECT SUMMARY

6.1.1 Purpose and objectives

ElectraWireless offered the project to investigate whether model-assisted patching could shorten the path from a firmware vulnerability finding to a tested fix while keeping a person responsible for each release. The objectives were to build the patching stages, connect them to the Vulnerability Analysis Team's scanner, validate candidate changes in Docker and prevent deployment before human approval.

The prototype accepts structured findings, resolves matching source, localises and explains the issue, looks for an existing fix or generates candidates, validates and ranks them, and presents evidence for approval. An approved candidate can be deployed to a test copy with a health check and automatic recovery on failure.

6.1.2 Release package and repositories

Elly-Defense-Orchestration orchestrator-dev	Connected workflow, shared records and copied patch-agent engine. Use the release working copy for the full runner/dashboard handover.
Elly-Defense main	Task implementation and review history. A merge here does not automatically update the engine copy in Orchestration.
Separate Next.js dashboard project	Obtained from the dashboard owner. Integration code is under lib/patching/ and app/api/patching/.
Release source ZIP	ProjectID-source.zip holds both repositories: Elly-Defense-Orchestration/ (the release working copy, including the runner, dashboard integration and reason-code changes that are not yet committed, Issue 23) and Elly-Defense/ (main at commit 1556b09, 10 October 2026, committed files only). ElectraWireless approved submitting the source code with the reports for audit.

Repositories: https://github.com/ElectraWireless/Elly-Defense-Orchestration and https://github.com/ElectraWireless/Elly-Defense. Elly-Defense-Orchestration and Elly-Defense are private repositories in the ElectraWireless GitHub organisation. Access is granted by ElectraWireless, so readers outside the project cannot open them. Both repositories are in ProjectID-source.zip, submitted with this report with ElectraWireless's approval for audit. When working from a Git checkout, record git rev-parse HEAD and git status --short for each one. Do not describe a fixed historical commit as the current complete release.

6.1.3 Scope and known boundaries

Task 3 prepared 30 Juliet and six CVEfixes samples, with separate IoTGoat cases. Task 4 established sandbox execution. The connected registry runs Tasks 6-12; Task 11 executes inside Task 10. Task 5's standalone intake logic and Orchestrator intake are distinct interfaces. Tasks 13 and 14 remain outside the unattended stage registry.

The release does not provide whole-firmware flashing, a device fleet rollout, dashboard authentication or a manual rollback command. C-code findings need a program-specific test spec, AFL++ image and harness; these were added on 15 October (Issue 16, closed), but a full rerun of every previously blocked C-code finding has not been recorded. A general validation engine does not imply complete component coverage.

6.1.4 Process flow and files

Scanner handoff	orchestrator/intake/intake.py	One tracked record per finding.
Source resolution	orchestrator/contracts/sources.py	Matching firmware file or upstream source; otherwise a named stop.

Task 6	patch-agent/pipeline/localize/	Affected source lines and any review flag.
Task 7	patch-agent/pipeline/understand/	Explanation tied to localisation.
Task 8	patch-agent/pipeline/check_existing/	Compatible existing patch or generation required.
Task 9	patch-agent/pipeline/generate/	Candidate diffs with source identity.
Tasks 10 and 11	patch-agent/pipeline/test_validate/	Application, build, normal tests and exploit/fuzz results.
Task 12	patch-agent/pipeline/score_rank/	Eligible ranking or tie for review.
Task 13	orchestrator/release/approve.py	Named human acceptance or rejection.
Task 14	orchestrator/release/deploy.py	Test-copy deployment, health result and automatic rollback on failure.
Reporting	orchestrator/outtake/status_report.py	Totals, stopping reason, responsible team and next action.
Runner	orchestrator/runner/	Inbox ingestion, serial jobs, bounded retries and dashboard requests.

orchestrator/pipeline/pipeline.py coordinates adapters in pipeline/stages/. The registry is stages/__init__.py. Engine modules implement task behaviour; adapters translate the shared record and report results. Approval and deployment are invoked separately or by the runner after a valid human decision.

6.1.5 Record and evidence ownership

The store is a local folder of UTF-8 JSON files named <finding_id>.json. Intake copies the scanner payload; later patching stages preserve it. Intake may refresh details only while a record is still new. Lifecycle updates pass through store.update_status() and lifecycle.py. Do not edit the store while a run is active.

Keep finding_id, candidate identity, source revision/checksum, validation evidence and approval together. Reports describe the latest store state; save run-specific copies before another run replaces summary files. Model output, a passing build and a completed test process are not interchangeable with a validated fix.

6.2 SETUP CLIENT AND DEVELOPER MACHINE

6.2.1 Prerequisites

Windows	Windows 10 or 11, 64-bit. PowerShell is the supported start-script route.
Memory and disk	32 GB RAM and at least 40 GB free disk recommended. The TSD estimates about 24 GB free RAM during model use and about 19 GB for the model download.
Tools	Git, Python 3.10+, Docker Desktop, Ollama, VS Code, Node.js and npm for the dashboard.
Access	GitHub organisation access to clone private repositories. No model API key for the local release route.
Dashboard	Obtain the separate integrated Next.js project from its owner; a pipeline checkout alone does not provide it.

6.2.2 Acquire and identify the code

The repositories are private to ElectraWireless, so the clone below works only for an account that ElectraWireless has given access to. Without access, extract ProjectID-source.zip instead and skip the clone: both repositories are in it

```
git clone --branch orchestrator-dev https://github.com/ElectraWireless/Elly-Defense-Orchestration.git
Set-Location Elly-Defense-Orchestration
git branch --show-current
git rev-parse HEAD
git status --short
```

For the complete release, use the handed-over source package if the required changes are not yet committed. Keep the ZIP extraction and existing Git checkout in separate folders; do not overwrite local work. If a folder already exists, inspect it instead of cloning over it. Git commands require a checkout containing .git; an extracted source ZIP may not have Git history.

```
Test-Path orchestrator\pipeline\pipeline.py
Test-Path orchestrator\runner\start_elly.ps1
Test-Path orchestrator\runner\runner.config.example.json
```

If either runner file is absent, obtain the release working copy or the reviewed commit containing it. The manual pipeline route remains available only to the extent implemented in that checkout.

6.2.3 Create a repository Python environment

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install openai python-dotenv pandas
python --version
git --version
docker info
node --version
npm --version
```

In VS Code, select this repository's .venv interpreter. If activation is blocked, invoke .\.venv\Scripts\python.exe directly. The runner must use this interpreter or another environment with the same dependencies. Scanner execution also requires its own requirements and Linux/WSL extraction tools.

6.2.4 Configure the local model

```
ollama pull qwen3-coder
setx OLLAMA_CONTEXT_LENGTH 32768
$env:OLLAMA_CONTEXT_LENGTH = "32768"
$env:OLLAMA_MODEL = "qwen3-coder"
ollama list
```

Quit Ollama completely and restart it after changing the context setting. setx affects future processes; an already-running server does not inherit it. If starting the server manually, run ollama serve in a separate terminal with the environment set. Do not start a second server when the app already owns its port.

The larger context reduces truncated or unusable answers; it does not guarantee a valid response. MODEL_ANSWER_UNUSABLE can still require review or a retry. The model must fit the machine's memory. Record any replacement model because it changes the experiment.

Use --local-model on pipeline commands so Tasks 7 and 9 use the same local Ollama route as Task 6. Qwen3-Coder through Ollama is the final-release configuration. OpenRouter is an optional hosted route, not required for the documented run. Enabling it sends supplied input to an external service and requires a separate client decision and configuration.

6.2.5 Create the work directories

```
$env:ELLY_WORK = "C:\elly"
$env:PATCH_AGENT = Join-Path (Get-Location) "patch-agent"
$env:ORCHESTRATOR_STORE = Join-Path $env:ELLY_WORK "store"
$folders = @("store", "inbox", "runner", "extracted_firmware",
        "source_cache", "validation_profiles", "pipeline_work")
foreach ($folder in $folders) {
  New-Item -ItemType Directory -Force `
    -Path (Join-Path $env:ELLY_WORK $folder) | Out-Null
}
$env:PATCH_AGENT
$env:ORCHESTRATOR_STORE
```

Set PATCH_AGENT and ORCHESTRATOR_STORE in every new manual-run terminal. These choose the engine and records actually used. For a private experiment, select a separate store; never reset the shared store to obtain a clean demo.

store	One JSON record per finding.
inbox	New scanner report files for the runner.
runner	requests/, jobs/, logs/ and state.json.
extracted_firmware	One extracted filesystem folder per asset_id.
source_cache	Upstream source at matching revisions.
validation_profiles	Reviewed per-finding profile.json and checks.
pipeline_work	Task output and deployment evidence.

6.2.6 Prepare Docker and firmware

```
docker build -t elly-iotgoat-component:local `
  patch-agent\datasets\iotgoat\case-001
docker pull gcc:13
docker image inspect elly-iotgoat-component:local
docker image inspect gcc:13
```

The component image provides Lua 5.1 and Git for the supplied profile. gcc:13 is required by the release deployment health-check runtime. Start Docker Desktop before these commands. Record image IDs and architecture for repeatable runs; moving tags can change the runtime.

Build-time package downloads are separate from runtime network access. Retain the configured timeouts, resource limits and privilege restrictions. Keep trusted checks separate from candidate source. A missing image, timeout or Docker startup error is not a successful security check.

The firmware archive is vuln-scanner/firmware/IoTGoat-x86.img.gz. Unpack it using the scanner team's documented Linux/WSL setup with binwalk, or use the matching extracted folder supplied by that team. Creating an empty directory is not extraction. Follow vuln-scanner/README.md for the scanner's --keep route and dependencies.

```
$shadow = Join-Path $env:ELLY_WORK `
  "extracted_firmware\asset_13f9a5407a85\squashfs-root\etc\shadow"
Test-Path $shadow
if (Test-Path $shadow) { Get-FileHash $shadow -Algorithm SHA256 }
Get-FileHash vuln-scanner\firmware\IoTGoat-x86.img.gz -Algorithm SHA256
```

Compare like-for-like hashes with the team: the compressed archive, decompressed image and extracted file each have different hashes. Use the report's actual asset_id for other cases. An extracted filesystem is not a running firmware VM, and a component container is not a full firmware boot.

6.2.7 Set reviewer identity

```
$Reviewer = Read-Host "Enter your own reviewer name"
git config user.name "$Reviewer"
git config user.name
```

This sets repository-local Git identity. For a source ZIP without .git, configure your own global Git name if required by approve.py. The recorded name supports attribution but is not authenticated login or role-based access control.

6.2.8 macOS and Linux alternative

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install openai python-dotenv pandas
export PATCH_AGENT="$PWD/patch-agent"
export ELLY_WORK="$HOME/elly"
export ORCHESTRATOR_STORE="$ELLY_WORK/store"
export OLLAMA_CONTEXT_LENGTH=32768
export OLLAMA_MODEL=qwen3-coder
```

Use forward-slash paths and Bash continuations in that shell, and restart Ollama with the setting applied. start_elly.ps1 is the Windows launcher; use the manual commands for other platforms. Do not mix Windows and WSL paths in a single command.

6.3 CODE AND COMMENT STANDARD

6.3.1 Python and documentation conventions

Use clear snake_case function and variable names, short modules with one responsibility and explicit imports. Put a short module docstring at the top explaining purpose, inputs, outputs and how to run it. Document public functions and non-obvious security decisions. Comments should explain why a check exists rather than repeat the statement below it.

Use pathlib for paths and parameterise workspace locations. Do not embed a developer's home directory or a virtual environment from another checkout. Preserve exact patch bytes across platforms: write_bytes(text.encode("utf-8")) avoids Windows newline translation when writing a generated diff. Preserve the expected encoding and newline semantics of target files.

Validate input shape, source identity and required files before running commands. Use argument lists for subprocess calls, explicit timeouts and captured output. Return a structured error or waiting reason for missing dependencies. Do not swallow exceptions, fabricate locations or count an unexecuted check as passed.

Keep scanner data immutable after intake, preserve finding_id and retain evidence for failed candidates. Avoid logging API keys, password hashes or full workstation paths in shared screenshots. Follow the client's local-model default and review third-party dependencies and licences before adding them.

6.3.2 Adding or changing a task

Agree the input fields, lifecycle states, dependencies, output evidence and stopping conditions first. The engine owns the task logic. The Orchestrator adapter translates the record, invokes that logic and records its result. Use the existing Task 10 and Task 12 adapters as concrete examples rather than creating another pipeline inside a task.

```
patch-agent/pipeline/new_stage/__init__.py
patch-agent/pipeline/new_stage/run_stage.py
patch-agent/tests/test_new_stage.py
patch-agent/docs/New_Stage.md
orchestrator/pipeline/stages/taskNN_new_stage.py
```

The paths above illustrate the structure for a new stage; they are not existing release files. Tasks 10 and 12 expose run(record). Other stages have their own interfaces, which their adapters handle. Do not impose a new common result shape on existing modules without updating their callers and tests.

```
def run(record: dict) -> dict:
  """Assess one finding and return its result and evidence."""
  # Follow the existing task contract and preserve finding_id.
  ...
```

Register the adapter in orchestrator/pipeline/stages/__init__.py with its Stage definition. Check the adapter's record, mapped and ctx inputs and its result/next_status outputs against neighbouring stages. Coordinate record-schema and lifecycle changes with both teams. Keep approval and deployment outside the unattended stage list.

6.3.3 Testing a task change

Cover valid input, malformed input, identity mismatch, missing source, dependency failure, partial results and repeat runs. Check that the scanner payload is preserved and that an invalid lifecycle move is refused. For code execution, add relevant Docker coverage and retain actual command logs. A mocked unit test is useful but does not demonstrate live sandbox execution.

For candidate generation, test full-file diffs, unchanged and duplicate rejection, source checksums and partial model failures. For validation, test an effective fix, an ineffective fix, build failure and a normal-function regression. For ranking, confirm failed mandatory checks exclude a candidate and ties remain available for human review.

Update the task documentation, dependency requirements, output contract and relevant test plan. Run the affected module tests and connected integration checks before submitting a pull request. Record skips, timeouts, expected negative cases and missing-profile blockers explicitly.

6.3.4 Validation profiles and security assertions

A profile must match the source and candidate files. It supplies trusted build, normal-test and exploit/fuzz commands, runtime image and limits. A credential-file profile is not a test profile for a C application. Program-specific tests may need the full source tree and build dependencies, not only the affected file.

The release's known-exploit route requires evidence that the original reproduces the issue and that the candidate blocks it. Expected markers include VULNERABILITY_REPRODUCED for the original and EXPLOIT_BLOCKED for the patched case. Inspect the command result and assertions together; a crash or a timeout must not be misread as a blocked exploit.

When using AFL++, provide a suitable instrumented target, harness and seed input, and run for the configured fixed duration. A short run with no new crash is limited evidence. A test spec, AFL++ image and harness for the C-code findings were added on 15 October (Issue 16, closed); other programs need their own.

6.3.5 Review and evidence checklist

Source identity	Finding ID, repository/reference, source checksum and selected profile.
Execution	Python and Docker versions, image IDs, exact commands and timeout settings.
Validation	Per-candidate apply/build/tests/security outcomes and logs.
Ranking	Eligible candidates, exclusions, scores and any tie.
Release decision	Reviewer, selected candidate, decision and evidence binding.
Recovery	Health result, deployment directory and restoration evidence on failure.

Do not modify tests merely to let a candidate reach approval. If a test assertion is wrong, correct and review the assertion, rerun the original baseline and candidate, and retain the earlier evidence so the change is traceable.

6.4 VERSION CONTROL POLICY

6.4.1 Branch from the correct repository

Engine changes originate in Elly-Defense, normally from main. Orchestrator adapters, source handling, lifecycle and runner work belong in Elly-Defense-Orchestration, normally from orchestrator-dev until the team agrees a new base. The dashboard is a separate project. Confirm the remote and branch before editing.

```
git remote -v
git status --short
git fetch origin
git switch orchestrator-dev
git pull --ff-only origin orchestrator-dev
git switch -c feature/validation-profile-update
```

Preserve existing local work before switching or updating. Use descriptive feature/, fix/, test/ or docs/ branch names. A source ZIP is not a Git checkout: apply reviewed release changes to the correct repository before committing them.

6.4.2 Commits and pull requests

```
git diff
git diff --check
git add -p -- patch-agent orchestrator
git status --short
git diff --cached
git commit -m "Align validation profile handling with the pipeline"
git push -u origin feature/validation-profile-update
```

Add new files explicitly by their actual paths. Include regression tests alongside the fix. Keep independent changes in separate commits. Do not commit .env, runner.config.json, secrets, firmware downloads, virtual environments or generated evidence unless a reviewed, sanitised fixture is intentionally required.

A pull request should explain the problem, resulting behaviour, validation commands and results, skipped checks, dependencies and limitations. Confirm the base branch and request the designated review. Code-merge approval is separate from approval to deploy a generated patch.

6.4.3 Synchronisation and conflicts

A merged engine change does not update the Orchestration engine copy automatically. Compare file contents and revisions, perform a reviewed sync, confirm PATCH_AGENT selects that copy and rerun the affected integration checks. Release Issue 23 records that some working-copy changes still awaited this process.

Keep both relevant .gitignore patterns when they protect different outputs. Resolve code conflicts by understanding both behaviours; accepting both may duplicate logic. Run git diff --check and relevant tests after resolution. Avoid force-pushing shared branches or rewriting another member’s history without agreement. Final release integration and acceptance remain with the client’s designated lead.

6.5 GETTING STARTED

6.5.1 Load the report and prepare sources

Run from the selected Orchestration root with .venv active and PATCH_AGENT/ORCHESTRATOR_STORE set. The example uses the supplied report and credential finding. Live model output can differ between runs; the recorded release success is not a guarantee of the same result today.

```
python orchestrator\intake\intake.py vuln-scanner\output\report.json
python orchestrator\contracts\fetch_sources.py `
  --cache "$env:ELLY_WORK\source_cache"
```

Inspect created, updated and skipped counts. Intake updates only findings that have not started; an in-progress store is not a fresh run. Source downloads must match the affected component and revision. An inventory file or changelog is not necessarily the vulnerable source. Missing or ambiguous source details must remain a named stop.

6.5.2 Create the credential profiles

```
$FindingId = "1ce6ca579626ca0a"
Test-Path "$env:ORCHESTRATOR_STORE\$FindingId.json"
python orchestrator\validation\make_profile.py `
  --record $FindingId `
  --spec orchestrator\validation\specs\credential-file.json `
  --param account=iotgoatuser `
  --firmware-dir "$env:ELLY_WORK\extracted_firmware" `
  --out "$env:ELLY_WORK\validation_profiles"
```

For the root finding in the same report, use --record 42b8a6012f4947f8 and --param account=root with the same source/profile directories. Confirm both IDs in the selected report. The profile generator refuses to overwrite an existing profile; do not remove reviewed evidence just to rerun it.

The credential build step is a structural parse, not compilation of a C program. The normal tests check preservation and the security assertion checks credential content. They do not replay a network login. Other vulnerabilities require their own trusted profile, source tree and security assertion.

6.5.3 Check stage availability

```
python orchestrator\pipeline\pipeline.py --stages
```

Expect Tasks 6-12 to be available in the connected release, with Tasks 13 and 14 shown separately. The phrase “6 of 6 pipeline stages run end to end” describes wiring, not six successful security checks or successful remediation of every finding. Check the printed engine path if availability is unexpected.

6.5.4 Stage availability example

```
========================================================================
PIPELINE STAGES
========================================================================
engine: [selected checkout]/patch-agent

  Task 6   localize         AVAILABLE
           Find the line that causes the defect
           runs on: new

  Task 7   understand       AVAILABLE
           Explain the defect for the reviewer
           runs on: localized

  Task 8   check_existing   AVAILABLE
           Is there already a fix for this?
           runs on: localized

  Task 9   generate         AVAILABLE
           Generate candidate patches (again if none passed)
           runs on: localized, matched_with_patch

  Task 10  validate         AVAILABLE
           Build, test and replay the exploit (tasks 10-11)
           runs on: matched_with_patch

  Task 12  score            AVAILABLE
           Rank the candidates that passed
           runs on: matched_with_patch

------------------------------------------------------------------------
AFTER APPROVAL -- shown, never run by this pipeline
------------------------------------------------------------------------
  Task 13  approval         present
           A person approves or rejects
           run by a person with orchestrator/release/approve.py

  Task 14  deploy           present
           Deploy, health-check, roll back on failure
           only after approval, with orchestrator/release/deploy.py

6 of 6 pipeline stages run end to end.
A finding travels as far as they allow, then waits.
```

Figure 18. Captured --stages output from the connected local checkout, 23 October 2026. The laptop path is redacted. This command checks registration and availability; it does not execute the model, validation or deployment.

6.5.5 Run the selected finding and read the report

```
python -u orchestrator\pipeline\pipeline.py `
  --record $FindingId --local-model `
  --firmware-dir "$env:ELLY_WORK\extracted_firmware" `
  --source-cache "$env:ELLY_WORK\source_cache" `
  --validation-profiles "$env:ELLY_WORK\validation_profiles" `
  --work-dir "$env:ELLY_WORK\pipeline_work"
python orchestrator\outtake\status_report.py
Get-Content reports\status_report.md
```

Use --all instead of --record $FindingId to process the full selected store after checking the single-record setup. Do not supply both. The --local-model option keeps all model stages on the documented local route. Proceed to Deployment only when the record is ready for human review.

Read each stage result and its reason code. An incomplete generation may still contain usable candidates; each still needs validation. A completed validation run may contain failed candidates. Missing profiles remain blockers. Ranking can exclude all candidates or leave a tie. Save reports/pipeline_run.json and the Markdown/CSV status report with the run evidence.

```
# Patching status report

Generated: 2026-10-10 02:47 UTC
Findings: 49

## Totals

| Outcome | Findings |
|---|---:|
| Stopped between Task 6 and Task 12 | 12 |
| Waiting before Task 6 | 37 |
```

Figure 19. Excerpt of an actual status_report.md generated on 10 October 2026 at 02:47 UTC during an earlier no-model run. All 49 findings stopped; this is a report-reading example, not the later release outcome. The original laptop store path is omitted.

The later release report records 1 deployed, 1 unsafe candidate blocked, 10 waiting between Tasks 6-12 and 37 returned before Task 6. These are historical results from release.txt. Do not combine them with the earlier report screenshot into a single run.

6.5.6 Dashboard review example

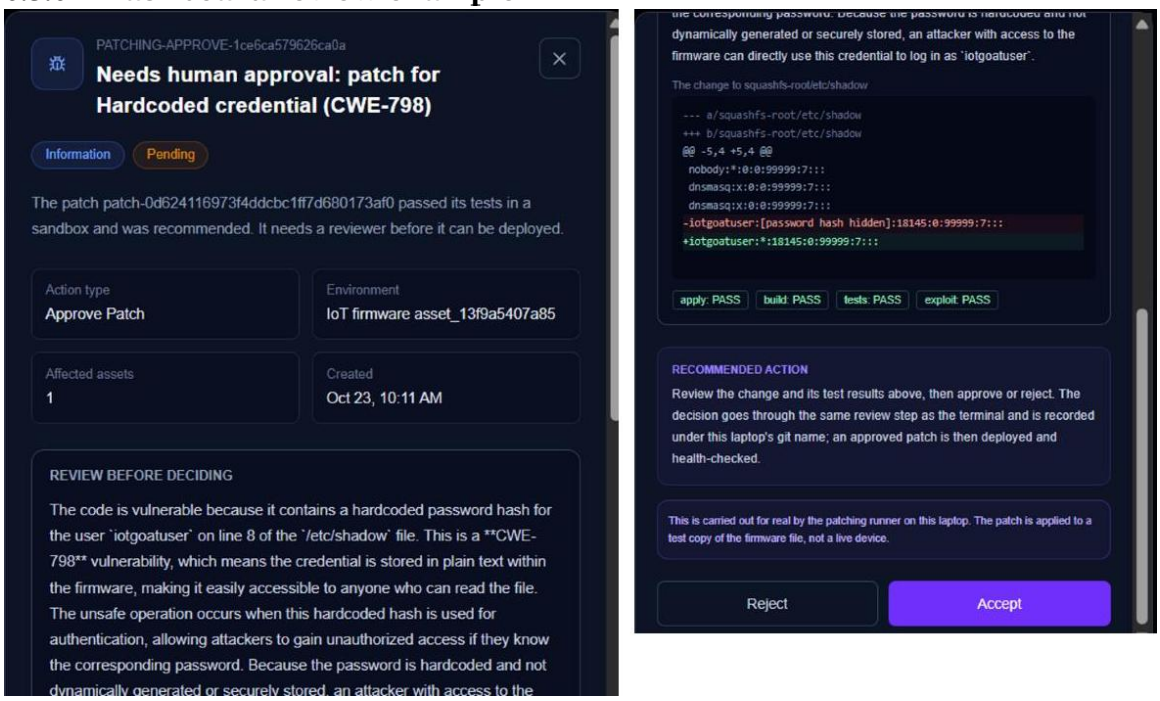


Figure 20. Dashboard review window for finding 1ce6ca579626ca0a on 23 October 2026 (left: top of the window, right: further down): the change with the password hash hidden, the four test results, and the Reject and Accept buttons. Hemanth clicked Accept; the runner then approved and deployed the patch with a passing health check (ProjectID-demo.pdf, section 4).

Compare the displayed finding ID, explanation, candidate diff and validation evidence before using Accept or Reject. The runner must be active on the same laptop and configured for the same store and queue. After a decision, confirm the resulting job and record state rather than relying only on the button response.

The dashboard also shows patching events, timelines and service health. Some other panels retain the dashboard's sample data; items without a patching finding ID should not be reported as scanner results. Password hashes and workstation paths are hidden in the integrated view.

If System Health shows Ollama or Docker unavailable, start the missing service and check again. A stored result remains historical evidence even while a service is down; it does not mean a new run can execute. Wider access requires authentication work before operational use.

6.5.7 Module and integration test commands

Run selected tests during development and the affected integration checks before review. These commands do not imply a predetermined pass count. Confirm test discovery found the intended cases and record any skips or optional Docker checks.

```
python patch-agent\tests\run_test_case.py T3
python patch-agent\tests\run_test_case.py T4 --docker
python patch-agent\tests\run_test_case.py T8
python patch-agent\tests\run_test_case.py T8-04
python patch-agent\tests\run_test_case.py T4-04 --docker
```

T3 covers dataset integrity and preparation; T4 covers sandbox behaviour and supported component cases; T8 covers patch lookup, source compatibility and identity. ALL in this runner means its registered cases, not every project task.

```
python -m unittest discover -s patch-agent\tests -p "test_task9*.py" -v
python -m unittest discover -s patch-agent\tests -p "test_task10*.py" -v
python -m unittest discover -s patch-agent\tests -p "test_task11*.py" -v
python -m unittest discover -s patch-agent\tests -p "test_task12*.py" -v
python -m unittest discover -s patch-agent\tests -p "test_task14*.py" -v
python -m unittest discover -s patch-agent\tests -p "test_stage_adapters.py" -v
```

Enable optional live checks only where the selected test module supports ELLY_RUN_DOCKER. The variable does not make every test a Docker test.

```
$env:ELLY_RUN_DOCKER = "1"
python -m unittest discover -s patch-agent\tests -p "test_task10*.py" -v
Remove-Item Env:ELLY_RUN_DOCKER
python orchestrator\outtake\run_integration_tests.py --help
python orchestrator\outtake\run_integration_tests.py
```

Integration tests use a throwaway copy of records. Source-dependent cases may be NOT RUN without the matching firmware/source settings; use the options listed by that checkout's --help. A passing lifecycle check does not show that every finding was patched.

Task 6's dedicated runner is patch-agent/pipeline/localize/run_task6_tests.py; inspect its --help and supply its required source checkout. Test Task 7's connected input/output in the pipeline, and test Task 13 through separate controlled acceptance and rejection cases. The old private deployment-demo helpers are not part of this release guide.

6.5.8 Standalone validation and ranking

The tracked Task 10 demo supplies matching Lua source, checks, profile and four controlled candidates. It makes no model call. Check the helper exists in the selected engine before using it.

```
Test-Path patch-agent\examples\run_task10_demo.py
$prep = python patch-agent\examples\run_task10_demo.py --prepare-only
$prep
$demoLine = $prep | Where-Object { $_ -like "Demo inputs: *" } |
  Select-Object -Last 1
if (-not $demoLine) { throw "Demo preparation returned no input path" }
$DemoDir = $demoLine.Substring("Demo inputs: ".Length).Trim()
if (-not (Test-Path "$DemoDir\profile.json")) {
  throw "Prepared validation profile is missing"
}
python patch-agent\pipeline\test_validate\validate_pipeline.py `
  --source "$DemoDir\source" --checks "$DemoDir\checks" `
  --profile "$DemoDir\profile.json" `
  --candidates "$DemoDir\candidates.json" `
  --output "$DemoDir\validation" |
  Set-Content -Encoding UTF8 "$DemoDir\validation-summary.json"
Get-Content "$DemoDir\validation-summary.json"
```

Expected fixture behaviour: one candidate passes; an ineffective fix fails its security check; a build failure skips later checks; and a regression fails normal tests. Actual result files determine the outcome. These fixtures are not newly generated patches or a live firmware test.

```
$summary = Get-Content "$DemoDir\validation-summary.json" -Raw |
  ConvertFrom-Json
$ValidationResult = $summary.evidence
if (-not $ValidationResult -or -not (Test-Path $ValidationResult)) {
  throw "No validation evidence file is available for ranking"
}
python patch-agent\pipeline\score_rank\run_scoring.py `
  --validation "$ValidationResult"
```

Run the blocks in the same terminal. Task 12 ranks eligible candidates; a favourable score cannot outweigh a failed required check. Review exclusions and ties. The TSD describes weights of 0.3 size, 0.3 sensitive-code impact and 0.4 test evidence; the release's missing per-test counts limit the test-evidence contribution.

For another case, replace the paths with its source, candidates, trusted checks and validation profile. Do not mix the Lua fixture profile with an unrelated finding. Standalone Task 9's provider route is optional; use the connected --local-model pipeline for the documented release workflow.

6.5.9 Additional sandbox checks and troubleshooting

```
python patch-agent\datasets\iotgoat\case-001\run_case.py
docker pull aflplusplus/aflplusplus:latest
python patch-agent\examples\run_task11_demo.py --seconds 5
```

The IoTGoat case covers fixture identity, patch application, vulnerable baseline, candidate and ineffective fix. The AFL++ demo is a synthetic harness with a fixed short duration; it does not establish fuzz coverage for every C component. Check image architecture support before interpreting a runtime failure as a patch defect.

The original sandbox smoke test is a Bash script. Run bash patch-agent/sandbox/test_sandbox.sh in the documented Linux/WSL or Bash environment with Docker access. AddressSanitizer detecting the deliberate overflow is an expected positive detection result, not a repaired sample.

Runner files absent	Use the release source package or reviewed runner commit; see release Issue 23.
ENGINE_NOT_SET / wrong engine	Set PATCH_AGENT in this terminal and inspect --stages.
MODEL_UNAVAILABLE	Start Ollama; confirm qwen3-coder is installed and reachable.
MODEL_ANSWER_UNUSABLE	Confirm 32768 context setting and restart Ollama; inspect evidence and retry within the documented limit.
SOURCE_NOT_DOWNLOADED	Run fetch_sources.py with the same cache passed to the pipeline.
Missing or ambiguous file	Return source metadata to the responsible team. Do not guess an affected file.
NO_VALIDATION_PROFILE	Create a reviewed program-specific profile. Credential settings are not generic.
VALIDATION_INCOMPLETE	Inspect Docker, image availability, command logs and timeouts.
No candidate passed	Review why; preserve failures and regenerate or correct the profile only with justification.
Deployment directory exists	Use a new work directory and preserve earlier evidence.
Dashboard action has no effect	Check same-laptop access, matching store/queue, runner state and job logs.
Risk score is nan	Expected: no task produces a risk score yet (Issue 18, closed). Use the test results and the Task 12 ranking.

6.5.10 Recorded results and handover limits

The supplied release.txt records the following outcomes for the 49 findings loaded on 9 October 2026. These are release evidence, not tests rerun while preparing this guide.

Patched, approved by a person and deployed with a passing health check	1
Unsafe candidate blocked by Task 10 before approval	1
Waiting for model review or validation settings	10
Returned before Task 6 for missing or unusable source detail	37
Total findings	49

The same release notes report 10 of 10 integration tests passing on 8 October. Integration checks and patch-success counts answer different questions. Model output varies, and a new run may stop at a different stage.

Open work at release (Issue Register issues 15, 19 to 23 and 27): model answers that are not usable, ordering of two approved patches for one file, deployment to a real device with a manual rollback, dashboard login, a hosted dashboard, committing the release working copy and merging it into main (the ElectraWireless team lead's decision), and moving two reason codes to the Vulnerability Analysis Team. Findings without a fix commit go back to the Vulnerability Analysis Team with a reason code. Issues 14, 16, 17 and 18 are closed. See release.txt and the Issue Register for ownership.

6.5.11 Backup and rerun safely

Stop the runner and other writers before copying the store. Back up the store, validation profiles, relevant evidence, source references and configuration separately. Protect machine-specific configuration and secrets; exclude them from public handover packages. A fresh store creates an independent test, whereas an existing store preserves work in progress.

```
$stamp = Get-Date -Format "yyyyMMdd-HHmmss"
$backup = Join-Path $env:ELLY_WORK "backup-store-$stamp"
Copy-Item -Recurse $env:ORCHESTRATOR_STORE $backup
$backup
```

Do not delete the shared store as a reset procedure. For another independent run, create a separate store directory and point ORCHESTRATOR_STORE to it before intake; update runner configuration separately if using the runner. Preserve the old record histories and review decisions.

6.5.12 Handover checklist

Supply the source package (ProjectID-source.zip, both repositories) and its version, the demo screenshots (ProjectID-demo.pdf), repository/branch revisions where available, readme.txt, this guide, TSD, data.txt, release.txt, record contract, test plan, Issue Register and sanitised demonstration evidence. Include the separate dashboard project and its owner’s setup instructions. List any changes present only in the working copy so the recipient can review and merge them deliberately.

The next developer should first reproduce one supported finding on a separate store, then address missing source metadata and validation profiles. Full firmware/device testing and the planned benchmark evaluation remain separate work; controlled component demonstrations do not replace them.

7 DEPLOYMENT INSTRUCTIONS

There is no cloud, AWS or CI/CD deployment in this release. Everything runs on one pipeline laptop: the patching runner and the Elly Defence dashboard run locally (the dashboard address is localhost), and Task 14 deploys an approved patch only to a test copy of the firmware file, runs a health check on it and puts the original file back by itself if the check fails. Hosting the dashboard is ElectraWireless's decision (Issue 22), deployment to a real device is future work (Issue 20), and merging orchestrator-dev into main is the ElectraWireless team lead's decision (Issue 23).

Set up the laptop first as described in the Development Guide (Section 6). The steps below cover the runner and dashboard route, the queue, the manual route and the acceptance check. ProjectID-demo.pdf shows both routes run from start to end on 23 October 2026.

7.1 RECOMMENDED RUNNER AND DASHBOARD ROUTE

The release runs on one pipeline laptop. The dashboard address is http://localhost:3000/command-center. There is no public pipeline deployment. A hosted dashboard cannot reach the laptop's store and queue; it can display an exported snapshot, but this does not provide remote approval or deployment.

Complete laptop setup and the intake/profile steps in Getting Started before starting the runner. Obtain the integrated dashboard project from its owner. The release has no dashboard login; Accept, Reject and Run again are restricted to requests from the pipeline laptop, and the runner rechecks the finding ID, state and requested action.

```
$config = "orchestrator\runner\runner.config.json"
if (-not (Test-Path $config)) {
  Copy-Item orchestrator\runner\runner.config.example.json $config
}
notepad $config
```

Edit the existing keys in the example rather than inventing a new schema. Set the engine, store, inbox, queue, extracted firmware, source cache, validation profiles and working/evidence folders to this laptop's paths. Check the Python/model settings provided by the example and retain the local-model route. runner.config.json is machine-specific and must not be committed.

```
$Dashboard = Read-Host "Full path to the integrated dashboard folder"
if (-not (Test-Path (Join-Path $Dashboard "package.json"))) {
  throw "Select the dashboard project containing package.json"
}
Push-Location $Dashboard
npm ci
Pop-Location
powershell -ExecutionPolicy Bypass -File `
  orchestrator\runner\start_elly.ps1 -Dashboard "$Dashboard"
```

The supplied launcher checks dependencies and starts the runner and dashboard. Use the release example configuration and launcher together; if either is missing from a clone, use the handed-over release package. The command applies ExecutionPolicy Bypass only to that invocation, not a persistent system-wide policy.

7.2 INBOX AND QUEUE BEHAVIOUR

Place a complete scanner report JSON in the configured inbox. The runner ingests reports, runs actionable findings, retries eligible failures up to three times and processes dashboard requests one job at a time. It does not retry model work while Ollama is unavailable. Do not run a second manual writer against the same store while the runner is active.

The dashboard reads records and writes request files; it does not directly update finding status. The runner queue contains requests/, jobs/, logs/ and state.json. A click is a submitted request, not proof of completion: inspect the job result and finding record. Only an accepted, eligible patch can proceed to deployment.

7.3 MANUAL APPROVAL AND COMPONENT DEPLOYMENT

Use this route when running each step manually. Stop the runner first so there is only one writer. Set the environment from Setup, choose a finding with validated evidence, and verify your own Git name.

```
$FindingId = "1ce6ca579626ca0a"
git config user.name
python orchestrator\release\approve.py --record $FindingId
```

Read the explanation, diff and test outcomes. Check that the change preserves required functionality. Enter y or n when asked. If Task 12 reports a tie, review the candidates; the first one explicitly approved proceeds. No task produces a risk score yet, so the review shows nan; this is expected, not a defect (Issue 18, closed). Decide on the test results and the Task 12 ranking.

```
python orchestrator\release\deploy.py `
  --record $FindingId `
  --work-dir "$env:ELLY_WORK\pipeline_work"
python orchestrator\outtake\status_report.py
```

Deployment checks approval and tested content, writes to a new test-copy directory and runs the configured health check. If the target directory already exists, use a new --work-dir for a new attempt; preserve the earlier evidence. Do not change a candidate after approval and assume the earlier decision still covers it.

DEPLOYED	The component was staged and its configured health check passed.
ROLLED_BACK	The health check failed and the previous component contents were restored. Inspect recovery evidence.
Error or waiting	A dependency, approval, checksum, candidate or profile requirement was not satisfied. This is not a deployment success.

The credential health check checks account-file structure. It does not demonstrate firmware boot, SSH authentication, device reachability or fleet health. Restoring a vulnerable original file is recovery of the previous state, not successful remediation. There is no supported manual rollback command in this release.

7.4 DASHBOARD ACCEPTANCE CHECK

On the pipeline laptop, open Command Center and review a finding marked Needs human approval in Priority Actions. Accept a suitable candidate or reject an unsuitable one. Check the runner job log, record history and regenerated status_report.md. For an accepted case, confirm deployed and a passing health result; for rejection, confirm rejected with no deployment.

Use separate controlled records for acceptance and rejection testing. Do not edit the live store to create a screenshot. Two patches for the same file require serial review and retesting against the changed source; automatic deployment ordering remains unresolved (Issue 19).

8 OTHER CONSIDERATIONS

8.1 SOURCE CODE SUBMITTED FOR AUDIT

ElectraWireless approved, through our mentor after his meeting with Shivam, that the project source code is submitted with the reports for audit. ProjectID-source.zip holds both repositories (Elly-Defense-Orchestration and Elly-Defense, main at commit 1556b09). It leaves out secrets (.env files and the runner configuration), the firmware image and unpacked firmware, pipeline evidence and build output. The only password hashes in it are the public OWASP IoTGoat training ones used by the test fixtures.

8.2 CONFIDENTIALITY AND PRIVACY

Client hosting details are not included in any submitted document. Screenshots hide password hashes, the laptop's folder paths and the computer name. The dashboard runs only on the pipeline laptop and accepts requests only from it, and no firmware or source code is sent to an outside service in the final runs, which use the local Ollama model.

8.3 RESPONSIBLE USE OF AI

Language model output is treated as an untrusted candidate. Every patch is tested in Docker by Task 10 (including the exploit check of Task 11) before anyone sees it, and nothing is deployed without a person's approval (Task 13). The root-account patch that would have emptied the password shows the check working: it was blocked before review (Issue 17, closed). Section 4.10 gives more detail.

8.4 HANDOVER

The code and documents are handed over to the ElectraWireless team lead: the source package, readme.txt, release.txt, data.txt, this report, the Technical Solution Design, the Issue Register, the Product Backlog workbook and ProjectID-demo.pdf. Changes present only in the release working copy are listed in release.txt so they can be reviewed and merged deliberately.

9 REFERENCES

1. ElectraWireless Patching Team. (2026). Autonomous Patching Engine - Technical Design Document (design.md). Assignment 1, Task 2.
2. ElectraWireless Patching Team. (2026). Design Rationale (design_rationale.md). Assignment 1, Task 2.
3. Sharma, K. (2026). Project Elly Defence - Autonomous Assisted Patching Agent: Project Charter V1.3.
4. Vulnerability Analysis Team. (2026). Orchestrator Record Contract (record_contract.md).
5. ElectraWireless Patching Team. (2026). Task 6 Testing Document, Task 12 Scoring, Full Run Guide. Elly-Defense-Orchestration repository.
6. OWASP. (n.d.). IoTGoat: a deliberately insecure firmware for IoT security training. https://github.com/OWASP/IoTGoat
7. NIST. (2017). Juliet Test Suite for C/C++ v1.3. Software Assurance Reference Dataset. https://samate.nist.gov/SARD/
8. Bhandari, G., Naseer, A. and Moonen, L. (2021). CVEfixes: automated collection of vulnerabilities and their fixes from open-source software. PROMISE 2021.
9. National Institute of Standards and Technology. (2022). Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities (NIST SP 800-218). U.S. Department of Commerce. https://csrc.nist.gov/pubs/sp/800/218/final
10. OWASP Foundation. (n.d.). Docker Security Cheat Sheet. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet
11. National Institute of Standards and Technology. (n.d.). National Vulnerability Database (NVD). https://nvd.nist.gov/vuln-metrics/cvss#
12. National Institute of Standards and Technology. (n.d.). Juliet Test Suite for C/C++. Software Assurance Reference Dataset. https://samate.nist.gov/SARD/